

LAYER: 1_Core

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: **/obj/**, **/bin/**, **/*.png, **/*.jpg, **/node_modules/**, **/*.Designer.cs
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

Configuration.Web/Components/_Imports.razor
Configuration.Web/Components/Layout/MainLayout.razor
Configuration.Web/Components/Layout/ReconnectModal.razor
Configuration.Web/Components/NavMenu.razor
Configuration.Web/Components/NotFound.razor
Configuration.Web/Components/Routes.razor
Configuration.Web/Configurator/IWebAppConfigurator.cs
Configuration.Web/Configurator/WebAppConfigurator.cs
Configuration.Web/Configurator/WebScanningExtensions.cs
Configuration.Web/Promatis.Net.Configuration.Web.csproj
Configuration.Web/Properties/launchSettings.json
Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs
Configuration.Web/Services/TabNavigationService/TabItem.cs
Configuration.Web/Services/TabNavigationService/TabNavigationService.cs
Configuration.Web/Services/UiModuleService.cs
Configuration.Web/UiCommandBus/IUiCommandBus.cs
Configuration.Web/UiCommandBus/UiCommandBus.cs
Configuration.Web/WebAppBootstrapper.cs
Configuration/AppBootstrapper.cs
Configuration/AppConfigurator.cs
Configuration/AppInfrastructure.cs
Configuration/IAppConfigurator.cs
Configuration/Promatis.Net.Configuration.csproj
Configuration/ServiceScanningExtensions.cs
Data/Configurations/AuditLogConfiguration.cs
Data/Configurations/ModelBuilderConventions.cs
Data/DbContext/ApplicationDbContext.cs

Data/DBContext/ModelBuilderExtensions.cs
Data/Interceptors/ChangeEntryModel.cs
Data/Interceptors/DatabaseTriggerInterceptor.cs
Data/Interceptors/IUserProvider.cs
Data/Promatis.Net.Data.csproj
Data/Triggers/Castom/AuditTrigger.cs
Data/Triggers/Global/FluentValidationTrigger.cs
Data/Triggers/Global/ReferenceTreeParentTrigger.cs
Data/TriggerService/DatabaseTriggerService.cs
Data/TriggerService/EntityStateChangeEnum.cs
Data/TriggerService/IDatabaseTriggerService.cs
Data/TriggerService/PropertyChangeInfo.cs
Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs
Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgsBase.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs
Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs
Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs
Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs
Data/Validation/GlobalPolymorphicValidator.cs
Domain.Interface/Enums/EnumExtensions.cs
Domain.Interface/Interfaces/IAudit.cs
Domain.Interface/Interfaces/IDomainObject.cs
Domain.Interface/Interfaces/IDomainObjectHasKey.cs
Domain.Interface/Interfaces/IEntityCloner.cs
Domain.Interface/Interfaces/ISoftDeletable.cs
Domain.Interface/Interfaces/ITreeNode.cs
Domain.Interface/Promatis.Net.Domain.Interface.csproj
Domain/AuditLog/AuditLog.cs
Domain/DomainObject/DomainObject.cs
Domain/DomainObject/DomainObjectBase.cs
Domain/DomainObject/DomainObjectValidator.cs
Domain/EntityCloner/JsonEntityCloner.cs
Domain/Promatis.Net.Domain.csproj
Domain/ReferenceBase/ReferenceBase.cs
Domain/ReferenceBase/ReferenceBaseValidator.cs
Domain/ReferenceTreeBase/ReferenceTreeBase.cs
Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs
Promatis.Net.UI/_Imports.razor
Promatis.Net.UI/Components/EditDialog/DialogTab.razor
Promatis.Net.UI/Components/EditDialog/DialogTab.razor.cs
Promatis.Net.UI/Components/EditDialog/EditDialog.razor
Promatis.Net.UI/Components/EditDialog/EditDialog.razor.cs
Promatis.Net.UI/Components/Grid/GridActionContext.cs
Promatis.Net.UI/Components/Grid/GridPage.razor
Promatis.Net.UI/Components/Grid/GridPage.razor.cs
Promatis.Net.UI/Components/Toolbar/IHasSelectedData.cs
Promatis.Net.UI/Components/Toolbar/IHasToolbar.cs
Promatis.Net.UI/Components/Toolbar/ToolbarActionContext.cs
Promatis.Net.UI/Components/Toolbar/ToolbarCustomAction.cs
Promatis.Net.UI/Components/Toolbar/ToolbarPosition.cs
Promatis.Net.UI/Components/Toolbar/UiToolbar.razor
Promatis.Net.UI/Components/Toolbar/UiToolbar.razor.cs
Promatis.Net.UI/Components/Tree/TreeActionContext.cs
Promatis.Net.UI/Components/Tree/TreePage.razor
Promatis.Net.UI/Components/Tree/TreePage.razor.cs
Promatis.Net.UI/Components/Workspace/IWorkspaceActionContext.cs
Promatis.Net.UI/Components/Workspace/WorkspaceActionContext.cs
Promatis.Net.UI/Components/Workspace/WorkspacePage.razor
Promatis.Net.UI/Components/Workspace/WorkspacePage.razor.cs
Promatis.Net.UI/Dashboard.razor
Promatis.Net.UI/Dashboard.razor.css
Promatis.Net.UI/Extensions/MudColorExtensions.cs
Promatis.Net.UI/IUiModule.cs
Promatis.Net.UI/Pages/AuditLog/AuditLogPage.razor
Promatis.Net.UI/Pages/AuditLog/AuditLogPageContext.cs
Promatis.Net.UI/Promatis.Net.UI.csproj
Promatis.Net.UI/Theme/PromatisTheme.cs
Promatis.Net.UI/UiDataBroker.cs
Promatis.Net.UI/UiModule.cs
Promatis.Net.UI/wwwroot/App.css
Service/Contracts/AuditLog/AuditLogSearchRequest.cs
Service/Contracts/AuditLog/PagedResult.cs
Service/Promatis.Net.Service.csproj
Service/Services/AuditLogService/AuditLogService.cs
Service/Services/AuditLogService/IAuditLogService.cs

```

Service/Services/BaseService/BaseService.cs
Service/Services/BaseService/IBaseService.cs
Service/Services/ReferenceService/IReferenceService.cs
Service/Services/ReferenceService/ReferenceService.cs
Service/Services/ReferenceTreeService/IReferenceTreeService.cs
Service/Services/ReferenceTreeService/ReferenceTreeService.cs
Service/Services/TreeService/ITreeService.cs
Service/Services/TreeService/TreeService.cs
Tests/AssemblyInfo.cs
Tests/Domain/DomainLogicTests.cs
Tests/Domain/DomainObjectBaseLogicTests.cs
Tests/Domain/DomainObjectBaseTests.cs
Tests/Domain/DomainObjectExtendedTests.cs
Tests/Domain/ReferenceBaseTests.cs
Tests/Domain/ReferenceTreeTests.cs
Tests/Domain/SoftDeletableTests.cs
Tests/Interceptor/DatabaseTriggerInterceptorTests.cs
Tests/Promatis.Net.ApplicationModel.Tests.csproj
Tests/Service/BaseServiceTests_Crud.cs
Tests/Service/BaseServiceTests.cs
Tests/Service/ReferenceServiceTests_Search.cs
Tests/Service/ReferenceTreeServiceTests.cs
Tests/Service/ServiceIntegrationTests.cs
Tests/Trigger/AuditTriggerTests.cs
Tests/Trigger/DatabaseTriggerServiceTests.cs
Tests/Trigger/FluentValidationTriggerTests.cs
Tests/Trigger/FullTriggerChainTests.cs
Tests/Trigger/ReferenceTreeParentTriggerTests.cs
Tests/Trigger/TreeTriggerChainTests.cs
Tests/Validator/DomainObjectValidatorTests.cs
Tests/Validator/ReferenceBaseValidatorTests.cs
Tests/Validator/ReferenceTreeBaseValidatorTests.cs
</directory_structure>

```

<files>

This section contains the contents of the repository's files.

<file path="Configuration.Web/Components/_Imports.razor">

```

@using System.Net.Http
@using System.Net.Http.Json
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.JSInterop
@using MudBlazor
@using Promatis.Net.Ui
@using Promatis.Net.Configuration.Web
@using Promatis.Net.Configuration.Web.Components
@using Promatis.Net.Configuration.Web.Components.Layout
@using static Microsoft.AspNetCore.Components.Web.RenderMode
</file>

```

<file path="Configuration.Web/Components/Layout/MainLayout.razor">

```

@inherits LayoutComponentBase
@using Promatis.Net.Ui.Theme
@
@inject UiModuleService UiService
@inject TabNavigationService TabService
@implements IDisposable
@
@* A,,!Aô A A' Aô : B 2Dô7D'2C 5CÂ ?D >C$0C"4CT@ D$5CÄK D DC'0C4>CÂ BE <Cô>C4> D 5Cd8CÄ0 *@
<MudThemeProvider Theme="PromatisTheme.DefaultTheme" @bind-IsDarkMode="_isDarkMode" />
<MudPopoverProvider />
@
<MudDialogProvider FullWidth="false"
    MaxWidth="MaxWidth.False"
    CloseButton="false"
    BackdropClick="false"
    NoHeader="false"
    Position="DialogPosition.Center"
    CloseOnEscapeKey="true" />
@
<MudSnackbarProvider />
@
<CascadingValue Value="this">
    <MudLayout>

```

```

    @* A$5D EC00D0 ?C =CT;DÂ ?D 8C´>Cd5C08D0 ¢
    <MudAppBar Color="Color.Default" Elevation="1">D
        @* A,,!A0 A A´ A0 : At0CÂ5C05C0> Color.Primary C00 Color.Default *@D
        <MudIconButton Icon="@Icons.Material.Filled.Menu"D
            Color="Color.Inherit"D
            Edge="Edge.Start"D
            Size="Size.Large"D
            onClick="ToggleDrawer" />D
        <MudStack Spacing="-1">D
            @* AD5DD>C´BC0>CR =C 7C$0C08CR ¢
            <MudText Typo="Typo.h6" Class="ml-2">D
                B$5D BCä2D´9 C0@Cä5C¢B Promatis NetD
            </MudText>D
        </MudStack>D
D
        <MudSpacer />D
D
        <MudIconButton Icon="@(_isDarkMode ? Icons.Material.Filled.LightMode :
Icons.Material.Filled.DarkMode)"D
            Color="Color.Inherit"D
            onClick="ToggleDarkMode" />D
    </MudAppBar>D
D
    @* A00C05C´L CÂ5C0N *@D
    <MudDrawer @bind-Open="_drawerOpen" ClipMode="DrawerClipMode.Always"
Variant="DrawerVariant.Temporary" Overlay="true" OverlayAutoClose="true" Width="220px"
Elevation="1">D
        <NavMenu />D
    </MudDrawer>D
D
    <MudMainContent Class="pt-16 px-4 pb-4 d-flex flex-column"D
        Style="height: calc(100vh - 8px); min-height: calc(100vh - 8px); margin-
top: 10px; display: flex; flex-direction: column; flex-grow: 1; overflow: hidden;">D
D
        @* A,,!A0 A A´ A0 : A4;Cä1C ;DÄ=D´9 ErrorBoundary D41D 0C0 A C$5D EC05C4> D4@Cä2C00. B BC,,;C, fÆW
        @if (TabService.OpenTabs.Count == 0)D
        {D
            <div class="flex-grow-1 overflow-auto">D
                <ErrorBoundary>D
                    <ChildContent>D
                        @BodyD
                    </ChildContent>D
                    <ErrorContent Context="exception">D
                        <MudAlert Severity="Severity.Error" Class="my-4">D
                            A0@Cä8Ct>D,,;C :D 8D$8Dt5D :C 0 CäHC,,1C¢0 C,,=D$5D DCT9D 0: @exception.Message
                        </MudAlert>D
                    </ErrorContent>D
                </ErrorBoundary>D
            </div>D
        }D
        elseD
        {D
            <MudTabs @bind-ActivePanelIndex="TabService.ActiveTabIndex"D
                Position="Position.Top"D
                Color="Color.Default"D
                SliderColor="Color.Primary"D
                Class="mdi-tabs-container w-100 flex-grow-1 d-flex flex-column"D
                Style="height: 100%; max-height: 100%; min-height: 0; display: flex; flex-
direction: column; flex-grow: 1;"D
                TabPanelsClass="pa-0 flex-grow-1 d-flex flex-column overflow-hidden"D
                ApplyEffectsToContainer="true"D
                TabButtonsClass="rounded-t-lg shadow-sm"D
                Outlined="true">D
D
                @for (int i = 0; i < TabService.OpenTabs.Count; i++)D
                {D
                    int localIndex = i;D
                    TabItem tab = TabService.OpenTabs[i];D
D
                    @* A,,!A0 A A´ A0 : AD>C 0C$;CT=C fÆW,0AD$@D4:D$CD 0 CD;D0 AC <Cä3Cä BC 1-C00C05C´0,
                    <MudTabPanel @key="tab" ID="@localIndex" Class="h-100 d-flex flex-column">D
                        <TabContent>D
                            <div class="d-flex align-center gap-2" style="cursor: pointer;">D
                                @if (!string.IsNullOrEmpty(tab.Icon))D
                                {D
                                    <MudIcon Icon="@tab.Icon" Size="Size.Small" />D
                                }
                            }
                        }
                    }
                }
            }
        }
    }

```

```

        }
        <MudText Typo="Typo.body2" Class="font-weight-
medium">@tab.Title</MudText>
        <MudIconButton Icon="@Icons.Material.Filled.Close"
            Size="Size.Small"
            Color="Color.Default"
            Class="pa-0 ms-1"
            Style="width: 16px; height: 16px; min-width:
16px;"
            OnClick="@((e) =>
TabService.CloseTabByIndex(localIndex))" />
        </div>
    </TabContent>
    <ChildContent>
        @* B0BCäB C=>C0BCT9C05D 6CTAD$:Cä 7C =C,,<C 5D" R 2D´ACäBD² 2C;C 4C8 C,
        <div class="pa-2 rounded-b-lg w-100 d-flex flex-column flex-grow-1"
            Style="height: 100%; max-height: 100%; min-height: 0;
overflow: hidden;">
        <ErrorBoundary>
            <ChildContent>
                @* A,,!A0 A A´ A0 : AÄK Cä1Cä@C GC,,2C 5CÄ G-æ 0-46ö× öæVÇB 2 DD8C
                @* C=>D$>D KC´ AC,,;Cä9 C$KD$0C48C$0CTB D BD 0C08DdC C00D,,5C4> CD5
                <div class="d-flex flex-column flex-grow-1 w-100"
style="height: 100%; min-height: 0;">
                    <DynamicComponent Type="@tab.ComponentType" />
                    </div>
                </ChildContent>
                <ErrorContent Context="exception">
                    <MudAlert Severity="Severity.Error" Class="my-4">
                        A0@Cä8Ct>D,,;C :D 8D$8Dt5D :C 0 CäHC,,1C=0 C,,=D$5D DCT9D 0 C$:
                    </MudAlert>
                </ErrorContent>
            </ErrorBoundary>
        </div>
    </ChildContent>
</MudTabPanel>
    }
</MudTabs>
}
</MudMainContent>
</MudLayout>
</CascadingValue>
}
@code {
    private bool _drawerOpen = true;
    private bool _isDarkMode; // A0> D4<Cä;Dt0C08Dä ?D 8C´>Cd5C08CR 7C ?D4AC=0CTBD 0 C" AC$5D$;Cä9 D$5CÄ5D
    protected override void OnInitialized()
    {
        base.OnInitialized();
        TabService.OnTabsChanged += HandleTabsChanged;
        _drawerOpen = false;
    }
    private void ToggleDrawer() => _drawerOpen = !_drawerOpen;
    /// <summary>
    /// A05D 5C;DäGC 5D" DC´0C2 BE <C0>C4> D 5Cd8CÄ0 C, 7C AD$0C$;D05D" ×VEF†VÖU &÷f-FW" <C4=Cä2CT=C0> C05D
    /// </summary>
    private void ToggleDarkMode() => _isDarkMode = !_isDarkMode;
    private void HandleTabsChanged() => InvokeAsync(StateHasChanged);
    public void CloseDrawer()
    {
        if (_drawerOpen)
        {
            _drawerOpen = false;
            StateHasChanged();
        }
    }
    public void Dispose() => TabService.OnTabsChanged -= HandleTabsChanged;

```

```

}
</file>

<file path="Configuration.Web/Components/Layout/ReconnectModal.razor">
<script type="module" src="@Assets["Components/Layout/ReconnectModal.razor.js"]"></script>@
@
<dialog id="components-reconnect-modal" data-nosnippet>@
    <div class="components-reconnect-container">@
        <div class="components-rejoining-animation" aria-hidden="true">@
            <div></div>@
            <div></div>@
        </div>@
        <p class="components-reconnect-first-attempt-visible">@
            Rejoining the server...@
        </p>@
        <p class="components-reconnect-repeated-attempt-visible">@
            Rejoin failed... trying again in <span id="components-seconds-to-next-attempt"></span>
seconds.@
        </p>@
        <p class="components-reconnect-failed-visible">@
            Failed to rejoin.<br />Please retry or reload the page.@
        </p>@
        <button id="components-reconnect-button" class="components-reconnect-failed-visible">@
            Retry@
        </button>@
        <p class="components-pause-visible">@
            The session has been paused by the server.@
        </p>@
        <p class="components-resume-failed-visible">@
            Failed to resume the session.<br />Please retry or reload the page.@
        </p>@
        <button id="components-resume-button" class="components-pause-visible components-resume-
failed-visible">@
            Resume@
        </button>@
    </div>@
</dialog>
</file>

<file path="Configuration.Web/Components/NavMenu.razor">
@using System.Reflection@
@inject UiModuleService UiService@
@inject TabNavigationService TabService@
@inject ComponentRegistry Registry@
@
<MudNavMenu>@
    @{@
        List<(string Title, string Href, string Icon, string? Group)> allItems = UiService.Modules@
            .SelectMany(m => m.GetMenuItems())@
            .ToList();@
    }@
    // AäBD 8D >C$KC$0CT< DÔ;CT<CT=D$K C$5D ECô5C4> D4@Cä2Cô0@
    IEnumerable<(string Title, string Href, string Icon, string? Group)> topLevelItems =
allItems.Where(i => string.IsNullOrEmpty(i.Group));@
    foreach ((string Title, string Href, string Icon, string? Group) item in topLevelItems)@
    {@
        <!-- A,,!Aô A A´ Aô : A$<CTAD$> Href C,,ACô>C´LCtCCT< OnClick CD;Dò >D$:D KD$8Dò 2Cα;C 4Cä: C 5Cr
item.Icon))">@
            @item.Title@
        </MudNavLink>@
    }@
    @
    // A4@D4?Cô8D CCT< DÔ;CT<CT=D$K Cô> Cα0D$5C4>D 8Dô<@
    IOrderedEnumerable<IGrouping<string, (string Title, string Href, string Icon, string?
Group)>> pooledGroups = allItems@
        .Where(i => !string.IsNullOrEmpty(i.Group))@
        .GroupBy(i => i.Group!)@
        .OrderBy(g => g.Key);@
    @
    foreach (IGrouping<string, (string Title, string Href, string Icon, string? Group)> group
in pooledGroups)@
    {@
        <MudNavGroup Title="@group.Key" Icon="@Icons.Material.Filled.Folder" Expanded="false">@
            @foreach ((string Title, string Href, string Icon, string? Group) item in group)@
            {@

```

```

        <!-- A,,!Aô A A´ Aô : B41D 0C0 ‡&VbÂ 4Cä1C 2C´5C0 öä6Æ-6² 00í
        <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title,
item.Href, item.Icon))">D
            @item.TitleD
        </MudNavLink>D
    }D
</MudNavGroup>D
}D
</MudNavMenu>D
D
@code {D
    /// <summary>D
    /// Aô5D 5DT2C BD´2C 5CÂ AC²2Cä7C0>C´ MC²7CT<C0;Dô@ D >CD8D$5C´LD :Cä3Cä <C :CTBC
    /// </summary>D
    [CascadingParameter]D
    protected MainLayout? MainLayoutInstance { get; set; }D
D
    protected override void OnInitialized()D
    {D
        base.OnInitialized();D
        IEnumerable<Assembly> assemblies = UiService.Modules.Select(m =>
m.GetType().Assembly).Distinct();D
        Registry.RegisterModules(assemblies);D
    }D
D
    private void OpenMenuAsTab(string title, string href, string icon)D
    {D
        Type? componentType = Registry.GetComponentTypeByRoute(href);D
D
        if (componentType != null)D
        {D
            // AäBC²@D´2C 5CÂ 2C²;C 4C²CD
            TabService.OpenTab(title, href, icon, componentType);D
D
            // A 2D$>CÄ0D$8Dt5D :C, 7C :D KC$0CT< C >C²>C$>CR <CT=Dâ ?Cä2CT@DR >C²=C ?CäAC´5 C²;C,,:C
            MainLayoutInstance?.CloseDrawer();D
        }D
        elseD
        {D
            System.Diagnostics.Debug.WriteLine($"AôCD$L {href} C05 Ct0D 5C48D BD 8D >C$0C0 2 C²>CÄ?Cä=CT=D$0D
        }D
    }D
}
</file>

<file path="Configuration.Web/Components/NotFound.razor">
@page "/not-found"
@layout MainLayoutD
D
<PageTitle>404 - B BD 0C08Dd0 C05 C00C"4CT=C Âö vUF-FÆSí
D
<MudContainer MaxWidth="MaxWidth.Small" Class="d-flex align-center justify-center" Style="height:
70vh;">D
    <MudPaper Elevation="3" Class="pa-8 text-center" Style="width: 100%;">D
        <MudIcon Icon="@Icons.Material.Filled.SearchOff" Color="Color.Error" Size="Size.Large"
Class="mb-4" />D
        <MudText Typo="Typo.h5" Class="mb-2">AäHC,,1C²0 404</MudText>D
        <MudText Typo="Typo.body2" Class="mud-text-secondary mb-6">D
            A,,7C$8C08D$5, Ct0C0@C HC,,2C 5CÄKC´ 0CD@CTA C05 D CD"5D BC$CCTB, C´8C > D >CäBC$5D$AD$2D4ND"8C´ <C
        </MudText>D
        <MudButton Variant="Variant.Filled" Color="Color.Primary" href="/">D
            A$5D =D4BDÄAD0 =C 3C´0C$=D4ND
        </MudButton>D
    </MudPaper>D
</MudContainer>
</file>

<file path="Configuration.Web/Components/Routes.razor">
@using System.ReflectionD
@inject UiModuleService UiServiceD
D
<Router AppAssembly="@typeof(Routes).Assembly"D
    AdditionalAssemblies="@_moduleAssemblies"D
    NotFoundPage="@typeof(NotFound)">D
    <Found Context="routeData">D

```

```

        <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />
        <FocusOnNavigate RouteData="@routeData" Selector="h1" />
    </Found>
</Router>
}
@code {
    private Assembly[] _moduleAssemblies = [];

    protected override void OnInitialized()
    {
        _moduleAssemblies = UiService.Modules
            .Select(m => m.GetType().Assembly)
            .Distinct()
            .ToArray();

        base.OnInitialized();
    }
}
</file>

<file path="Configuration.Web/Configurator/IWebAppConfigurator.cs">
namespace Promatis.Net.Configuration.Web;
{
    public interface IWebAppConfigurator : IAppConfigurator
    {
        void ConfigureMiddleware(WebApplication app);
    }
}
</file>

<file path="Configuration.Web/Configurator/WebAppConfigurator.cs">
using System.Reflection;
using Microsoft.AspNetCore.Components;
using MudBlazor.Services;

namespace Promatis.Net.Configuration.Web;
{
    /// <summary>
    /// A <C$K$> :Cä=DD8C4CD 0D$>D 4C`O Web-Cô@C,,;Cä6CT=C,,9 C00 C 0Ct5 Blazor C, xVD&Æ |÷"í
    /// </summary>
    public class WebAppConfigurator<TRootComponent> : AppConfigurator, IWebAppConfigurator
    where TRootComponent : IComponent
    {
        public override void ConfigureServices(IServiceCollection services, IConfiguration
configuration)
        {
            // 1. A 0Ct>C$0D0 8C0DD 0D BD CC=BD4@C ,, ABÂ !CT@C$8D K, A$0C`8CD0D$>D K, B$@C,,3C45D K, B :C =CT@ DL
            base.ConfigureServices(services, configuration);

            // 2. UI C,,=DD@C AD$@D4:D$CD 0 (Aô>C,,ACç •V"ÖöGVÆR 2 D 1Cä@C=0DRÂ =C 2C,,3C FC,,0)
            services.AddWebInfrastructure(projectPrefix: "Promatis.");

            // B B "AT A A= A Aä (MDI): B 5C48D BD 8D CCT< C,,=DD@C AD$@D4:D$CD C CÄ=Cä3Cä2C=;C 4CäGC0>C4> C,,
            services.AddSingleton<ComponentRegistry>();
            services.AddScoped<TabNavigationService>();

            // 3. B ?CTFC,,DC,,:C &Æ |÷" ,,~çFW& 7F~fR 6W'fW"•
            services.AddRazorComponents()
                .AddInteractiveServerComponents();

            // 4. B 5C48D BD 0Dd8D0 8 C00D BD >C":C xVD&Æ |÷-
            services.AddMudServices(config =>
            {
                config.SnackbarConfiguration.PositionClass =
MudBlazor.Defaults.Classes.Position.BottomRight;
                config.SnackbarConfiguration.PreventDuplicates = false;
                config.SnackbarConfiguration.NewestOnTop = true;
                config.SnackbarConfiguration.ShowCloseIcon = true;
                config.SnackbarConfiguration.VisibleStateDuration = 5000;
            });

            // 5. B,,8C00 UI-D >C KD$8C•
            services.AddScoped<IUiCommandBus, UiCommandBus>();

            // B 5C48D BD 0Dd8D0 AC,,AD$5CÄ=Cä3Cä 0C=ACTAD >D 0 CD;D0 ?Cä4CD5D 6C=8 C`5C08C$>C4> D 0Ct@CTHCT=C,,O S
            services.AddHttpContextAccessor();
        }
    }
}

```



```

D
/// <summary>D
/// A00D BD >C":C :Cä=C$5C"5D 0 Cä1D 0C >D$:C, ...EE 07C ?D >D >C" „Ö-FFÆWv &R'í
/// </summary>D
public virtual void ConfigureMiddleware(WebApplication app)D
{D
    // A 0Ct>C$KCR Ö-FFÆWv &W2 4C'0 D 0C >D$K Web-C0@C,,;Cä6CT=C,,0D
    app.UseStaticFiles();D
    app.UseAntiforgery();D
D
    // 1. A0>C'CDt0CT< Ct0D 5C48D BD 8D >C$0C0=D'5 UI-D 1Cä@Cª8 C,,7 DI-Cª>C0BCT9C05D 0 DT>D BC
    using IServiceScope scope = app.Services.CreateScope();D
    var uiService = scope.ServiceProvider.GetRequiredService<UiModuleService>();D
D
    Assembly[] moduleAssemblies = uiService.ModulesD
        .Select(m => m.GetType().Assembly)D
        .Distinct()D
        .ToArray();D
D
    // A,, A,,&A,, A' At Bd B0 A B$+ A$ A' AD A£¢ C ?Cä;C00CT< D 5CTAD$@ D >D4BCä2 Cª>CÄ?Cä=CT=D$0CÄ8 C,,7
    var componentRegistry = scope.ServiceProvider.GetRequiredService<ComponentRegistry>();D
    componentRegistry.RegisterModules(moduleAssemblies);D
D
    // 2. B 5C48D BD 8D CCT< Cª>D =CT2Cä9 Cª>CÄ?Cä=CT=D" 8 D :C =C,,@D45CÄ AD$@C =C,,FD² <Cä4D4;CT9 C00 D 5
    app.MapRazorComponents<TRootComponent>()D
        .AddInteractiveServerRenderMode()D
        .AddAdditionalAssemblies(moduleAssemblies);D
D
}D
D
public override void ConfigureApp(IHost app)D
{D
    // A$Kct>C" 1C 7Cä2Cä9 C,,=C,,FC,,0C'8Ct0Dd8C, „0C$BCä@CT3C,,AD$@C FC,,0 D$@C,,3C45D >C" AB GCT@CT7 D :C =
    base.ConfigureApp(app);D
D
    // At4CTADÂ <Cä6C0> CD>C 0C$8D$L C0@Cä2CT@CªC Ct4Cä@Cä2DÄ0 A C,,;C, =C GC ;DÄ=D'9 Seed CD0C0=D'E CD;
D
}
}
</file>

<file path="Configuration.Web/Configurator/WebScanningExtensions.cs">
using Promatis.Net.UI;D
using Promatis.Net.UI.Components.Workspace;D
using System.Reflection;D
D
namespace Promatis.Net.Configuration.Web;D
D
public static class WebInfrastructureExtensionsD
{D
    public static void AddWebInfrastructure(this IServiceCollection services, string projectPrefix
= "Promatis.")D
    {D
        Console.WriteLine("[SCANNER] At0C0CD : C0@Cä3D 5C$0 D 1Cä@Cä: CÄ>C0>C'8D$0 (B$>C'LCª> UI-CÄ>CDCC'8)..
D
        string binPath = AppContext.BaseDirectory;D
D
        // A,,!A0 A A' A0 : A00 D4@Cä2C05 DD0C";Cä2Cä9 D 8D BCT<D² 8D"5CÄ BCä;DÄ:Cä BCR FÆÄÄ :CäBCä@D'5 C00Dt
        string[] assemblyFiles = Directory.GetFiles(binPath, $"{projectPrefix}*.UI.dll",
SearchOption.TopDirectoryOnly);D
D
        foreach (string file in assemblyFiles)D
        {D
            tryD
            {D
                // A0@C,,=D44C,,BCT;DÄ=Cä 7C 3D CCD0CT< C" 76VÖ&Ç"Æö D6öçFW‡BäFVf VçM
                Assembly.LoadFrom(file);D
            }D
            catch (Exception ex)D
            {D
                Console.WriteLine($"[SCANNER] A05 D44C ;CäADÂ 7C 3D CCT8D$L DD0C"; D 1Cä@Cª8 {Path.GetFileName
}D
        }D
D
        // A,,!A0 A A' A0 : BD8C'LD$@D45CÄ FöÖ -âÄ 1CT@D0 AC >D :C, AD$@Cä3Cä A C0@CTDC,,:D >CÄ 8 Cä:Cä=Dt0
        Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()D
            .Where(a => a.FullName != nullD
                && a.FullName.StartsWith(projectPrefix))D

```

```

        && a.GetName().Name!.EndsWith(".UI"))ð
        .Distinct()ð
        .ToArray();ð
ð
Console.WriteLine($"[SCANNER] B :C =C,,@Cä2C =C,,5 Ct0C$5D HCT=Cää C 9CD5C0> {assemblies.Length} Dd5C'
ð
// A B$ AÄ B$ Bt B A / B A4 B "B Bd Bò T'Ô Aä B$ A!B$ A" A´ B$$Aä AÄ+ (AD A A$ AT Aä•
Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò 22Ô:Cä=D$5CðAD$>C" AD$@C =C,,F Cò> CÄ0D :CT@D2 •v÷&·7
var contextTypes = assembliesð
    .SelectMany(s => s.GetTypes())ð
    .Where(t => typeof(IWorkspaceActionContext).IsAssignableFrom(t)ð
        && !t.IsAbstractð
        && !t.IsInterfaceð
        // A,,!Aô A A´ AÔ : Aô@Cä?D4ACð0CT< CäBCð@D´BD´5 generic-D,,0C ;Cä=D² AC <Cä3Cä OCD@C
        && !t.IsGenericTypeDefinition);ð
ð
int registeredContextsCount = 0;ð
foreach (Type type in contextTypes)ð
{ð
    // 1. B 5C48D BD 8D CCT< D 0CÄ :Cä=Cð@CTBCÔKC´ ?D 8Cð;C 4CÔ>C´ :Cä=D$5CðAD" ,,=C ?D 8CÄ5D Ä VF-DÄ
    services.AddTransient(type);ð
    registeredContextsCount++;ð
ð
    Console.WriteLine($"          % % [Að Aô"AT B "] {type.Name}");ð
ð
    // 2. A 5D 5CÄ AD$@Cä3Cä 1C´8Cd0C"HC,,9 C 0Ct>C$KC´ BC,,? (Cô0Cô@C,,<CT@, GridActionContext<AuditLog
    Type? baseType = type.BaseType;ð
    if (baseType != null && baseType.IsGenericType)ð
    {ð
        services.AddTransient(baseType, type);ð
ð
        string genericArgs = string.Join(" ", baseType.GetGenericArguments().Select(a =>
a.Name));ð
        string baseTypeName = baseType.Name.Split('.')[0];ð
        Console.WriteLine($"          % % % AÄ0D :C ¢ ¶& 6UG- Tæ ÖWÓÇ¶vVæW&-4 &w7Óâ""½
    }ð
}ð
Console.WriteLine($"[SCANNER] B4ACô5D,,=Cä @C 7C$5D =D4BCä ·&Vv-7FW&VD6öçFW‡G46÷VçGò :Cä=D$5CðAD$>C" C
ð
Console.WriteLine();ð
ð
Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T'Ô:Cä<Cò>CÔ5CÔBCä2 C, <Cä4D4;CT9 CÔ0C$8C40Dd8C, GCT@C
ð
// A 2D$>CÄ0D$8Dt5D :Cä5 D :C =C,,@Cä2C =C,,5 C, @CT3C,,AD$@C FC,,O C,,=D$5D DCT9D >C" •V"ööGVÆ]
services.Scan(scan => scanð
    .FromAssemblies(assemblies)ð
    .AddClasses(c => c.AssignableTo<IUiModule>().Where(t => !t.IsAbstract))ð
    .AsImplementedInterfaces()ð
    .WithScopedLifetime()ð
);ð
ð
// B 5C48D BD 0Dd8Dò 2C HCT3Cä 0C4@CT3C BCä@C <Cä4D4;CT9ð
services.AddScoped<UiModuleService>();ð
ð
// A´>C48D >C$0CÔ8CR >C =C @D46CT=CÔKDR ?D4=CðBCä2 CÄ5CÔNð
IEnumerable<Type> uiModuleTypes = assembliesð
    .SelectMany(a => a.GetTypes())ð
    .Where(t => typeof(IUiModule).IsAssignableFrom(t) && !t.IsInterface && !t.IsAbstract);ð
ð
foreach (Type type in uiModuleTypes)ð
{ð
    tryð
    {ð
        if (Activator.CreateInstance(type) is IUiModule module)ð
        {ð
            Console.WriteLine($"[SCANNER] UI AÄ>CDCC´L: {module.Name}");ð
            List<(string Title, string Href, string Icon, string? Group)> menuItems =
module.GetMenuItems()?.ToList() ?? new();ð
ð
            if (!menuItems.Any())ð
            {ð
                Console.WriteLine($"          % % [AôCD BCä5 CÄ5CÔN]");ð
                continue;ð
            }ð
            IEnumerable<(string Title, string Href, string Icon, string? Group)> rootItems

```

```

= menuItems.Where(i => string.IsNullOrEmpty(i.Group));
IEnumerable<IGrouping<string?, (string Title, string Href, string Icon, string?
Group)>> groupedItems = menuItems.Where(i => !string.IsNullOrEmpty(i.Group)).GroupBy(i => i.Group);
foreach ((string Title, string Href, string Icon, string? Group) item in
rootItems)
{
    Console.WriteLine($"          % % {item.Title} -> {item.Href}");
}
foreach (IGrouping<string?, (string Title, string Href, string Icon, string?
Group)> group in groupedItems)
{
    Console.WriteLine($"          % % [{group.Key}]");
    List<(string Title, string Href, string Icon, string? Group)> itemsInGroup
= group.ToList();
    for (int i = 0; i < itemsInGroup.Count; i++)
    {
        string prefix = (i == itemsInGroup.Count - 1) ? "          % % % " :
"          % % % ";
        Console.WriteLine($"{prefix} {itemsInGroup[i].Title} ->
{itemsInGroup[i].Href}");
    }
}
}
catch (Exception ex)
{
    Console.WriteLine($"[SCANNER] AäHC,,1C=0 DtBCT=C,,0 CÄ5CÔN CD;Dò BC,,?C .G- Räæ ÖWÓ¢ ¶W,äÖW76 v
");
}
}
Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T'Ô:Cä<Cô>CÔ5CÔBCä2 D4ACô5D,,=Cä 7C 2CT@D,,5CÔ0");
Console.WriteLine();
}
}
</file>

<file path="Configuration.Web/Promatis.Net.Configuration.Web.csproj">
<Project Sdk="Microsoft.NET.Sdk.Web">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />
        <ProjectReference Include="..\Promatis.Net.UI\Promatis.Net.UI.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Configuration.Web/Properties/launchSettings.json">
{
    "profiles": {
        "Promatis.Net.Configuration.Web": {
            "commandName": "Project",
            "launchBrowser": true,
            "environmentVariables": {
                "ASPNETCORE_ENVIRONMENT": "Development"
            },
            "applicationUrl": "https://localhost:56819;http://localhost:56820"
        }
    }
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs">
using System.Reflection;

```

```

using Microsoft.AspNetCore.Components;
{
namespace Promatis.Net.Configuration.Web;
{
public class ComponentRegistry
{
    private readonly Dictionary<string, Type> _routeMap = new(StringComparer.OrdinalIgnoreCase);

    /// <summary>
    /// B :C =C,,@D45D" AC >D :C, <Cä4D4;CT9 C, AD$@Cä8D" :C @D$: URL -> B Æ 72 "C,,? C@>CÄ?Cä=CT=D$0D
    /// </summary>
    public void RegisterModules(IEnumerable<Assembly> assemblies)
    {
        foreach (Assembly assembly in assemblies)
        {
            IEnumerable<Type> componentTypes = assembly.GetTypes()
                .Where(t => typeof(IComponent).IsAssignableFrom(t) && !t.IsAbstract);

            foreach (Type type in componentTypes)
            {
                // A,,ICT< C BD 8C CD" µ&÷WFT GG&- 'WfU0Â 2 C@>D$>D KC' :Cä<C08C'8D CCTBD 0 CD8D 5C@BC,,2C
                IEnumerable<RouteAttribute> routeAttributes =
                    type.GetCustomAttributes<RouteAttribute>();
                foreach (RouteAttribute attr in routeAttributes)
                {
                    _routeMap[attr.Template] = type;
                }
            }
        }
    }

    public Type? GetComponentTypeByRoute(string route)
    {
        return _routeMap.TryGetValue(route, out Type? type) ? type : null;
    }
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabItem.cs">
namespace Promatis.Net.Configuration.Web;
{
public class TabItem
{
    public string Id { get; init; } = string.Empty; // A00D, ‡&Vm
    public string Title { get; init; } = string.Empty;
    public string Icon { get; init; } = string.Empty;
    public Type ComponentType { get; init; } = null!;
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabNavigationService.cs">
namespace Promatis.Net.Configuration.Web;
{
public class TabNavigationService
{
    public List<TabItem> OpenTabs { get; } = new();
    public int ActiveTabIndex { get; set; }
    public event Action? OnTabsChanged;

    public void OpenTab(string title, string href, string icon, Type componentType)
    {
        TabItem? existingTab = OpenTabs.FirstOrDefault(t => t.Id.Equals(href,
            StringComparer.OrdinalIgnoreCase));

        if (existingTab != null)
        {
            ActiveTabIndex = OpenTabs.IndexOf(existingTab);
        }
        else
        {
            var newTab = new TabItem
            {
                Id = href,
                Title = title,
                Icon = icon,
                ComponentType = componentType
            }
        }
    }
}
}

```

```

        };
        OpenTabs.Add(newTab);
        ActiveTabIndex = OpenTabs.Count - 1;
    }

    NotifyStateChanged();
}

/// <summary>
/// A 5Ct>C00D =Că5 Ct0C@D`BC,,5 C$:C`0CD:C, ?Că 5E GC,,AC`>C$>CĂC C,,=CD5C=AD=
/// </summary>
public void CloseTabByIndex(int index)
{
    if (index < 0 || index >= OpenTabs.Count) return;

    OpenTabs.RemoveAt(index);

    // A,,=D$5C`;CT:D$CC ;DĂ=D`9 D 0D GCTB DD>C=CD 0 CÔ0 CăAD$0C$HC,,5D 0 C$:C`0CD:C•
    if (ActiveTabIndex >= OpenTabs.Count)
    {
        ActiveTabIndex = OpenTabs.Count - 1;
    }

    if (ActiveTabIndex < 0 && OpenTabs.Count > 0)
    {
        ActiveTabIndex = 0;
    }

    NotifyStateChanged();
}

private void NotifyStateChanged() => OnTabsChanged?.Invoke();
}
</file>

<file path="Configuration.Web/Services/UiModuleService.cs">
using Promatis.Net.UI;
namespace Promatis.Net.Configuration.Web;
// B 5D 2C,,A, C=D$>D KC' ACă1C,,@C 5D" 2D Q C$>CT4C,,=Cı
public class UiModuleService(IEnumerable<IUiModule> modules)
{
    public IEnumerable<IUiModule> Modules => modules;
}
</file>

<file path="Configuration.Web/UiCommandBus/IUiCommandBus.cs">
using MudBlazor;
namespace Promatis.Net.Configuration.Web;
/// <summary>
/// A,,=D$5D DCT9D HC,,=D² T'ÔACă1D`BC,,9D
/// </summary>
public interface IUiCommandBus
{
    /// <summary>
    /// B 5C48D BD 0Dd8D0 >C @C 1CăBDt8C=0 C$Kct>C$0 DD>D <D² ,,2D`?Că;CÔ0CTBD 0 Cô@C,,=C,,<C ND"8CĂ <Că4D4;CT<,
    /// </summary>
    void RegisterHandler(string commandName, Func<Dictionary<string, object>, Task<DialogResult?>> handler);
}

/// <summary>
/// A$7C 8CĂ>CD5C"AD$2C,,5 CĂ5Cd4D2 <Că4D4;Dô<C, 2D ;CT?D4N (C$KctKC$0CTBD 0 C,,7 MDM)D
/// </summary>
Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object> parameters);
}
</file>

<file path="Configuration.Web/UiCommandBus/UiCommandBus.cs">
using MudBlazor;
namespace Promatis.Net.Configuration.Web;

```

```

/// <summary>Ð
/// B,,8C00 UI-D >C KD$8C•
/// </summary>Ð
public class UiCommandBus : IUiCommandBusÐ
{Ð
    // A0>D$>C¤>C 5Ct>C00D =D´9 D ;Cä2C @DÄ 4C´O D 5C48D BD 0Dd8C, :Cä<C =CB 2 D 0C0BC 9CÄ5 SignalRÐ
    private readonly System.Collections.Concurrent.ConcurrentDictionary<string,
Func<Dictionary<string, object>, Task<DialogResult?>>> _handlers = new();Ð
    Ð
    public void RegisterHandler(string commandName, Func<Dictionary<string, object>,
Task<DialogResult?>> handler)Ð
    {Ð
        _handlers[commandName] = handler ?? throw new ArgumentNullException(nameof(handler));Ð
    }Ð
    Ð
    public async Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object>
parameters)Ð
    {Ð
        if (_handlers.TryGetValue(commandName, out Func<Dictionary<string, object>,
Task<DialogResult?>>? handler))Ð
        {Ð
            return await handler(parameters);Ð
        }Ð
    }Ð
    Ð
    // ATAC´8 CÄ>CDCC´L (C00C0@C,,<CT@, DCA) C05 C0>CD:C´NDt5C0 2 C´0D4=Dt5D 5, D 8D BCT<C =CR CC00CD5D"A
    return null;Ð
}Ð
}
</file>

<file path="Configuration.Web/WebAppBootstrapper.cs">
using Microsoft.AspNetCore.Components;Ð
Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
// A00D ;CT4C08C¢ &ö÷G7G& W"Ä :CäBCä@D´9 D4<CT5D" 7C ?D4AC¤0D$L Web-D 5D 2CT@Ð
public abstract class WebAppBootstrapper<TRootComponent> : AppBootstrapperÐ
    where TRootComponent : IComponentÐ
{Ð
    public override void Run(string[] args)Ð
    {Ð
        // B =Cä2C 7C 3D CCd0CT< DLL (C00 D ;D4GC 9 C0@D0<Cä3Cä 7C ?D4AC¤0)Ð
        // LoadProjectAssemblies ("Promatis.") D46CR 2D´7C$0C00 C" & 6RÄ'VäÄ
        // C0> Ct4CTADÄ <D² 8D ?Cä;DÄ7D45CÄ vV$ Ä-6 F-öä'V-ÆFW"í
    }Ð
    Ð
    // Aä B0 A "AT BÄ Aä¢ CT7 D0BCä3Cä F0Ö -ä =CR CC$8CD8D" <Cä4D4;C, ?D 8 Type.GetTypeÐ
    LoadProjectAssemblies("Promatis.");Ð
    Ð
    WebApplicationBuilder builder = WebApplication.CreateBuilder(args);Ð
    List<IAppConfigurator> configs = GetConfigurators(builder.Configuration).ToList();Ð
    Ð
    foreach (IAppConfigurator cfg in configs)Ð
        cfg.ConfigureServices(builder.Services, builder.Configuration);Ð
    Ð
    WebApplication app = builder.Build();Ð
    Ð
    // A00D BD >C":C Ö-FFÆWv &R <Cä4D4;CT9Ð
    foreach (IWebAppConfigurator cfg in configs.OfType<IWebAppConfigurator>())Ð
    {Ð
        cfg.ConfigureMiddleware(app);Ð
    }Ð
    Ð
    foreach (IAppConfigurator cfg in configs)Ð
        cfg.ConfigureApp(app);Ð
    Ð
    app.Run();Ð
    }Ð
}
</file>

<file path="Configuration/AppBootstrapper.cs">
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.Hosting;Ð
using System.Reflection;Ð
using System.Runtime.Loader;Ð
Ð

```

```

namespace Promatis.Net.Configuration;
{
    public abstract class AppBootstrapper
    {
        public virtual void Run(string[] args)
        {
            // 1. Aö A,, B4 A,, "AT BÄ A / At A4 B4 A Dll (DtBCä1D² G- RävWEG- R 8DR 2C,,4CT;)
            LoadProjectAssemblies("Promatis.");

            HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

            // 2. B4 Aô+A' Aä B A Aô$A,, B4 A "Aä Aä A,, JSON
            List<IAppConfigurator> configurators = GetConfigurators(builder.Configuration).ToList();

            foreach (IAppConfigurator config in configurators)
            {
                config.ConfigureServices(builder.Services, builder.Configuration);
            }

            IHost host = builder.Build();

            foreach (IAppConfigurator config in configurators)
            {
                config.ConfigureApp(host);
            }

            host.Run();
        }

        protected virtual IEnumerable<IAppConfigurator> GetConfigurators(IConfiguration configuration)
        {
            string[] typeNames = configuration.GetSection("LauncherSettings:Modules").Get<string[]>()
                ?? [];

            foreach (string name in typeNames)
            {
                // A,,ICT< D$8Cò 2Cä 2D 5DR 7C 3D CCd5C0=D´E D 1Cä@C=0D]
                Type? type = AppDomain.CurrentDomain.GetAssemblies()
                    .Select(a => a.GetType(name))
                    .FirstOrDefault(t => t != null);

                if (type != null && typeof(IAppConfigurator).IsAssignableFrom(type))
                {
                    if (Activator.CreateInstance(type) is IAppConfigurator instance)
                    {
                        yield return instance;
                    }
                }
                else
                {
                    Console.WriteLine($"[BOOTSTRAPPER][WARN] B$8Cò =CR =C 9CD5C0 8C´8 C05 C$0C´8CD5C0¢ ¶æ ÖW0""½
                }
            }
        }

        protected void LoadProjectAssemblies(string prefix)
        {
            string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
            if (path == null) return;

            foreach (string file in Directory.GetFiles(path, $"{prefix}*.dll"))
            {
                try
                {
                    AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
                }
                catch { /* A,,3C0>D 8D CCT< CäHC,,1C=8 Ct0C4@D47C=8 */ }
            }
        }
    }
}
</file>

```

```

<file path="Configuration/AppConfigurator.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

```

```

using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using System.Text.Json.Serialization;

namespace Promatis.Net.Configuration;

public class AppConfigurator : IAppConfigurator
{
    public virtual void ConfigureServices(IServiceCollection services, IConfiguration configuration)
    {
        // B :C =C,,@D45CÂ 2D Q (C$:C´NDt0Dò VF-EG&-vvW" 2 Cò@Cä5C▯BCR F F •
        services.AddDomainInfrastructure(projectPrefix: "Promatis.");
    }

    // B 5C48D BD 8D CCT< C 0Ct>C$KCR ACT@C$8D Kð
    services.AddScoped<DatabaseTriggerService>();
    services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());
    services.AddScoped<DatabaseTriggerInterceptor>();

    // Aä Bd Af¢ ¥4ôâ =C AD$@Cä9C▯8ð
    services.AddSingleton(new JsonSerializerOptions
    {
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
        DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
        WriteIndented = false
    });

    // B A4 B "B Bd Bò A´ B$Aä AÄ AÔ Aä Aâ A´ AÔ B B #B" Aä!B$ A•
    services.AddSingleton<IEntityCloner, JsonEntityCloner>();
}

public virtual void ConfigureApp(IHost app)
{
    if (app == null) throw new ArgumentNullException(nameof(app));

    // A,,=C,,FC,,0C´8Ct8D CCT< C4;Cä1C ;DÄ=D´9 Service Locator Cò@C, AD$0D BCR ?D 8C´>Cd5C08Dý
    AppInfrastructure.Initialize(app.Services);

    using IServiceScope scope = app.Services.CreateScope();
    scope.ServiceProvider.AutoRegisterTriggers();
}
}
</file>

<file path="Configuration/AppInfrastructure.cs">
namespace Promatis.Net.Configuration;

/// <summary>
/// A4;Cä1C ;DÄ=C 0 D 8D BCT<C00Dò BCäGC▯0 CD>D BD4?C : C,,=DD@C AD$@D4:D$CD =D´< D 5D 2C,,AC < Cò;C BDD>D <D²
/// A,,=C,,FC,,0C´8Ct8D CCTBD 0 Cä4C0>C▯@C BC0> Cò@C, AD$0D BCR ?D 8C´>Cd5C08Dò 8 C,,AC▯;DäGC 5D" @C 7CDCC$0C08CR
/// </summary>
public static class AppInfrastructure
{
    private static IServiceProvider? _provider;
    private static readonly object _lock = new();

    /// <summary>
    /// Read-only CD>D BD4? C¢ 3C´>C 0C´LCÔ>CÄC C▯>C0BCT9C05D C Ct0C$8D 8CÄ>D BCT9.D
    /// </summary>
    public static IServiceProvider Provider
    {
        get
        {
            if (_provider == null)
            {
                throw new InvalidOperationException(
                    "A▯@C,,BC,,GCTAC▯0Dò >D,,8C :C ¢ -æg& 7G'V7GW&R =CR 8C08Dd8C ;C,,7C,,@Cä2C =. " +
                    "B41CT4C,,BCTADÄÄ GD$> CÄ5D$>CB -æg& 7G'V7GW&Rä-æ-F- Æ-¡R,' 2D´7C$0C0 2 Program.cs C,,;C
                );
            }
            return _provider;
        }
    }
}

/// <summary>

```



```

    /// Aô>D$>C¤>C 5Ct>Cô0D =C 0 C,,=C,,FC,,0C`8Ct0Dd8Dò ;Cä:C BCä@C â D´7D´2C 5D$ADò AD$@Cä3Cä >CD8C0 @C 7 C0@
    /// </summary>
    public static void Initialize(IServiceProvider provider)
    {
        if (provider == null) throw new ArgumentNullException(nameof(provider));
        if (_provider != null) return;

        lock (_lock)
        {
            _provider ??= provider;
        }
    }
}
</file>

<file path="Configuration/IAppConfigurator.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
namespace Promatis.Net.Configuration;
public interface IAppConfigurator
{
    void ConfigureServices(IServiceCollection services, IConfiguration configuration);
    void ConfigureApp(IHost app);
}
</file>

<file path="Configuration/Promatis.Net.Configuration.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="Microsoft.Extensions.Configuration.Abstractions" Version="10.0.0" />
        <PackageReference Include="Microsoft.Extensions.DependencyInjection.Abstractions" Version="10.0.0" />
        <PackageReference Include="Microsoft.Extensions.Hosting.Abstractions" Version="10.0.0" />
        <PackageReference Include="Promatis.Net.Data" Version="1.0.0" />
        <PackageReference Include="Promatis.Net.Domain" Version="1.0.0" />
        <PackageReference Include="Promatis.Net.Service" Version="1.0.0" />
        <PackageReference Include="System.Reflection" Version="4.7.0" />
        <PackageReference Include="System.Runtime.Loader" Version="4.7.0" />
    </ItemGroup>
</Project>
</file>

<file path="Configuration/ServiceScanningExtensions.cs">
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using System.Reflection;
using System.Runtime.Loader;
namespace Promatis.Net.Configuration;
public static class ServiceScanningExtensions
{
    public static void AddDomainInfrastructure(this IServiceCollection services, string
        projectPrefix = "Promatis.")
    {

```

```

Console.WriteLine();
Console.WriteLine("[SCANNER] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, 2 DI...");

// 1. Aö A,, B4 A,, "AT BÄ A / At A4 B4 A B Aä Aä D
string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
if (path != null)
{
    foreach (string file in Directory.GetFiles(path, $"{projectPrefix}*.dll"))
    {
        try
        {
            AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
        }
        catch (Exception ex)
        {
            string fileName = Path.GetFileName(file);
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"[SCANNER][ERROR] Aö5 D44C ;CäADÄ 7C 3D CCT8D$L D 1Cä@C C {fileName}");
            Console.ResetColor();
        }
    }
}

// A 5D 5CÄ 2D 5 Ct0C4@D46CT=CÖKCR AC >D :C, A C$0D,,8CÄ AC,,AD$5CÄ=D´< Cö@CTDC,,:D >Cí
Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()
    .Where(a => a.FullName != null && a.FullName.StartsWith(projectPrefix))
    .Distinct()
    .ToArray();

int countBefore = services.Count;

// 2. A B$ AÄ B$ Bt B Aä B A A,, Aä A A,, A, AT A,,!B$ A &A,,/ Bt B Ar 45%UDö-
services.Scan(scan =>
{
    // A´>C=C0C´LCÖ0Dò DD4=C=FC,,0 CD;Dò @CT:D4@D 8C$=Cä9 Cö@Cä2CT@C=8 C$ACT9 Dd5Cö>Dt:C, =C AC´5CD>C$0
    bool IsSubclassOfRawGeneric(Type generic, Type? toCheck)
    {
        while (toCheck != null && toCheck != typeof(object))
        {
            Type cur = toCheck.IsGenericType ? toCheck.GetGenericTypeDefinition() : toCheck;
            if (generic == cur) return true;
            toCheck = toCheck.BaseType;
        }
        return false;
    }

    scan.FromAssemblies(assemblies)
        // A,,!Aö A A´ Aö : B 5C48D BD 0Dd8Dò !AT A$ B A" A Cö>CD4CT@Cd:Cä9 C4;D41Cä:Cä3Cä =C AC´5CD
        .AddClasses(classes => classes.Where(type =>
            !type.IsAbstract &&
            !type.IsGenericTypeDefinition &&
            IsSubclassOfRawGeneric(typeof(BaseService<,,>), type)))
        .AsSelf()
        .AsImplementedInterfaces()
        .WithScopedLifetime()

        // B 5C48D BD 0Dd8Dò A A,, A "Aä Aä (FluentValidation)
        .AddClasses(c => c.AssignableTo(typeof(IValidator<>))
            .Where(t => !t.IsAbstract && !t.IsGenericTypeDefinition))
        .AsImplementedInterfaces()
        .WithTransientLifetime()

        // B 5C48D BD 0Dd8Dò "B A4 AT Aä (Before/After Save)
        .AddClasses(c => c.AssignableToAny(typeof(IBeforeSaveTrigger<>),
            typeof(IAfterSaveTrigger<>)).Where(t => !t.IsAbstract))
        .AsSelfWithInterfaces()
        .WithScopedLifetime();
});

// --- B A4 B "B Bd Bò A´ A A´,Aö A4 Aö A´ Aä B $Aö A4 A$ A´ AD B$ B Bò B ---
// B 5C48D BD 8D CCT< C=C0C C>D$:D KD$KC´ 4Cd5CÖ5D 8C$ä "CT?CT@DÄ ?D 8 Ct0Cö@CäACR •f Æ-F F÷#Ä Dä1Cä9A
// DI-C=Cö>CÖBCT9CÖ5D ääUB 3C @C =D$8D >C$0CÖ=Cä >D$4C AD" =C H Cö>C´8CÄ>D DCÖKC´ 4C,,ACö5D$GCT@
services.AddScoped(typeof(IValidator<>), typeof(GlobalPolymorphicValidator<>));

// 3. A,,!Aö A A´ Aö Aä A, A AT Aö AR Aä A,, Aä A A,, B At#A´,B$ B$ A-
List<ServiceDescriptor> newRegistrations = services.Skip(countBefore).ToList();

```

```

var loggedTypes = new HashSet<string>();
foreach (ServiceDescriptor reg in newRegistrations)
{
    // A 5D 5CÂ @CT0C'LC0KC' BC,,? D 5C ;C,,7C FC,,8 (C" ?D 8Cä@C,,BCTBCR 8Cr -x ÆVÖçF F-öâG- RÂ 8C00Dt5
    Type? implType = reg.ImplementationType ?? reg.ImplementationInstance?.GetType();
    if (implType == null) continue;
    // --- A A A,, A$ A' AD B$ B A" 00Ý
    bool isValidator = reg.ServiceType.IsGenericType &&
        reg.ServiceType.GetGenericTypeDefinition() == typeof(IValidator<>);
    if (isValidator && loggedTypes.Add($"Val:{implType.FullName}"))
    {
        var inheritanceChain = new List<string>();
        Type? currentType = implType;
        while (currentType != null && currentType != typeof(object))
        {
            string typeName = currentType.Name.Contains('.') ? currentType.Name.Split('.')[0] : currentType.Name;
            inheritanceChain.Add(typeName);
            currentType = currentType.BaseType;
            if (typeName == "AbstractValidator") break;
        }
        string chainDisplay = string.Join(" -> ", inheritanceChain);
        Console.WriteLine($"[SCANNER] A$0C'8CD0D$>D ¢ ¶6† -âF-7 Æ -0 ¢ •f Æ-F F÷""½
        continue; // A'>C2 4C'O D0BCä9 Ct0C08D 8 C$KC$5CD5C0Â 8CD5CÂ : D ;CT4D4ND"5C•
    }
    // --- A A A,, B B A,,!Aä (A00D ;CT4C08Cα>C" & 6U6W'f-6R' 00Ý
    // A,,!A0 A A' A0 : B41D 0C0> Cd5D BCα>CR CD ;Cä2C,,5 reg.ServiceType == reg.ImplementationType,
    // D$0C¢ :C : D 5D 2C,,AD² BCT?CT@DÂ @CT3C,,AD$@C,,@D4ND$AD0 ?C @C <C, C'0D A + A,,=D$5D DCT9D
    bool isService = false;
    var serviceChain = new List<string>();
    Type? currentServiceType = implType;
    while (currentServiceType != null && currentServiceType != typeof(object))
    {
        string typeName = currentServiceType.Name.Contains('.') ?
currentServiceType.Name.Split('.')[0] : currentServiceType.Name;
        serviceChain.Add(typeName);
        if (currentServiceType.IsGenericType &&
currentServiceType.GetGenericTypeDefinition() == typeof(BaseService<,,>))
        {
            isService = true;
            break;
        }
        currentServiceType = currentServiceType.BaseType;
    }
    if (isService && loggedTypes.Add($"Srv:{implType.FullName}"))
    {
        string chainDisplay = string.Join(" -> ", serviceChain);
        Console.WriteLine($"[SCANNER] B 5D 2C,,A: {chainDisplay}");
        continue;
    }
    // --- A A A,, B$ A,, A4 B A" 00Ý
    bool isTrigger = implType.GetInterfaces().Any(i => i.IsGenericType &&
        (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
        i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));
    if (isTrigger && loggedTypes.Add($"Tri:{implType.FullName}"))
    {
        Console.WriteLine($"[SCANNER] B$@C,,3C45D ¢ ¶-x ÂG- Ræä ÖW0""½
    }
}
Console.WriteLine("[SCANNER] A 2D$>CÄ0D$8Dt5D :C O D 5C48D BD 0Dd8D0 CD ?CTHC0> Ct0C$5D HCT=C â""½
Console.WriteLine();
}

```

```

public static void AutoRegisterTriggers(this IServiceProvider serviceProvider)
{
    Console.WriteLine("[AUTOREG] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, BD 8C43CT@Cä2 C" F F &
    var triggerService = serviceProvider.GetRequiredService<IDatabaseTriggerService>();
    // 1. A0>C,,AC 2D 5DR :C´0D ACä2 D$@C,,3C45D >C" 2 D 1Cä@C0DR &ö F-2ä-
    IEnumerable<Type> triggerTypes = AppDomain.CurrentDomain.GetAssemblies()
        .Where(a => a.FullName?.StartsWith("Promatis.") == true)
        .SelectMany(s => s.GetTypes())
        .Where(t => t is { IsClass: true, IsAbstract: false } &&
            t.GetInterfaces().Any(i => i.IsGenericType &&
                (i.GetGenericTypeDefinition() ==
                    typeof(IBeforeSaveTrigger<>) ||
                    typeof(IAfterSaveTrigger<>))))
        .ToList();
    MethodInfo? registerMethod =
        typeof(IDatabaseTriggerService).GetMethod(nameof(IDatabaseTriggerService.Register));
    int totalBindings = 0;
    foreach (Type triggerType in triggerTypes)
    {
        // A00DT>CD8CÄ 2D 5 C,,=D$5D DCT9D K D$@C,,3C45D >C"Â :CäBCä@D´5 D 5C ;C,,7D45D" 4C =C0KC´ :C´0D A0
        IEnumerable<Type> interfaces = triggerType.GetInterfaces()
            .Where(i => i.IsGenericType &&
                (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
                    i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));
        foreach (Type @interface in interfaces)
        {
            // A0>C´CDt0CT< D$8C0 AD4IC0>D BC, ...B 8Cr "&Vf÷&U6 fUG&-vvW#ÄCä•
            Type entityType = @interface.GetGenericArguments()[0];
            // Aä?D 5CD5C´0CT< D$8C0 BD 8C43CT@C 4C´0 C@C AC,,2Cä3Cä ;Cä3C
            string triggerKind = @interface.GetGenericTypeDefinition() ==
                typeof(IBeforeSaveTrigger<>)
                    ? "Before"
                    : "After ";
            try
            {
                // A$KctKC$0CT< Register<TEntity, TTrigger>()
                MethodInfo genericRegister = registerMethod!.MakeGenericMethod(entityType,
                    triggerType);
                genericRegister.Invoke(triggerService, null);
                // A´ A4 B A$ A0 AR #B AT(A0 A´ B A$/At A•
                Console.WriteLine($"[AUTOREG] [{triggerKind}] {entityType.Name} <---
                {triggerType.Name}");
                totalBindings++;
            }
            catch (Exception ex)
            {
                Console.ForegroundColor = ConsoleColor.Red;
                Console.WriteLine($"[AUTOREG][ERROR] AäHC,,1C0 C0@C,,2D07C08 {triggerType.Name} C0 ¶VçF-G•
                {ex.InnerException?.Message ?? ex.Message}");
                Console.ResetColor();
            }
        }
    }
    Console.WriteLine($"[AUTOREG] At0C$5D HCT=Cäâ !Cä7CD0C0> C0@C,,2D07Cä:: {totalBindings}");
    Console.WriteLine();
}
}
</file>

```

```

<file path="Data/Configurations/AuditLogConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
namespace Promatis.Net.Data;

```

```
<file path="Data/Configurations/ModelBuilderConventions.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata;
using System.Linq.Expressions;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Data;

public static class ModelBuilderConventions
{
    public static void ApplyGlobalConventions(this ModelBuilder modelBuilder)
    {
        foreach (ImmutableEntityType entityType in modelBuilder.Model.GetEntityTypes())
        {
            Type type = entityType.ClrType;

            // A`8CÄ8D$K CD;Dò AD$@Cä: (255 Cõ> D4<Cä;Dt0CÔ8Dâ•
            foreach (IMutableProperty property in entityType.GetProperties())
            {
                if (property.ClrType == typeof(string))
                {
                    if (property.GetMaxLength() == null)
                        property.SetMaxLength(255);
                }

                // A 2D$>-Cα;DäGC, 4C´O Guid (ValueGeneratedNever, D$0Cϕ :C : Id D >Ct4C 5D$ADò 2 DomainObject)
                if (typeof(IDomainObjectHasKey<Guid>).IsAssignableFrom(type))
                {
                    modelBuilder.Entity(type).Property("Id").ValueGeneratedNever();
                }

                // A B$ -BD A´,B$ C, A$"Aâô Aô AT B 4C´O SoftDelete CälDρ5CρBCä2ð
                if (typeof(ISoftDeletable).IsAssignableFrom(type))
                {
                    ImmutableEntityType? baseType = entityType.BaseType;

                    // B BC 2C,,< DD8C´LD$@ D$>C´LCα> CÔ0 Cα>D 5CÔL C,,5D ðD EC,,8 (C$:C´NDt0Dò 0C AD$@C :D$=D´9 Uni
                    if (baseType == null || !typeof(ISoftDeletable).IsAssignableFrom(baseType.ClrType))
                    {
                        // B4AD$0CÔ>C$:C VW'"f-ÇFW-
                        modelBuilder.SetSoftDeleteFilter(type);

                        // B4AD$0CÔ>C$:C 8CÔ4CT:D θ CÔ0 DeletedAt CD;Dò CD :Cä@CT=C,,O CtθCô@CäACä2ð
                        modelBuilder.Entity(type).HasIndex("DeletedAt");
                    }
                }
            }
        }

        private static void SetSoftDeleteFilter(this ModelBuilder modelBuilder, Type entityType)
        {
            // AD;Dò E BöE , 2C 6CÔ> D BC 2C,,BDÂ DC,,;DÄBD =C :Cä@CT=DÂÂ 4C 6CR 5D ;C, >Cò 0C AD$@C :D$=D´9.D
            // B4AC´>C$8CR —46Æ 72 4CäAD$0D$>Dt=Cái
            if (entityType.IsClass)
            {
                modelBuilder.Entity(entityType).HasQueryFilter(GenerateFilter(entityType));
            }

            private static LambdaExpression GenerateFilter(Type type)
```

```

    {
        ParameterExpression parameter = Expression.Parameter(type, "e");
        // A,,ACô>C`LCtCCT< Property CD;Dò 4CäAD$CCô0 C¢ FVÆWFD B GCT@CT7 C,,=D$5D DCT9D 8C`8 D 2Cä9D BC$>D
        MemberExpression property = Expression.Property(parameter,
nameof(ISoftDeletable.DeletedAt));
        ConstantExpression nullConstant = Expression.Constant(null, typeof(DateTime?));
        BinaryExpression body = Expression.Equal(property, nullConstant);
    }

    return Expression.Lambda(body, parameter);
}
}
</file>

<file path="Data/DbContext/ApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;
namespace Promatis.Net.Data;
public class ApplicationDbContext(
    DbContextOptions options,
    IConfiguration configuration,
    IServiceProvider? serviceProvider = null)
    : DbContext(options)
{
    protected virtual string Schema => "public";
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.HasDefaultSchema(Schema);
        modelBuilder.ApplyModuleConfigurations(this);
        modelBuilder.ApplyGlobalConventions();
        base.OnModelCreating(modelBuilder);
    }

    // A,,!Aô A A` Aô B` A, A AT Aô+A` A A,, Aô"D
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        base.OnConfiguring(optionsBuilder);

        // 1. Aô@Cä2CT@Dô5CÄâ GD$> CÄK C" @C =D$0C"<CRÂ 0 CÔ5 C" ?D >Dd5D ACR <C,,3D 0Dd8C, 4Ä•
        if (!EF.IsDesignTime && serviceProvider != null)
        {
            DatabaseTriggerInterceptor? interceptor = null;

            try
            {
                // Aô>CôKD$:C ¢ D >C CCT< D 0Ct@CTHC,,BDÂ =C ?D OCÄCDâ „5D ;C, ?D >C$0C"4CT@ Cä:C 7C ;D O S
                interceptor = serviceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as
DatabaseTriggerInterceptor;
            }
            catch (InvalidOperationException)
            {
                // Aô>CôKD$:C #¢ D ;C, ?D >C$0C"4CT@ Cæ>D =CT2Cä9 (Root), D >Ct4C 5CÄ 66÷ R 2D CDT=D4N!D
                // BôBCâ 3C @C =D$8D CCTB, DtBCâ 66÷ VBô8CôBCT@Dd5CôBCâ@ D 0Ct@CTHC,,BD O C 5Cr >D,,8C >C¢ > Ro
                var scopeFactory = serviceProvider.GetService(typeof(IServiceScopeFactory)) as
IServiceScopeFactory;

                if (scopeFactory != null)
                {
                    // B >Ct4C 5CÄ 2D 5CÄ5Cô=D4N Cä1C`0D BDÂ 2C,,4C,,<CäAD$8D
                    using IServiceScope scope = scopeFactory.CreateScope();
                    interceptor =
scope.ServiceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as DatabaseTriggerInterceptor;
                }
            }

            // ATAC`8 C,,=D$5D FCT?D$>D CD ?CTHCô> Cô0C"4CT= Cä4Cô8CÄ 8Cr ACô>D >C >C" r ?Cä4C¢;DäGC 5CÄ 5C4>
            if (interceptor != null)
            {
                optionsBuilder.AddInterceptors(interceptor);
            }
        }
    }
}

```

```

    }
}
</file>

```

```

<file path="Data/DbContext/ModelBuilderExtensions.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;
using System.Reflection;
using Microsoft.EntityFrameworkCore.Metadata;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace Promatis.Net.Data;

public static class ModelBuilderExtensions
{
    public static void ApplyModuleConfigurations(this ModelBuilder modelBuilder, DbContext context)
    {
        // 1. B :C =C,,@D45CÂ BCä;DÄ:Câ AC >D :C, 4C =CÔKDR ?D >CT:D$0 Promatis.*.Data
        List<Assembly> dataAssemblies = AppDomain.CurrentDomain.GetAssemblies()
            .Where(a => {
                string? name = a.GetName().Name;
                return name != null &&
                    name.StartsWith("Promatis.") &&
                    name.EndsWith(".Data");
            }).ToList();

        // 2. Aô>C´CDtøCT< D$8CÔK D CD"=CäAD$5C'Â :CäBCä@D´5 Dô2CÔ> Cä1D=0C$;CT=D² 2 D$5C=CD"5CÂ :Cä=D$5C=AD$
        HashSet<Type> contextDbSetTypes = context.GetType()
            .GetProperties(BindingFlags.Public | BindingFlags.Instance)
            .Where(p => p.PropertyType.IsGenericType && p.PropertyType.GetGenericTypeDefinition()
                == typeof(DbSet<>))
            .Select(p => p.PropertyType.GetGenericArguments()[0])
            .ToHashSet();

        foreach (Assembly assembly in dataAssemblies)
        {
            // Aô@C,,<CT=Dô5CÂ :Cä=DD8C4CD ØDd8C, BCä;DÄ:Câ 4C´O D$5DR BC,,?Cä2, C= >D$>D KCR <C BCT@C,,øC´LCÔK (
            modelBuilder.ApplyConfigurationsFromAssembly(assembly, type =>
            {
                if (type is not { IsClass: true, IsAbstract: false, IsGenericTypeDefinition:
                    false })
                    return false;

                // AôøDT>CD8CÂ 8CÔBCT@DD5C"A C= >CÔDC,,3D4@C FC,,8 C, BC,,? D CD"=CäAD$8, C= >D$>D CDâ >CÔø CÔøD B
                Type? configInterface = type.GetInterfaces()
                    .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
                        typeof(IEntityTypeConfiguration<>));

                if (configInterface == null) return false;

                Type entityType = configInterface.GetGenericArguments()[0];

                // A= A,, "A,, 'AT!A= A' $A,, BÄ"B ¢ D ;C, =C AD$@C 8C$øCT<C O D CD"=CäAD$L C 1D BD øC=BCÔø,
                // Cô@Cä2CT@Dô5CÂÄ 5D BDÄ ;C, C CÔ5E ECäBDÄ >CD8CÔ 6C,,2Cä9 CÔøD ;CT4CÔ8C¢ 2 D$5C=CD"5CÂ F%6W
                if (entityType.IsAbstract)
                {
                    bool hasConcreteDerivedInDbSet = contextDbSetTypes.Any(t =>
                        t.IsSubclassOf(entityType));
                    if (!hasConcreteDerivedInDbSet) return false; // Aô@Cä?D4AC=øCT< C= >CÔDC,,3D4@C FC,,N C 1D
                }

                return type.GetConstructor(Type.EmptyTypes) != null;
            });
        }

        // 3. A B$ -A,, Aô B B A$ Aô AR´ B B B$ A$(A,,%» A B "B A= &A,, ø
        IEnumerable<Type> allAbstractEntities = dataAssemblies
            .SelectMany(a => a.GetTypes())
            .Where(t => t is { IsClass: true, IsAbstract: true } && !t.IsGenericTypeDefinition);

        foreach (Type abstractType in allAbstractEntities)
        {
            bool isUsedInCurrentContext = contextDbSetTypes.Any(t => t == abstractType ||
                t.IsSubclassOf(abstractType));

            if (!isUsedInCurrentContext)

```

```

        {
            modelBuilder.Ignore(abstractType);
        }
    }

    // 4. A B$ AÄ B$ Bt B A / AÔ B "B A" A At A´ B A$ AÔ B´% AD B A$,AT A, AÔ AT B A-
    foreach (IMutableEntityType entityType in modelBuilder.Model.GetEntityTypes())
    {
        Type? clrType = entityType.ClrType;
        if (clrType == null) continue;

        // A,,ICT< D 5C ;C,,7C FC,,N C,,=D$5D DCT9D 0 ITreeNode<>
        Type? treeInterface = clrType.GetInterfaces()
            .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(ITreeNode<>));

        if (treeInterface != null)
        {
            Type targetTreeType = treeInterface.GetGenericArguments()[0];

            // AÔ0D BD 0C,,2C 5CÂ BCä;DÄ:Câ =C 1C 7Cä2Cä< D4@Cä2C05 Cä1D=0C$;CT=C,,O D 2Cä9D BC" „GD$>C K
            if (clrType == targetTreeType)
            {
                EntityTypeBuilder builder = modelBuilder.Entity(clrType);

                // 1. AÔ0D BD >C":C AC$0Ct8 B >CD8D$5C´L-Aô>D$>CÄ>C-
                builder.HasOne("Parent")
                    .WithMany("Children")
                    .HasForeignKey("ParentId")
                    .OnDelete(DeleteBehavior.Restrict);

                // 2. A B$ -A,, AD A=!: B >Ct4C 5CÂ 8CÔ4CT:D 4C´O C$=CTHCÔ5C4> C=;DäGC &VçD-M
                // BÔBCâ CD :Cä@C,,B CÔ>C,,AC¢ MC´5CÄ5CÔBCä2 CÔ> ParentId CÔ@C, @C 1CäBCR "Æöö·W 2 D 5D 2C
                builder.HasIndex("ParentId")
                    .HasDatabaseName($"IX_{clrType.Name}_ParentId");
            }
        }
    }
}
</file>

<file path="Data/Interceptors/ChangeEntryModel.cs">
namespace Promatis.Net.Data;

/// <summary>
/// AÄ>CD5C´L DD8C=AC FC,,8 C,,7CÄ5CÔ5CÔ8C´ „BD 0CÔACô>D BCÔKC´ >C JCT:D""Á ?D 5CDAD$0C$;DôND"0Dò ACÔ8CÄ>C¢ ACä
/// CD>CÄ5CÔ=Cä9 D CD"=CäAD$8 C" <Cä<CT=D" 5E 4Cä1C 2C´5CÔ8DòÄ <Cä4C,,DC,,:C FC,,8 C,,;C, CCD0C´5CÔ8Dòí
/// </summary>
internal class ChangeEntryModel
{
    /// <summary>
    /// A,,7CÄ5CÔ0CT<D´9 DÔ:Ct5CÄ?C´OD 4Cä<CT=CÔ>C4> Cä1D=5C=BC „AD4ICÔ>D BC,´í
    /// Aô@C,,2Cä4C,,BD 0 C¢ :Cä=C=CTBCÔ>CÄC D$8CÔC C$=D4BD 8 D$@C,,3C45D >C"í
    /// </summary>
    public object Entity { get; init; } = null!;

    /// <summary>
    /// B$5C=CD"8C´ AD$0D$CD 8Ct<CT=CT=C,,O D CD"=CäAD$8 C" AC,,AD$5CÄ5 (Added, Modified, Deleted, SoftDeleted)
    /// </summary>
    public EntityStateChangeEnum State { get; init; }

    /// <summary>
    /// A=;>C´;CT:Dd8Dò BCäGCTGCÔKDR 8Ct<CT=CT=C,,9 D 2Cä9D BC" „?Cä;CT9) D CD"=CäAD$8.
    /// At0CÔ>C´=Dô5D$ADò 4CT;DÄBCä9 Ct=C GCT=C,,9 "B BC @Cä5 -> AÔ>C$>CR"â C´O CÔ>C$KDR AD4ICÔ>D BCT9 D >CD5
    /// </summary>
    public List<PropertyChangeInfo> Changes { get; init; } = [];

    /// <summary>
    /// A,,4CT=D$8DD8C=0D$>D „8CÄO C,,;C, ;Cä3C,,=) Cô>C´Lct>C$0D$5C´O, D >C$5D HC,,2D,,5C4> CD0CÔ=Cä5 CD5C"AD$2C
    /// </summary>
    public string? ChangedBy { get; init; }

    /// <summary>
    /// AD0D$0 C, 2D 5CÄO DD8C=AC FC,,8 C,,7CÄ5CÔ5CÔ8Dò 2 DD>D <C BCR UD2í
    /// </summary>

```



```

        public DateTime ChangedAt { get; init; }
    }
</file>

<file path="Data/Interceptors/DatabaseTriggerInterceptor.cs">
using System.Collections.Concurrent;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Microsoft.EntityFrameworkCore.Diagnostics;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Data;

/// <summary>
/// A05D 5DT2C Bdt8C$ >C05D 0Dd8C' F$6öçFW†BÂ >C 5D ?CTGC,,2C ND"8C' @C 1CäBD2 $<Dô3C¤>C4> D44C ;CT=C,,0" (Soft
/// C0@CT4C$0D 8D$5C'LC0CDâ 2C ;C,,4C FC,,N C,,7CÄ5C05C08C' 8 D :C$>Ct=Câ5 D$@C,,3C45D =Câ5 D42CT4Cä<C'5C08CR AC,,
/// </summary>
/// <param name="scopeFactory">BD0C @C,,:C 4C'0 D >Ct4C =C,,0 C,,7Cä;C,,@Cä2C =C0KDR >C ;C AD$5C' 2C,,4C,,<CäAD$8
public class DatabaseTriggerInterceptor(IServiceScopeFactory scopeFactory) : SaveChangesInterceptor
{
    /// <summary>
    /// A0>D$>C¤>C 5Ct>C00D =Câ5 DT@C =C,,;C,,ICR 7C DC,,:D 8D >C$0C0=D'E C,,7CÄ5C05C08C' 2 D 0CÄ:C E D$5C¤CD"5C'
    /// A¤;DäG – D4=C,,:C ;DÄ=D'9 C,,4CT=D$8DD8C¤0D$>D MC¤7CT<C0;D0@C F$6öçFW†Bi
    /// </summary>
    private readonly ConcurrentDictionary<Guid, List<ChangeEntryModel>> _capturedChanges = new();

    /// <summary>
    /// A AC,,=DT@Cä=C0KC' ?CT@CTEC$0D$GC,,:, C$KC0>C'=Dô5CÄKC' Aâ DC :D$8Dt5D :Cä3Câ ACäED 0C05C08D0 8Ct<CT=C
    /// AâBC$5Dt0CTB Ct0 C'>C48C¤C Soft Delete, DD8C¤AC FC,,N D =C,,<C¤0 C,,7CÄ5C05C08C' 8 Ct0C0CD : D$@C,,3C45D
    /// </summary>
    public override async ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result, CancellationToken ct =
        default)
    {
        if (eventData.Context == null) return result;

        // B >Ct4C 5CÄ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =C0> Cd8C$>C' 66÷ R 4C'0 C0@CT4CäBC$@C ICT=C,,0 C0@Cä1C'
        using IServiceScope scope = scopeFactory.CreateScope();
        var triggerService = scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();

        string? userName = await GetUserNameInternalAsync(scope.ServiceProvider);

        // 1. A0@CT4C$0D 8D$5C'LC00D0 >C @C 1CäBC¤0 Soft Delete (AÄ0C4:Câ5 D44C ;CT=C,,5)
        // A00DT>CD8CÄ 2D 5 D CD"=CäAD$8 D 8C0BCT@DD5C"ACä< ISoftDeletable, D2 :CäBCä@D'E D BC BD4A "B44C ;C
        IEnumerable<EntityEntry<ISoftDeletable>> entries =
            eventData.Context.ChangeTracker.Entries<ISoftDeletable>()
                .Where(e => e.State == EntityState.Deleted);

        foreach (EntityEntry<ISoftDeletable> entry in entries)
        {
            // A0>CD<CT=Dô5CÄ DC,,7C,,GCTAC¤>CR CCD0C'5C08CR =C >C =Cä2C'5C08CR 4C =C0KDR 2 A 0
            entry.State = EntityState.Modified;
            entry.Entity.DeletedAt = DateTime.UtcNow;
            entry.Entity.DeletedBy = userName;
        }

        // A0@C,,=D44C,,BCT;DÄ=Câ 7C AD$0C$;Dô5CÄ Tb 6÷&R ?CT@CTADt8D$0D$L CD5D 5C$> C,,7CÄ5C05C08C' ?CäAC'5 C0>
        eventData.Context.ChangeTracker.DetectChanges();

        // 2. At0DT2C B C,,7CÄ5C05C08C' ,,ACäED 0C00CTB C00D$8C$=D'5 D$8C0K CD0C0=D'E object? CD;D0 BCTAD$>C".
        List<ChangeEntryModel> captured = CaptureChanges(eventData.Context, userName);

        // A0@C,,2D07D'2C 5CÄ :Cä;C'5C¤FC,,N C,,7CÄ5C05C08C' : C¤>C0:D 5D$=Cä<D2 MC¤7CT<C0;D0@D2 :Cä=D$5C¤AD$0 C
        _capturedChanges[eventData.Context.Id.InstanceId] = captured;

        // 3. A$0C'8CD0Dd8D0 „&Vf÷&U6 fr' GCT@CT7 Cd8C$>C' G&-vvW%6W'f-6]
        // At0C0CD :C 5D" 4Cä<CT=C0KCR ?D 0C$8C'0 C, 2C ;C,,4C BCä@D² ?CT@CT4 D$5CÄÄ :C : CD0C0=D'5 C0>C00CDCD
        foreach (ChangeEntryModel item in captured)
            await triggerService.ValidateAsync(item.Entity, item.State, item.Changes,
                eventData.Context);

        return result;
    }
}
/// <summary>

```

```

/// A AC,,=DT@Cä=CÖKC' ?CT@CTEC$0D$GC,,:, C$KCô>C'=Dô5CÄKC' Aä!A' D4ACô5D,,=Cä3Cä ACäED 0C05C08Dò 8Ct<CT=C
/// AäBC$5Dt0CTB Ct0 D42CT4Cä<C'5C08CR ?Cä4C08D GC,,:Cä2 C, BD 8C43CT@Cä2 Cö>D B-Cä1D 0C >D$:C, ,,DC 7C 6
/// </summary>ð
public override async ValueTask<int> SavedChangesAsync(ð
    SaveChangesCompletedEventData eventData, int result, CancellationToken ct = default)ð
{ð
    // A,,7C$;CT:C 5CÄ 7C EC$0Dt5C0=D'5 C,,7CÄ5C05C08Dò 8 D @C 7D2 0D$>CÄ0D =Cä CCD0C'0CT< C,,E C,,7 D ;Cä2C
    if (eventData.Context != null &&
        _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out List<ChangeEntryModel>?
entries))ð
    {ð
        // B >Ct4C 5CÄ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =C0> Cd8C$>C' 66÷ R 4C'0 DD0CtK Saved (C$KCô>C'=Dô5
        using IServiceScope scope = scopeFactory.CreateScope();ð
        var triggerService =
scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();ð
ð
        // A AC,,=DT@Cä=C0> D42CT4Cä<C'0CT< D 8D BCT<D2 >C 8Ct<CT=CT=C,,ODR ,,=C ?D 8CÄ5D Ä 4C'0 Cä1C0>C$;C
        foreach (ChangeEntryModel item in entries)ð
            await triggerService.NotifyAsync(item.Entity, item.State, item.Changes,
item.ChangedBy, item.ChangedAt);ð
        }ð
        return result;ð
    }ð
}ð
ð
/// <summary>ð
/// Aô5D 5DT2C BDt8C$ AC >Dò ?D 8 D >DT@C =CT=C,,8 C,,7CÄ5C05C08C'â C @C =D$8D CCTB CäGC,,AD$:D2 :DôHC 7C
/// C0@CT4CäBC$@C IC 0 D4BCTGC=8 C00CÄ0D$8 C0@C, ?C 4CT=C,,8 D$@C =Ct0C=FC,,9.ð
/// </summary>ð
public override Task SaveChangesFailedAsync(DbContextErrorEventData eventData,
CancellationToken ct = default)ð
{ð
    if (eventData.Context != null)ð
        _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out _);ð
ð
    return base.SaveChangesFailedAsync(eventData, ct);ð
}ð
ð
/// <summary>ð
/// A$=D4BD 5C0=C,,9 CÄ5D$>CB 0C00C'8Ct0 D$@CT:CT@C 8Ct<CT=CT=C,,9 EF Core. ð
/// A$KDt8D ;Dô5D" 4CT;DÄBD2 ,,AD$0D KCR0=Cä2D'5 Ct=C GCT=C,,0 C0>C'5C'' 8 DD>D <C,,@D45D" <Cä4CT;C, 8Ct<CT=
/// </summary>ð
/// <param name="context">B$5C=CD"8C' :Cä=D$5C=AD" 1C 7D² 4C =C0KDRäÄ÷ & Óí
/// <param name="userName">A,,<Dò ?Cä;DÄ7Cä2C BCT;DòÄ ACä2CT@D,,8C$HCT3Cä >C05D 0Dd8DäâÄ÷ & Óí
/// <returns>B ?C,,ACä: D BD CC=BD4@C,,@Cä2C =C0KDR <Cä4CT;CT9 C,,7CÄ5C05C08C' Ç6VR 7&Vc0$6† ævTVÇG'"ÖöFVÄ"ö
private List<ChangeEntryModel> CaptureChanges(DbContext context, string? userName)ð
{ð
    // BD8C'LD$@D45CÄ BCä;DÄ:Cä AD4IC0>D BC, 2 D >D BCä0C08DòE AD>C 0C$;CT=C,,0, A,,7CÄ5C05C08Dò 8C'8 B44C
    List<EntityEntry> entries = context.ChangeTracker.Entries()ð
        .Where(e => e.State is EntityState.Added or EntityState.Modified or EntityState.Deleted)ð
        .Where(e => !e.Entity.GetType().Name.Contains("AuditLog"))ð
        .ToList();ð
ð
    var result = new List<ChangeEntryModel>();ð
ð
    foreach (EntityEntry e in entries)ð
    {ð
        EntityStateChangeEnum state = MapState(e.State);ð
ð
        // B ?CTFC,,DC,,GC0KC' <C ?C08C03 D >D BCä0C08Dò 4C'0 "CÄ0C4:Cä CCD0C'5C0=D'E" Ct0C08D 5C•
        if (e.Entity is ISoftDeletable soft)ð
        {ð
            PropertyEntry prop = e.Property(nameof(ISoftDeletable.DeletedAt));ð
            if (prop.IsModified && soft.DeletedAt != null) state =
EntityStateChangeEnum.SoftDeleted;ð
        }ð
ð
        List<PropertyChangeInfo> changes = new();ð
ð
        // B FCT=C @C,,9 1: B CD"=CäAD$L CD>C 0C$;CT=C ä D 5 D$5C=CD"8CR 7C00Dt5C08Dò AC$>C"AD$2 D GC,,BC
        if (state == EntityStateChangeEnum.Added)ð
        {ð
            changes = e.Propertiesð
                .Select(p => new PropertyChangeInfoð
                {ð
                    PropertyName = p.Metadata.Name,ð
                    OriginalValue = null,ð

```

```

        CurrentValue = p.CurrentValue
    }).ToList();
}
// B FCT=C @C,,9 2: B CD"=CäAD$L C,,7CÄ5C05C00 C,,;C, <D03C> D44C ;CT=C â D´GC,,AC´OCT< D 0Ct=C,,FD2
else if (state is EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted)
{
    PropertyValues originalValues = e.OriginalValues;

    foreach (PropertyEntry p in e.Properties.Where(p => p.IsModified))
    {
        string propertyName = p.Metadata.Name;
        object? originalValue = originalValues[propertyName];
        object? currentValue = p.CurrentValue;

        // BD8C=AC,,@D45CÄ 8Ct<CT=CT=C,,5 D$>C´LC> CTAC´8 Ct=C GCT=C,,0 CD5C"AD$2C,,BCT;DÄ=Câ @C 7C´
        if (!Equals(originalValue, currentValue))
        {
            changes.Add(new PropertyChangeInfo
            {
                PropertyName = propertyName,
                OriginalValue = originalValue,
                CurrentValue = currentValue
            });
        }
    }

    // AD0Cd5 CTAC´8 C>C´;CT:Dd8D0 BCäGCTGC0KDR 8Ct<CT=CT=C,,9 C0>C´5C´ ,,6† ævW2´ ?D4AD$0
    // C,,7-Ct0 D 0C0BC 9CÄ01C,,=CD8C03C &Æ |÷"Ä <D² B B A$ Aâ 4Cä1C 2C´OCT< D CD"=CäAD$L C" AC08D
    // CTAC´8 DD0C @C,,:C 7C DC,,:D 8D >C$0C´0 D >D BCäOC08CR 0öF-f-VB -D$> C40D 0C0BC,,@D45D" R 4C
    if (state == EntityStateChangeEnum.Modified && changes.Count == 0)
    {
        // B >Ct4C 5CÄ DC,,:D$8C$=D4N Ct0C08D L C,,7CÄ5C05C08D0 4C´O CD>CÄ5C0=D´E D$@C,,3C45D >C"Ä
        // DtBCä1D² =CR ;Cä<C BDÄ 2C ;C,,4C FC,,N, C0> D >DT@C =C,,BDÄ 8CÄ?D4;DÄA CD;D0 T•
        changes.Add(new PropertyChangeInfo
        {
            PropertyName = "Id",
            OriginalValue = e.Property("Id").OriginalValue,
            CurrentValue = e.Property("Id").CurrentValue
        });
    }

    result.Add(new ChangeEntryModel
    {
        Entity = e.Entity,
        State = state,
        ChangedBy = userName,
        ChangedAt = DateTime.UtcNow,
        Changes = changes
    });
}

return result;
}

/// <summary>
/// AÄ0C0?C,,=C2 2C0CD$@CT=C08DR ACäAD$>D0=C,,9 EntityFramework State C$> C$=D4BD 5C0=C,,9 CD>CÄ5C0=D´9 C05D
/// </summary>
private EntityStateChangeEnum MapState(EntityState state) => state switch
{
    EntityState.Added => EntityStateChangeEnum.Added,
    EntityState.Deleted => EntityStateChangeEnum.Deleted,
    _ => EntityStateChangeEnum.Modified
};

/// <summary>
/// A 5Ct>C00D =Cä5 C0>C´CDt5C08CR 8CÄ5C08 D$5C=CD"5C4> C0>C´LCt>C$0D$5C´O C,,7 C>C0BCT:D BC 0C$BCä@C,,7C
/// </summary>
/// <param name="provider">A´>C=0C´LC0KC´ ?D >C$0C"4CT@ D ;D46C ,,D´ 6öçF -æw"´äÄ÷ & Óí
private async Task<string?> GetUserNameInternalAsync(IServiceProvider provider)
{
    try
    {
        var userProvider = provider.GetService<IUserProvider>();
        if (userProvider != null)
            return await userProvider.GetCurrentUserNameAsync();
    }
}

```

```

        }
        catch
        {
            // ignored
        }
    }
    return "System";
}
}
</file>

<file path="Data/Interceptors/IUserProvider.cs">
namespace Promatis.Net.Data;
{
    public interface IUserProvider
    {
        // AÄ5D$>CB <Cä6CTB C KD$L C AC,,=DT@Cä=CÔKCÂÂ 5D ;C, ?Cä;D4GCT=C,,5 C,,<CT=C, BD 5C CCTB Ct0Cô@CäAC
        Task<string?> GetCurrentUserAsync();
    }
}
</file>

<file path="Data/Promatis.Net.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">
{
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
{
    <ItemGroup>
        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />
        <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.1" />
    </ItemGroup>
{
    <ItemGroup>
        <ProjectReference Include="..\Domain\Promatis.Net.Domain.csproj" />
    </ItemGroup>
}
    "Ä-FVÔw&÷W í
    "<AssemblyAttribute Include="System.Runtime.CompilerServices.InternalsVisibleTo">
    "Äö & ÖWFw# ä &öÖ F-2äæWBä Ä-6 F-öäÖöFVÄâFW7G3Äöö & ÖWFw# ä Ä Öö CÄO D$5D BCä2Cä3Cä ?D >CT:D$0 -->
    "</AssemblyAttribute>
    "Äö-FVÔw&÷W í
}
}
</Project>
</file>

<file path="Data/Triggers/Castom/AuditTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
{
    namespace Promatis.Net.Data;
{
    public class AuditTrigger(
        IServiceScopeFactory scopeFactory,
        JsonSerializerOptions jsonOptions)
        : IAfterSaveTrigger<DomainObject>
    {
        public async Task HandleAsync(EntityChangedEventArgs<DomainObject> args)
        {
            // AäGC,,IC 5CÂ @C =D$0C"<-D$8Cö >D" 2Cä7CÄ>Cd=Cä9 CD8CÔ0CÄ8Dt5D :Cä9 Cö@Cä:D 8-Cä1Cä;CäGC=8 EF Core (
            Type entityType = args.Entity.GetType();
            if (entityType.Namespace == "Castle.Proxies" && entityType.BaseType != null)
            {
                entityType = entityType.BaseType;
            }
        }
    }
}
    // 1. BD8C´LD$@: C´>C48D CCT< D$>C´LC= D$5 D CD"=CäAD$8, C= >D$>D KCR ?Cä<CTGCT=D² " VF-M
    if (entityType.GetInterfaces().All(i => i.Name != nameof(IAudit)))
    {
        return;
    }
}

```

```

    }
    // 2. BD>D <C,,@D45CÂ 4C =CÔKCR 4C´O JSOND
    object changesToSerialize = args.State switch
    {
        EntityStateChangeEnum.Added => args.Changes.Select(c => new
        {
            c.PropertyName,
            NewValue = c.CurrentValue
        }),
        EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted =>
args.Changes.Select(c => new
        {
            c.PropertyName,
            OldValue = c.OriginalValue,
            NewValue = c.CurrentValue
        }),
        EntityStateChangeEnum.Deleted => new { Message = "Aâ1D¤5C¤B D44C ;CT= C¤>C´=CâAD$LDâ" ÔÍ
        _ => args.Changes
    };

    // 3. B >Ct4C 5CÂ 7C ?C,,ADÂ ;Câ3C
    var auditLog = new AuditLog
    {
        Id = Guid.NewGuid(),
        EntityName = entityType.Name, // A,,!Aô A A´ Aô : Aô8D,,5CÂ GC,,AD$>CR 8CÃO C¤;C AD 0 Câ@C4AD$@D4:D
        EntityId = args.Entity.Id,
        Action = args.State.ToString(),
        ChangedAt = args.ChangedAt,
        ChangedBy = args.ChangedBy,
        ChangesJson = JsonSerializer.Serialize(changesToSerialize, jsonOptions)
    };

    // 4. A,,!Aô A A´ Aô : A 5Ct>Cô0D =Câ5 D >DT@C =CT=C,,5 C" AB GCT@CT7 C$KCD5C´5CÔ=D´9, C40D 0CÔBC,,@Câ
    using IServiceScope scope = scopeFactory.CreateScope();

    // A,,ICT< DD0C @C,,:D2 1C 7Câ2Câ3Câ :Câ=D$5C¤AD$0 Dt5D 5Cr ?D >C$0C"4CT@ C´>C¤0C´LCÔ>C4> Scope
    var factory = scope.ServiceProvider.GetService<IDbContextFactory<ApplicationDbContext>>();

    if (factory != null)
    {
        await using ApplicationDbContext context = await factory.CreateDbContextAsync();
        context.Set<AuditLog>().Add(auditLog);
        await context.SaveChangesAsync();
    }
    else
    {
        Console.WriteLine("[AuditTrigger][Error] Aô5 D44C ;CâADÂ =C 9D$8 IDbContextFactory<ApplicationDbC
    }
}
</file>

<file path="Data/Triggers/Global/FluentValidationTrigger.cs">
using FluentValidation;
using FluentValidation.Results;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Data;
public class FluentValidationTrigger(IServiceScopeFactory scopeFactory) // A,,!Aô A A´ Aô : A,,=Cd5C¤BC,,@D45CÂ
    : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>
{
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)
    {
        Type entityType = args.Entity.GetType();
        Type validatorType = typeof(IValidator<>).MakeGenericType(entityType);

        // A,,!Aô A A´ Aô : B >Ct4C 5CÂ 8Ct>C´8D >C$0CÔ=D4N, C 5Ct>Cô0D =D4N Câ1C´0D BDÂ 2C,,4C,,<CâAD$8 (Scope
        // BÔBCâ 3C @C =D$8D CCTB, DtBCâ •6W'f-6U &÷f-FW" 1D44CTB "Cd8C$KCÂ" 8 D4ACô5D,,=Câ @C 7D 5D,,8D" =C H
        using IServiceScope scope = scopeFactory.CreateScope();

```

```

// At0C0@C HC,,2C 5CÂ 2C ;C,,4C BCä@ C,,7 D 2CT6CT3CâÂ 3C @C =D$8D >C$0C0=Câ 6C,,2Cä3Câ 66÷ R0?D >C$0C"4C
if (scope.ServiceProvider.GetService(validatorType) is IValidator validator)ð
{ð
    var context = new ValidationContext<object>(args.Entity);ð
    ValidationResult result = await validator.ValidateAsync(context);ð
ð
    if (!result.IsValid)ð
    {ð
        args.Cancel = true;ð
        // BD>D <C,,@D45CÂ :D 0D 8C$>CRÂ GC,,AD$>CR ACä>C ICT=C,,5 Cä1 CäHC,,1C=0DR 4C´O UIð
        args.ErrorMessage = string.Join(" | ", result.Errors.Select(e => e.ErrorMessage));ð
    }ð
}ð
}
</file>

<file path="Data/Triggers/Global/ReferenceTreeParentTrigger.cs">
using Microsoft.EntityFrameworkCore;ð
using Microsoft.EntityFrameworkCore.Internal;ð
using Microsoft.EntityFrameworkCore.Metadata;ð
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using System.Reflection;ð
ð
namespace Promatis.Net.Data;ð
ð
/// <summary>ð
/// B4=C,,2CT@D 0C´LC0KC´ 8C0DD 0D BD CC=BD4@C0KC´ BD 8C43CT@ CD;Dò ?D >C$5D :C, FCT;CäAD$=CäAD$8 C,,5D 0D EC,,G
/// C, AC=2Cä7C0>C´ 7C IC,,BD² >D" FC,,:C´8Dt5D :C,,E Ct0C$8D 8CÄ>D BCT9 C05D 5CB ACäED 0C05C08CT< C" !B4 ABí
/// </summary>ð
public class ReferenceTreeParentTrigger : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>>ð
{ð
    private static readonly MethodInfo DbContextSetMethod = typeof(DbContext)ð
        .GetMethods(BindingFlags.Public | BindingFlags.Instance)ð
        .First(m => m.Name == nameof(DbContext.Set) && m.IsGenericMethod &&
m.GetParameters().Length == 0);ð
ð
    private static readonly MethodInfo AsNoTrackingMethod =
typeof(EntityFrameworkQueryableExtensions)ð
        .GetMethods(BindingFlags.Public | BindingFlags.Static)ð
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.AsNoTracking) &&
m.IsGenericMethod);ð
ð
    private static readonly MethodInfo IgnoreQueryFiltersMethod =
typeof(EntityFrameworkQueryableExtensions)ð
        .GetMethods(BindingFlags.Public | BindingFlags.Static)ð
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.IgnoreQueryFilters) &&
m.IsGenericMethod);ð
ð
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)ð
    {ð
        // A0 Aä!B$ AR AT(AT A,, : Bt5D 5Cr @CTDC´5C=AC,,N C0@Cä2CT@D05CÄÂ 5D BDÂ ;C, C Cä1D=5C=BC AC$>C"AD$2
        // ATAC´8 D 2Cä9D BC$0 C05D" r MD$>D" >C JCT:D" =CR 4CT@CT2CâÂ <D² ?D >D BCâ 2D´ECä4C,,<!ð
        PropertyInfo? parentIdProp = args.Entity.GetType().GetProperty("ParentId");ð
        if (parentIdProp == null) return;ð
ð
        // Bt8D$0CT< Ct=C GCT=C,,0 D 2Cä9D BC" 4C,,=C <C,,GCTAC=8D
        Guid entityId = args.Entity.Id;ð
        Guid? parentId = (Guid?)parentIdProp.GetValue(args.Entity);ð
ð
        PropertyInfo? deletedAtProp = args.Entity.GetType().GetProperty("DeletedAt");ð
ð
        // 1. At0D"8D$0 CäB C0@D0<Cä3Câ AC <CäFC,,BC,,@Cä2C =C,,0D
        if (parentId.HasValue && parentId == entityId)ð
        {ð
            args.Cancel = true;ð
            args.ErrorMessage = "Aä1D=5C=B C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.";ð
            return;ð
        }ð
ð
        if (parentId.HasValue && parentId.Value != Guid.Empty)ð
        {ð
            // A,,ICT< D >CD8D$5C´O C" 6† ævUG& 6¶W" AD 5CD8 C´NC KDR >C JCT:D$>C"Â C C=ðD$>D KDR ACä2C00CD00CT
            object? localParent = args.Context.ChangeTracker.Entries()ð
                .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
parentId.Value)?.Entity;ð

```

```

    bool exists;
    bool isDeleted;

    if (localParent != null)
    {
        exists = true;
        var localDeletedAt = (DateTime?)deletedAtProp?.GetValue(localParent);
        isDeleted = localDeletedAt != null;
    }
    else
    {
        // A$0D,,0 Că@C,,3C,,=C ;DÄ=C 0 D 5DD;CT:D 8Dò 4C´O C$KDt8D ;CT=C,,0 Cα>D =Dò 8 Query C" Tb 6÷&]
        IEntityType? entityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());
        Type rootClrType = entityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();
        object? rawSet =
DbContextSetMethod.MakeGenericMethod(rootClrType).Invoke(args.Context, null);
        object? noTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(rootClrType).Invoke(null, [rawSet]);
        object? ignoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(rootClrType).Invoke(null, [noTrackingQuery]);
        IQueryable<object> query = ((IQueryable)ignoreFiltersQuery!).Cast<object>();
        // A,,ICT< Că1Dα5CαB Cò> CD8C00CÄ8Dt5D :Că<D2 AC$>C"AD$2D2 $-B" !B4 AM
        object? dbParent = await query.FirstOrDefaultAsync(x => EF.Property<Guid>(x, "Id")
== parentId.Value);
        exists = dbParent != null;
        var dbDeletedAt = dbParent != null ? (DateTime?)deletedAtProp?.GetValue(dbParent) :
null;
        isDeleted = dbDeletedAt != null;
    }

    // 2. A0@Că2CT@Cα0 DD8Ct8Dt5D :Că3Că AD4ICTAD$2Că2C =C,,0 D >CD8D$5C´LD :Că3Că CCt;C
    if (!exists)
    {
        args.Cancel = true;
        args.ErrorMessage = $"B4:C 7C =C0KC' @Că4C,,BCT;DÄACα8C' >C JCT:D" ,,"Cϕ . &VçD-G0' =CR =C 9CD
        return;
    }

    // 3. A0@Că2CT@Cα0 D BC BD4AC <D03Cα>C4> D44C ;CT=C,,0D
    if (isDeleted)
    {
        args.Cancel = true;
        args.ErrorMessage = "A05C´Lct0 C00Ct=C GC,,BDÂ @Că4C,,BCT;CT< D44C ;CT=C0KC' >C JCT:D"â#½
        return;
    }

    // 4. A4 B4 Aă A / At B" B$ Aă" Bd Aα Aă (A -> B -> C -> A)
    Guid? currentCheckId = parentId;
    var visitedIds = new HashSet<Guid>();

    while (currentCheckId.HasValue && currentCheckId.Value != Guid.Empty)
    {
        if (currentCheckId == entityId)
        {
            args.Cancel = true;
            args.ErrorMessage = "Bd8Cα;C,,GCTACα0Dò 7C 2C,,AC,,<CăAD$L: C05C´Lct0 C05D 5CĂ5D BC,,BDÂ @Că4
            return;
        }

        if (!visitedIds.Add(currentCheckId.Value))
            break;

        // A,,ICT< D0;CT<CT=D" FCT?CăGCα8 D =C GC ;C 2 ChangeTracker
        object? nextLocalElement = args.Context.ChangeTracker.Entries()
            .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
currentCheckId.Value)?.Entity;

        if (nextLocalElement != null)
    }

```

```

        currentCheckId = (Guid?)parentIdProp.GetValue(nextLocalElement);
    }
    else
    {
        Guid? id = currentCheckId;

        IEntityType? loopEntityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());
        Type loopRootClrType = loopEntityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();
        object? loopRawSet =
DbContextSetMethod.MakeGenericMethod(loopRootClrType).Invoke(args.Context, null);
        object? loopNoTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopRawSet]);
        object? loopIgnoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopNoTrackingQuery]);
        IQueryable<object> loopQuery =
((IQueryable)loopIgnoreFiltersQuery!).Cast<object>();
        object? parentNodeObj = await loopQuery.FirstOrDefaultAsync(x =>
EF.Property<Guid>(x, "Id") == id.Value);
        currentCheckId = parentNodeObj != null ?
(Guid?)parentIdProp.GetValue(parentNodeObj) : null;
    }
}
}
}
</file>

```

```

<file path="Data/TriggerService/DatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using System.Collections.Concurrent;
namespace Promatis.Net.Data;
/// <summary>
/// B 5C ;C,,7C FC,,0 Dd5C0BD 0C'LC0>C4> CD8D ?CTBDt5D 0 D$@C,,3C45D >C"â #C0@C 2C'0CTB D 5C48D BD 0Dd8CT9 CD>CÄ
/// C, :Cä>D 4C,,=C,,@D45D" DC 7D² 2C ;C,,4C FC,,8 (CD> D >DT@C =CT=C,,0) C, CC$5CD>CÄ;CT=C,,0 (C0>D ;CR ACäED 0C05
/// </summary>
/// <param name="serviceProvider">A :D$CC ;DÄ=D'9 C¤>C0BCT:D B Ct0C$8D 8CÄ>D BCT9 (DI Container) D$5C¤CD"5C4>
public class DatabaseTriggerService(IServiceProvider serviceProvider) : IDatabaseTriggerService
{
    /// <summary>
    /// A4;Cä1C ;DÄ=Cä5 D >C KD$8CRÄ CC$5CD>CÄ;D0ND"5CR AC,,AD$5CÄC Cä1 D4AC05D,,=Cä< C¤>CÄ<C,,BCR AD4IC0>D BC,
    /// A,,AC0>C' LctCCTBD 0 CD;D0 @CT0C¤BC,,2C0>C4> Cä1C0>C$;CT=C,,0 C,,=D$5D DCT9D 0 (MudBlazor) C,,;C, AC¤2Cä7C0
    /// </summary>
    public static event Action<EntityStateChangeEnum, object>? OnEntityCommitted;

    /// <summary>
    /// B 5CTAD$@ C0>CD?C,,Adt8C¤>C"Â 2D' ?Cä;C00DäIC,,ED 0 AD D >DT@C =CT=C,,0 C,,7CÄ5C05C08C' ,, $C 7C C ;C,,4C
    /// AD5C'5C40D" ?D 8C08CÄ0CTB C @C4CCÄ5C0BD² ACä1D'BC,,0 C, 0C¤BD40C'LC0KC' 66÷ VB 6W'f-6U &÷f-FW"i
    /// </summary>
    private static readonly Dictionary<Type, List<Func<EntityCancelArgsBase, IServiceProvider,
Task>>> BeforeSubscribers = new();

    /// <summary>
    /// B 5CTAD$@ C0>CD?C,,Adt8C¤>C"Â 2D' ?Cä;C00DäIC,,ED 0 A0 B AR CD ?CTHC0>C4> D >DT@C =CT=C,,0 C,,7CÄ5C05C08C
    /// </summary>
    private static readonly Dictionary<Type, List<Func<EntityChangedArgsBase, IServiceProvider,
Task>>> AfterSubscribers = new();

    /// <summary>
    /// A0>D$>C¤>C 5Ct>C00D =D'9 C¤MD, 8CT@C @DT8C, BC,, ?Cä2. A,,AC¤;DäGC 5D" ?Cä2D$>D =Cä5 C,,AC0>C' Lct>C$0C08C
    /// C0@C, >C0@CT4CT;CT=C,,8 C 0Ct>C$KDR :C'0D ACä2 C, 8C0BCT@DD5C"ACä2 Cä1D 0C 0D$KC$0CT<D'E D CD"=CäAD$5C
    /// </summary>
    private static readonly ConcurrentDictionary<Type, List<Type>> HierarchyCache = new();

    /// <summary>
    /// B 5C48D BD 8D CCTB CD>CÄ5C0=D'9 D$@C,,3C45D 4C'0 C¤>C0:D 5D$=Cä3Cä BC,, ?C AD4IC0>D BC,i
    /// A$KcTkc$0CTBD 0 C 2D$>CÄ0D$8Dt5D :C, ?D 8 D :C =C,,@Cä2C =C,,8 D 1Cä@Cä: C$> C$@CT<D0 8C08Dd8C ;C,,7C FC
    /// </summary>

```



```

/// <typeparam name="TEntity">B$8Cò 4Cä<CT=CÔ>C' AD4ICÔ>D BC,ãÄ÷G— W & Óí
/// <typeparam name="TTrigger">B$8Cò :C' 0D AC ÔBD 8C43CT@C Ä @CT0C'8CtCDäICT3Câ ;Cä3C,, :D2 >C @C 1CäBC=8.<
public void Register<TEntity, TTrigger>()
    where TEntity : class
    where TTrigger : class
{
    Type entityType = typeof(TEntity);
    Type triggerType = typeof(TTrigger);

    // Aô@Cä2CT@Dô5CÄÄ :C :C,,5 C,,=D$5D DCT9D K Cd8Ct=CT=CÔ>C4> Dd8C=;C @CT0C'8CtCCTB CD0CÔ=D'9 D$@C,,3C45
    bool isBefore = typeof(IBeforeSaveTrigger<TEntity>).IsAssignableFrom(triggerType);
    bool isAfter = typeof(IAfterSaveTrigger<TEntity>).IsAssignableFrom(triggerType);

    if (isBefore)
    {
        // BD>D <C,,@D45CÄ >D$;Cä6CT=CÔCDä ;Dô<C 4D2â Cä=D$5C'=CT@ 'sp' Cô5D 5CD0CTBD O C" <Cä<CT=D" 2D'7
        // DtBCä ?D 5CD>D$2D 0D"0CTB D4BCTGC=C CD>C'3Cä6C,,2D4IC,,E Cä1D=5C=BCä2 (Captive Dependency).
        AddSubscriber(BeforeSubscribers, entityType, async (argsBase, sp) =>
        {
            // B 0Ct@CTHC 5CÄ MC=7CT<Cô;Dô@ D$@C,,3C45D 0 C,,7 C :D$CC ;DÄ=Cä3Cä 66÷ R BCT:D4ICT3Cä 7C ?D >
            if (sp.GetRequiredService<TTrigger>() is IBeforeSaveTrigger<TEntity> handler)
            {
                // Aä1Cä@C GC,,2C 5CÄ 1C 7Cä2D'5 C @C4CCÄ5CÔBD² 2 D BD >C4> D$8Cô8Ct8D >C$0CÔ=D'9 C= >CÔBCT
                var typedArgs = new EntityCancelEventArgs<TEntity>(
                    (TEntity)argsBase.Entity,
                    argsBase.State,
                    argsBase.Changes,
                    argsBase.Context);

                await handler.HandleAsync(typedArgs);

                // A$>Ct2D 0D"0CT< D 5CtCC'LD$0D$K C$KCô>C'=CT=C,,O D$@C,,3C45D 0 Cä1D 0D$=Cä 2 C 0Ct>C$KC'
                argsBase.Cancel = typedArgs.Cancel;
                argsBase.ErrorMessage = typedArgs.ErrorMessage;
                argsBase.Handled = typedArgs.Handled;
            }
        });
    }

    if (isAfter)
    {
        AddSubscriber(AfterSubscribers, entityType, async (argsBase, sp) =>
        {
            if (sp.GetRequiredService<TTrigger>() is IAfterSaveTrigger<TEntity> handler)
            {
                var typedArgs = new EntityChangedEventArgs<TEntity>(
                    (TEntity)argsBase.Entity,
                    argsBase.State,
                    argsBase.Changes,
                    argsBase.ChangedBy,
                    argsBase.ChangedAt);

                await handler.HandleAsync(typedArgs);
                argsBase.Handled = typedArgs.Handled;
            }
        });
    }

    // ATAC'8 C=;C AD =CR @CT0C'8CtCCTB CÔ8 Cä4C,,= C= >CÔBD 0C=B — DÔBCä :D 8D$8Dt5D :C O CäHC,,1C=0 C= >CÔ
    if (!isBefore && !isAfter)
    {
        throw new InvalidOperationException(
            $"A=;C AD .G&-vvw%G— Ræœ ŌWŌ =CR @CT0C'8CtCCTB IBeforeSaveTrigger C,,;C, " gFW%6 fUG&-vvw" 4C
        );
    }

    /// <summary>
    /// A$KCô>C'=Dô5D" AC=2Cä7CÔCDä 2C ;C,,4C FC,,N C,,7CÄ5CÔ0CT<Cä9 D CD"=CäAD$8 Cô5D 5CB 5E >D$?D 0C$:Cä9 C"
    /// Aä?D 0D,,8C$0CTB C$ADä FCT?CäGC=C CÔ0D ;CT4Cä2C =C,,O D CD"=CäAD$8. Aô@CT@D'2C 5D" >Cô5D 0Dd8Dä ?D 8 CÔ
    /// </summary>
    public async Task ValidateAsync(object entity, EntityStateChangeEnum state,
        List<PropertyChangeInfo> changes, DbContext context)
    {
        var args = new EntityCancelArgsBase(entity, state, changes, context);

        // Aä1DT>CD8CÄ 8CT@C @DT8Dä BC,,?Cä2 (D 0CÄ>C' AD4ICÔ>D BC,Ä 1C 7Cä2D'E C=;C AD >C" 8 C,,=D$5D DCT9D >C

```

```

        foreach (Type type in GetTypesHierarchy(entity.GetType()))
        {
            if (BeforeSubscribers.TryGetValue(type, out List<Func<EntityCancelArgsBase,
IServiceProvider, Task>>? actions))
            {
                foreach (Func<EntityCancelArgsBase, IServiceProvider, Task> action in actions)
                {
                    // A05D 5CD0CT< C'>C0C'LC0KC' 6W'f-6U &+f-FW"Â A C0>D$>D KCÂ 1D'; D >Ct4C = CD0C0=D'9 D0
                    await action(args, serviceProvider);
                }
            }
        }

        // ATAC'8 D$@C,,3C45D 2D'AD$0C$8C^2 DC'0C2 6 æ6VÂ r =CT<CT4C'5C0=Câ ?D 5D KC$0CT< D$@C =Ct
        if (args.Cancel) throw new OperationCanceledException(args.ErrorMessage);
    }
}

/// <summary>
/// B 0D AD';C 5D" CC$5CD>CÄ;CT=C,,0 Cäl D4AC05D,,=Cä< D >DT@C =CT=C,,8 C,,7CÄ5C05C08C' 2 C 0CtC CD0C0=D'E.D
/// A0>CD4CT@Cd8C$0CTB C0D :C 4C0KC' ?CT@CTEC$0D" ,,DC'0C2 † æFÆVB' 8 C0CC ;C,,:D45D" ACälD'BC,,5 C" 3C'>C
/// </summary>
public async Task NotifyAsync(object entity, EntityStateChangeEnum state,
List<PropertyChangeInfo> changes, string? user, DateTime at)
{
    var args = new EntityChangedEventArgs(entity, state, changes, user, at);

    foreach (Type type in GetTypesHierarchy(entity.GetType()))
    {
        if (AfterSubscribers.TryGetValue(type, out List<Func<EntityChangedEventArgs,
IServiceProvider, Task>>? actions))
        {
            foreach (Func<EntityChangedEventArgs, IServiceProvider, Task> action in actions)
            {
                await action(args, serviceProvider);

                // ATAC'8 Cä4C,,= C,,7 D$@C,,3C45D >C" 2D'AD$0C$8C^2 † æFÆVB 0 G'VRÂ >C @C 1CäBC00 Dd5C0>Dt:C
                if (args.Handled) return;
            }
        }
    }

    // A0CC ;C,,:C FC,,0 C" AD$0D$8Dt5D :C,,9 D02CT=D" 4C'0 D 5C :D$8C$=Cä3Cä >C =Cä2C'5C08D0 :Cä<C0>C05C0BC
    OnEntityCommitted?.Invoke(state, entity);
}

/// <summary>
/// A$AC0>CÄ>C40D$5C'LC0KC' <CTBCä4 CD;D0 1CT7Cä?C AC0>C4> CD>C 0C$;CT=C,,0 C0>CD?C,,ADt8C0>C" 2 D ;Cä2C @C
/// </summary>
private void AddSubscriber<TArgs>(Dictionary<Type, List<Func<TArgs, IServiceProvider, Task>>?
dict, Type type, Func<TArgs, IServiceProvider, Task> action)
{
    if (!dict.TryGetValue(type, out List<Func<TArgs, IServiceProvider, Task>>? list))
    {
        list = new List<Func<TArgs, IServiceProvider, Task>>();
        dict[type] = list;
    }
    list.Add(action);
}

/// <summary>
/// A,,7C$;CT:C 5D" ?Cä;C0CDä 8CT@C @DT8Dä BC,,?Cä2 CD;D0 CC0Ct0C0=Cä3Cä >C JCT:D$0 (C$:C'NDt0D0 1C 7Cä2D'
/// B 5CtCC'LD$0D$K C0MD,,8D CDäBD 0 CD;D0 >C 5D ?CTGCT=C,,0 C$KD >C0>C' ?D >C,,7C$>CD8D$5C'LC0>D BC, ?Cä4 C
/// </summary>
private IEnumerable<Type> GetTypesHierarchy(Type type) =>
{
    HierarchyCache.GetOrAdd(type, t => {
        var types = new List<Type>();
        // B 5C0CD AC,,2C0> C0>CD=C,,<C 5CÄAD0 2C$5D E C0> CD@CT2D2 =C AC'5CD>C$0C08D0 :C'0D ACä2, C,,AC0;Dä
        for (Type? c = t; c != null && c != typeof(object); c = c.BaseType) types.Add(c);
        // AD>C 0C$;D05CÄ 2D 5 D 5C ;C,,7Cä2C =C0KCR 8C0BCT@DD5C"AD^2 8 D41C,,@C 5CÄ 4D41C'8C0D$K0
        return types.Concat(t.GetInterfaces()).Distinct().ToList();
    });
}

/// <summary>
/// B @CäGC0KC' AC @CäA D BC BC,,GCTAC08DR @CT3C,,AD$@C FC,,9. D
/// A,,AC0>C'LCtCCTBD 0 C0@CT8CÄCD"5D BC$5C0=Cä 2 Unit-D$5D BC E CD;D0 8Ct>C'ODd8C, ?D >C4>C0>C" BCTAD$>C"
/// </summary>

```

```

        internal static void ClearInternalRegistrations()
        {
            BeforeSubscribers.Clear();
            AfterSubscribers.Clear();
            HierarchyCache.Clear();
        }
    }
</file>

<file path="Data/TriggerService/EntityStateChangeEnum.cs">
namespace Promatis.Net.Data;
{
    public enum EntityStateChangeEnum
    {
        Added,
        Modified,
        Deleted,
        SoftDeleted
    }
}
</file>

<file path="Data/TriggerService/IDatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public interface IDatabaseTriggerService
        {
            // AA5D$>CB 4C'0 C00D BD >C":C, AC$0Ct5C' $!D4IC0>D BDÂ0"D 8C43CT@"
            void Register<TEntity, TTrigger>()
                where TEntity : class
                where TTrigger : class;
        }

        // AA5D$>CDK C$KC0>C'=CT=C,,0 (C$KctKC$0DäBD 0 C,,=D$5D FCT?D$>D >CÂ•
        Task ValidateAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo>
changes, DbContext context);
        Task NotifyAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo> changes,
string? user, DateTime at);
    }
}
</file>

<file path="Data/TriggerService/PropertyChangeInfo.cs">
namespace Promatis.Net.Data;
{
    public class PropertyChangeInfo
    {
        public string PropertyName { get; set; } = string.Empty;
        public object? OriginalValue { get; set; }
        public object? CurrentValue { get; set; }
    }
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public class EntityCancelArgsBase(
            object entity,
            EntityStateChangeEnum state,
            List<PropertyChangeInfo> changes,
            DbContext context) : EntityEventArgsBase(entity, state, changes)
        {
            public DbContext Context { get; } = context;
            public bool Cancel { get; set; }
            public string? ErrorMessage { get; set; }
        }
    }
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public class EntityCancelEventArgs<T>(
            T entity,

```

```

        EntityStateChangeEnum state, Ø
        List<PropertyChangeInfo> changes, Ø
        DbContext context) Ø
        : EntityCancelArgsBase(entity!, state, changes, context) where T : class Ø
    { Ø
        public new T Entity => (T)base.Entity; Ø
    }
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs">
namespace Promatis.Net.Data; Ø
Ø
// A ØCt>C$KC' :C'ØD A CD;Dò 2CÔCD$@CT=CÔ5C4> C,,ACô>C'LCt>C$ØCÔ8Dò 2 D 5D 2C,,AC]
public class EntityChangedEventArgs(Ø
    object entity, Ø
    EntityStateChangeEnum state, Ø
    List<PropertyChangeInfo> changes, Ø
    string? changedBy, Ø
    DateTime changedAt) : EntityEventArgsBase(entity, state, changes) // <-- AØØD ;CT4D45CÂ 7CD5D LØ
{ Ø
    public string? ChangedBy { get; } = changedBy; Ø
    public DateTime ChangedAt { get; } = changedAt; Ø
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs">
namespace Promatis.Net.Data; Ø
Ø
// A$ØD, BC,,?C,,7C,,@Câ2C =CÔKC' :C'ØD A CD;Dò BD 8C43CT@Câ2Ø
public class EntityChangedEventArgs<T>(Ø
    T entity, EntityStateChangeEnum state, Ø
    List<PropertyChangeInfo> changes, Ø
    string? changedBy, Ø
    DateTime changedAt) Ø
    : EntityChangedEventArgsBase(entity!, state, changes, changedBy, changedAt) where T : class Ø
{ Ø
    public new T Entity => (T)base.Entity; Ø
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs">
namespace Promatis.Net.Data; Ø
Ø
public abstract class EventArgsBase(Ø
    object entity, Ø
    EntityStateChangeEnum state, Ø
    List<PropertyChangeInfo> changes) : EventArgs Ø
{ Ø
    public object Entity { get; } = entity; Ø
    public EntityStateChangeEnum State { get; } = state; Ø
    public List<PropertyChangeInfo> Changes { get; } = changes; Ø
    public bool Handled { get; set; } Ø
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs">
namespace Promatis.Net.Data; Ø
Ø
// AD;Dò ?D ØC$8C² $ Aâ!A' " D >DT@C =CT=C,,Ø (D42CT4Câ<C'5CÔ8DòðHC,,=C •
public interface IAfterSaveTrigger<T> where T : class Ø
{ Ø
    Task HandleAsync(EntityChangedEventArgs<T> args); Ø
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs">
namespace Promatis.Net.Data; Ø
Ø
// AD;Dò ?D ØC$8C² $ Aâ" ACâED ØCÔ5CÔ8Dò ,,2C ;C,,4C FC,,Ø)Ø
public interface IBeforeSaveTrigger<T> where T : class Ø
{ Ø
    Task HandleAsync(EntityCancelEventArgs<T> args); Ø
}
</file>

<file path="Data/Validation/GlobalPolymorphicValidator.cs">

```

```

using FluentValidation;
using FluentValidation.Results;
using Microsoft.Extensions.DependencyInjection;

namespace Promatis.Net.Domain;

public class GlobalPolymorphicValidator<T> : AbstractValidator<T> where T : class
{
    private readonly IServiceProvider _serviceProvider;

    public GlobalPolymorphicValidator(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider ?? throw new
ArgumentNullException(nameof(serviceProvider));
    }

    protected override bool PreValidate(ValidationContext<T> context, ValidationResult result)
    {
        if (context.InstanceToValidate == null) return true;

        // 1. A0>C'CDt0CT< D 5C ;DÄ=D'9 D$8C0 >C JCT:D$0 C" @C =D$0C"<CR „=C ?D 8CÄ5D Ä FW 'FÖVÇEVæ-B 8C'8 A
        Type realType = context.InstanceToValidate.GetType();

        // ATAC'8 D$8C0 ACä2C00CD0CTB D B „B.CRâ MD$> C0>C05Dt=D'9 C0;C AD Ä 0 C05 C 0Ct>C$KC' 0C AD$@C :D$=
        // D$> C„AC00D$L CD>Dt5D =C„9 C$0C'8CD0D$>D =CR =D46C0>, DtBCä1D² =CR CC"BC, 2 C 5D :Cä=CTGC0CDâ @CT
        if (realType == typeof(T)) return true;

        // 2. B BD >C„< D$8C0 8C0BCT@DD5C"AC 4C'0 D$>Dt5Dt=Cä3Cä 2C ;C„4C BCä@C „=C ?D 8CÄ5D Ä •f Æ-F F÷#ÄF
        Type specificValidatorType = typeof(IValidator<>).MakeGenericType(realType);

        // 3. At0C0@C HC„2C 5CÄ 8Cr D' B C$0C'8CD0D$>D K CD;D0 MD$>C4> C0>C0:D 5D$=Cä3Cä BC„?C
        // B0BCä 2C 6C0>, D$0C0 :C : CD;D0 >CD=Cä3Cä BC„?C <Cä6CTB C KD$L Ct0D 5C48D BD 8D >C$0C0> C05D :Cä;
        List<IValidator> specificValidators =
_serviceProvider.GetServices(specificValidatorType).Cast<IValidator>().ToList();

        if (specificValidators.Any())
        {
            // B >Ct4C 5CÄ =CR04Cd5C05D 8C0 :Cä=D$5C0AD" 2C ;C„4C FC„8 FluentValidation CD;D0 ?CT@CT4C GC, 4C
            ValidationContext<object>? newContext =
ValidationContext<object>.CreateWithOptions(context.InstanceToValidate, options =>
            {
                // A„!A0 A A' A0 : A0C0>C08Dt=D'9 CÄ5D$>CB fÇVVÇEf Æ-F F-öâ 4C'0 C0@D0<Cä9 C0@Cä1D >D :C, A
                if (context.Selector != null)
                {
                    options.UseCustomSelector(context.Selector);
                }
            });

            // At0C0CD :C 5CÄ :C 6CDKC' =C 9CD5C0=D'9 D$>Dt5Dt=D'9 C$0C'8CD0D$>D =C AC'5CD=C„:C
            foreach (IValidator validator in specificValidators)
            {
                // BtBCä1D² 8Ct1CT6C BDÄ 7C FC„:C'8C$0C08D0Ä ?D >C$5D 0CT<, DtBCä =C 9CD5C0=D'9 C$0C'8CD0D$>D
                if (validator.GetType().IsGenericType &&
validator.GetType().GetGenericTypeDefinition() == typeof(GlobalPolymorphicValidator<>))
                    continue;

                ValidationResult specificResult = validator.Validate(newContext);

                // A05D 5C0>D 8CÄ 2D 5 Cä1C00D CCd5C0=D'5 CäHC„1C08 C" >C IC„9 D 5CtCC'LD$0D" 2C ;C„4C FC„8 D
                foreach (ValidationFailure error in specificResult.Errors)
                {
                    result.Errors.Add(error);
                }
            }
        }

        return true;
    }
}
</file>

```

```

<file path="Domain.Interface/Enums/EnumExtensions.cs">
using System.ComponentModel;
using System.Reflection;

namespace Promatis.Net.Domain.Interface;

```

```

Ð
public static class EnumExtensionsÐ
{Ð
    /// <summary>Ð
    /// A$>Ct2D 0D"0CTB Ct=C GCT=C,,5 C BD 8C CD$0 [Description] CD;Dò ;Dä1Cä3Cä MC´5CÄ5C0BC ?CT@CTGC,,AC´5C08
    /// ATAC´8 C BD 8C CD" >D$AD4BD BC$CCTB, C$>Ct2D 0D"0CTB D BC =CD0D BC0>CR 8CÄ0 .ToString();Ð
    /// </summary>Ð
    public static string GetDescription(this Enum value)Ð
    {Ð
        FieldInfo? field = value.GetType().GetField(value.ToString());Ð
        if (field == null) return value.ToString();Ð
    }
    var attribute = field.GetCustomAttribute<DescriptionAttribute>();Ð
    return attribute != null ? attribute.Description : value.ToString();Ð
}Ð
}
</file>

<file path="Domain.Interface/Interfaces/IAudit.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
/// <summary>Ð
/// B CD"=CäAD$8, D 5C ;C,,7D4ND"8CR MD$>D" 8C0BCT@DD5C"A, C CCDCD" 7C ?C,,AD´2C BDÄADò 2 AuditLogÐ
/// </summary>Ð
public interface IAuditÐ
{Ð
    Ð
}
}
</file>

<file path="Domain.Interface/Interfaces/IDomainObject.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
public interface IDomainObjectÐ
{Ð
    DateTime CreatedAt { get; init; }Ð
}
}
</file>

<file path="Domain.Interface/Interfaces/IDomainObjectHasKey.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
public interface IDomainObjectHasKey<TKey> : IDomainObjectÐ
{Ð
    TKey Id { get; set; }Ð
}
}
</file>

<file path="Domain.Interface/Interfaces/IEntityCloner.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
/// <summary>Ð
/// A,,=D$5D DCT9D ?C´0D$DCä@CÄ5C0=Cä3Cä 4C$8Cd:C 8Ct>C´8D >C$0C0=Cä3Cä :C´>C08D >C$0C08Dò 4Cä<CT=C0KDR AD4I
/// </summary>Ð
public interface IEntityClonerÐ
{Ð
    T CloneEntity<T>(T entity) where T : class;Ð
}
}
</file>

<file path="Domain.Interface/Interfaces/ISoftDeletable.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
/// <summary>Ð
/// A,,=D$5D DCT9D 4C´O CÄOC4:Cä3Cä CCD0C´5C08Dò >C JCT:D$>C-
/// </summary>Ð
public interface ISoftDeletableÐ
{Ð
    DateTime? DeletedAt { get; set; }Ð
    Ð
    string? DeletedBy { get; set; }Ð
}
}
</file>

<file path="Domain.Interface/Interfaces/ITreeNode.cs">
namespace Promatis.Net.Domain.Interface;Ð

```

```

Ð
Ð
/// <summary>Ð
/// A4;Că1C ;DĂ=D´9 Cô;C BDD>D <CT=CÔKC´ :Că=D$@C :D" 4C´O C$ACTE C,,5D 0D EC,,GCTAC=8DR ,,4D 5C$>C$8CD=D´E) D C
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Că<CT=CÔ>C´ <Că4CT;C,ăÂ÷G– W & Óí
public interface ITreeNode<T> : IDomainObjectHasKey<Guid> where T : classÐ
{Ð
    /// <summary>Ð
    /// A,,4CT=D$8DD8C=0D$>D @Că4C,,BCT;DĂAC=8C' >D47C´0. B 0C$5CÔ çVÆÂ 4C´O C= >D =CT2D´E DÔ;CT<CT=D$>C"í
    /// A,,!Aô A A´ AÔ : AD>C 0C$;CT= set CD;Dò 2Că7CĂ>Cd=CăAD$8 C,,7CĂ5CÔ5CÔ8Dò AC$0Ct5C´ ,,?CT@CT<CTICT=C,,O C
    /// </summary>Ð
    Guid? ParentId { get; set; }Ð
Ð
    /// <summary>Ð
    /// AÔ0C$8C40Dd8Că=CÔ>CR AC$>C"AD$2Că AD KC´:C, =C @Că4C,,BCT;DĂAC=8C' >C JCT:D"í
    /// </summary>Ð
    T? Parent { get; set; }Ð
Ð
    /// <summary>Ð
    /// A= >C´;CT:Dd8Dò 4CăGCT@CÔ8DR MC´5CĂ5CÔBCă2 (Cô>CDGC,,=CT=CÔKDR Cct;Că2).Ð
    /// </summary>Ð
    ICollection<T> Children { get; }Ð
}
</file>

<file path="Domain.Interface/Promatis.Net.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
    Ð
    <PropertyGroup>Ð
        <TargetFramework>net10.0</TargetFramework>Ð
        <ImplicitUsings>enable</ImplicitUsings>Ð
        <Nullable>enable</Nullable>Ð
    </PropertyGroup>Ð
    Ð
</Project>
</file>

<file path="Domain/AuditLog/AuditLog.cs">
namespace Promatis.Net.Domain;Ð
Ð
/// <summary>Ð
/// A CCD8D" AC,,AD$5CĂKÐ
/// </summary>Ð
public class AuditLog : DomainObjectÐ
{Ð
    /// <summary>Ð
    /// AÔ0C,,<CT=Că2C =C,,5 D >C KD$8Dý
    /// </summary>Ð
    public string EntityName { get; init; } = string.Empty;Ð
    Ð
    /// <summary>Ð
    /// A,,4CT=D$8DD8C=0D$>D ACă1D´BC,,0Ð
    /// </summary>Ð
    public Guid EntityId { get; init; }Ð
    Ð
    /// <summary>Ð
    /// AD5C"AD$2C,,5Ð
    /// </summary>Ð
    public string Action { get; init; } = string.Empty;Ð
    Ð
    /// <summary>Ð
    /// AD0D$0 C,,7CĂ5CÔ5CÔ8Dý
    /// </summary>Ð
    public DateTime ChangedAt { get; init; }Ð
    Ð
    /// <summary>Ð
    /// A,,7CĂ5CÔ5CÔ>Ð
    /// </summary>Ð
    public string? ChangedBy { get; init; }Ð
    Ð
    /// <summary>Ð
    /// B BD >C=0 C,,7CĂ5CÔ5CÔ8C•
    /// </summary>Ð
    public string ChangesJson { get; init; } = string.Empty;Ð
}

```

</file>

```
<file path="Domain/DomainObject/DomainObject.cs">
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// AäACÔ>C$=Cä9 C;C AD „uT"B ?Cä CCÄ>C´GC =C„N)
    /// </summary>
    public abstract class DomainObject : DomainObjectBase<Guid>
    {
        protected DomainObject()
        {
            Id = Guid.NewGuid();
        }
    }
}</file>
```

```
<file path="Domain/DomainObject/DomainObjectBase.cs">
using Promatis.Net.Domain.Interface;
{
    namespace Promatis.Net.Domain;
    {
        /// <summary>
        /// B4=C„2CT@D 0C´LCÔKC´ 1C 7Cä2D´9 C;C AD A CÔ>CD4CT@Cd:Cä9 C´NC >C4> D$8Cô0 C;DäGC
        /// </summary>
        /// <typeparam name="TKey">B$8Cò :C´NDt0</typeparam>
        public abstract class DomainObjectBase<TKey> : IDomainObjectHasKey<TKey>
        {
            public TKey Id { get; set; } = default!;
            public DateTime CreatedAt { get; init; } = DateTime.UtcNow;
        }
    }
}</file>
```

```
<file path="Domain/DomainObject/DomainObjectValidator.cs">
using FluentValidation;
{
    namespace Promatis.Net.Domain;
    {
        /// <summary>
        /// B4=C„2CT@D 0C´LCÔKC´ 2C ;C„4C BCä@ CD;Dò 2D 5DR :C´0D ACä2, CÔ0D ;CT4D45CÄKDR >D" FöÖ -äö&|V7M
        /// </summary>
        public abstract class DomainObjectValidator<T> : AbstractValidator<T> where T : DomainObject
        {
            protected DomainObjectValidator()
            {
                RuleFor(x => x.Id)
                    .NotEmpty().WithMessage("A„4CT=D$8DD8C;0D$>D >C JCT:D$0 CÔ5 CÄ>Cd5D" 1D´BDÄ ?D4AD$KCÄâ""½
                RuleFor(x => x.CreatedAt)
                    .Must(date => date <= DateTime.UtcNow.AddSeconds(1))
                    .WithMessage("AD0D$0 D >Ct4C =C„0 CÔ5 CÄ>Cd5D" 1D´BDÄ 2 C CCDCD"5CÄâ""½
            }
        }
    }
}</file>
```

```
<file path="Domain/EntityCloner/JsonEntityCloner.cs">
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Text.Json.Serialization.Metadata;
using Promatis.Net.Domain.Interface;
{
    namespace Promatis.Net.Domain;
    {
        public class JsonEntityCloner : IEntityCloner
        {
            private static readonly JsonSerializerOptions JsonOptions = new()
            {
                ReferenceHandler = ReferenceHandler.IgnoreCycles,
                TypeInfoResolver = new DefaultJsonTypeInfoResolver
                {
                    Modifiers = { typeInfo =>
                        {
                            foreach (var property in typeInfo.Properties)
                            {
                                // 1. A$KD 5Ct0CT< Dd8C;C„GCTAC;8CR =C 2C„3C FC„>CÔ=D´5 D 2Cä9D BC$0 C„5D 0D EC„9D
                            }
                        }
                    }
                }
            }
        }
    }
}</file>
```



```

        if (property.Name is "Parent" or "Children")
        {
            property.ShouldSerialize = (_, _) => false;
            continue;
        }

        // 2. A$KD 5Ct0CT< D ;Cä6C0KCR AC$0Ct0C0=D´5 CD>CÄ5C0=D´5 Cä1D=5C=BD² 1D0:CT=CD0 C, :Cä;C
        Type propType = property.PropertyType;
        bool isDomainClass = typeof(Domain.DomainObject).IsAssignableFrom(propType);
        bool isDomainCollection = propType.IsGenericType &&

typeof(System.Collections.IEnumerable).IsAssignableFrom(propType) &&

typeof(Domain.DomainObject).IsAssignableFrom(propType.GetGenericArguments().FirstOrDefault());
        if (isDomainClass || isDomainCollection)
        {
            property.ShouldSerialize = (_, _) => false;
        }
    }
}

public T CloneEntity<T>(T entity) where T : class
{
    if (entity == null) throw new ArgumentNullException(nameof(entity));

    // B 5D 8C ;C,,7D45CÄ 8 CD5D 5D 8C ;C,,7D45CÄÄ AD$@Cä3Cä ACäED 0C00D0 @CT0C´LC0KC´ @C =D$0C"<-D$8C0 =C
    string json = JsonSerializer.Serialize(entity, entity.GetType(), JsonOptions);
    return (T)JsonSerializer.Deserialize(json, entity.GetType(), JsonOptions)!;
}
</file>

<file path="Domain/Promatis.Net.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="FluentValidation" Version="12.1.1" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Domain/ReferenceBase/ReferenceBase.cs">
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Domain;
/// <summary>
/// A 0Ct>C$KC´ :C´0D A CD;D0 2D 5DR AC0@C 2CäGC08C= >C-
/// </summary>
public abstract class ReferenceBase : DomainObject, ISoftDeletable
{
    public string Name { get; set; } = string.Empty;

    public string? Description { get; set; }

    public string? Code { get; set; }

    public DateTime? DeletedAt { get; set; }

    public string? DeletedBy { get; set; }
}
</file>

```

```

<file path="Domain/ReferenceBase/ReferenceBaseValidator.cs">
using FluentValidation;ð
using FluentValidation.Results;ð
ð
namespace Promatis.Net.Domain;ð
ð
/// <summary>ð
/// B4=C,,2CT@D 0C´LCÔKC´ 2C ;C,,4C BCâ@ CD;Dð 2D 5DR :C´0D ACâ2, CÔ0D ;CT4D45CÄKDR >D" &VfW&Væ6T& 6]
/// </summary>ð
public abstract class ReferenceBaseValidator<T> : DomainObjectValidator<T> where T : ReferenceBaseð
{ð
    /// <summary>ð
    /// A$8D BD40C´LCÔKC´ ?C BD$5D = D 5C4CC´OD =Câ3Câ 2D´@C 6CT=C,,0 CD;Dð ?D >C$5D :C, Câ4C í
    /// Að> D4<Câ;Dt0C08Däç AD$@Câ3Câ ;C BC,,=C,,FC 2 C$5D ECÔ5Câ @CT3C,,AD$@CRâ FC,,DD K C, òí
    /// </summary>ð
    protected virtual string CodeRegexPattern => @"^[A-Z0-9_]*$";ð
ð
    /// <summary>ð
    /// A$8D BD40C´LCÔ>CR ACâ>C ICT=C,,5 Câ1 CâHC,,1Cð5 C$0C´8CD0Dd8C, @CT3D4;Dð@CÔ>C4> C$KD 0Cd5CÔ8Dð Câ4C í
    /// </summary>ð
    protected virtual string CodeValidationMessage => "Að>CB <Câ6CTB D >CD5D 6C BDÂ BCâ;DÄ:Câ ;C BC,,=C,,FD2 2
ð
    protected ReferenceBaseValidator()ð
    {ð
        RuleFor(x => x.Name)ð
            .NotEmpty().WithMessage("AÔ0C,,<CT=Câ2C =C,,5 Câ1Dô7C BCT;DÄ=Câ 4C´O Ct0CÔ>C´=CT=C,,0")ð
            // Að@Câ2CT@Cð0 CÔ0 Cð@Câ1CT;D² 2 CÔ0Dt0C´5 C, :Câ=Dd5ð
            .Must(name => name == null || name.Trim() == name)ð
            .WithMessage("AÔ0C,,<CT=Câ2C =C,,5 CÔ5 CÄ>Cd5D" =C GC,,=C BDÄADð 8C´8 Ct0Cð0CÔGC,,2C BDÄADð ?D >C 5C´
            .MinimumLength(2).WithMessage("AÔ0C,,<CT=Câ2C =C,,5 CD>C´6CÔ> D >CD5D 6C BDÂ <C,,=C,,<D4< 2 D 8CÄ2Câ;
            .MaximumLength(250).WithMessage("Að@CT2D´HCT=C <C :D 8CÄ0C´LCÔ0Dð 4C´8CÔ0 CÔ0C,,<CT=Câ2C =C,,0 (25
ð
        RuleFor(x => x.Code)ð
            .Matches(x => CodeRegexPattern)ð
            .When(x => !string.IsNullOrEmpty(x.Code))ð
            .WithMessage(x => CodeValidationMessage);ð
    }ð
ð
    public Func<object, string, Task<IEnumerable<string>>> ValidateValue => async (model,
propertyName) =>ð
    {ð
        ValidationResult? result = await
ValidateAsync(ValidationContext<T>.CreateWithOptions((T)model, x =>
x.IncludeProperties(propertyName)));ð
        return result.IsValid ? [] : result.Errors.Select(e => e.ErrorMessage);ð
    };ð
}
</file>

```

```

<file path="Domain/ReferenceTreeBase/ReferenceTreeBase.cs">
using Promatis.Net.Domain.Interface;ð
ð
namespace Promatis.Net.Domain;ð
ð
/// <summary>ð
/// A 0Ct>C$KC´ :C´0D A CD;Dð 2D 5DR ACô@C 2CâGCÔ8Cð>C" ACâ4CT@Cd0D"8DR 4CT@CT2DÄ0ð
/// </summary>ð
/// <summary>ð
/// A 0Ct>C$KC´ :C´0D A CD;Dð 2D 5DR 4D 5C$>C$8CD=D´E D ?D 0C$>Dt=C,,:Câ2 (MDM).ð
/// A,,ACô>C´LctCCTB Cð0D$BCT@CÔ 5%E 4C´O CÔ5D 5CD0Dt8 D BD >C4> D$8CÔ8Ct8D >C$0CÔ=D´E CÔ0C$8C40Dd8Câ=CÔKDR A
/// </summary>ð
/// <typeparam name="T">B$8Cò :Câ=Cð@CTBCÔ>C4> CD>CÄ5CÔ=Câ3Câ >C JCT:D$0 (CÔ0D ;CT4CÔ8Cð0).</typeparam>ð
public abstract class ReferenceTreeBase<T> : ReferenceBase, ITreeNode<T> where T : classð
{ð
    /// <summary>ð
    /// A,,4CT=D$8DD8Cð0D$>D @Câ4C,,BCT;DÄACð>C4> D47C´0 C" !B4 ABí
    /// </summary>ð
    public Guid? ParentId { get; set; }ð
ð
    /// <summary>ð
    /// AÔ0C$8C40Dd8Câ=CÔ>CR AC$>C"AD$2Câ AD KC´:C, =C @Câ4C,,BCT;DÄACð8C´ >C JCT:D" 2 Cä?CT@C BC,,2CÔ>C´ ?C <
    /// Að5D 5Câ?D 5CD5C´OCTBD 0 Cð0Cç f–'GV Â 2 Cð>CÔ5Dt=D´E Cð;C AD 0DR 4C´O Cð>CD4CT@Cd:C, Æ $´ Æð F–æri
    /// </summary>ð
    public virtual T? Parent { get; set; }ð
ð
}

```

```

    /// <summary>ð
    /// Að>C´;CT:Dd8Dð 4CägCT@CÔ8DR MC´5CÄ5CÔBCä2 (Cö>CDGC,,=CT=CÔKDR Cct;Cä2).ð
    /// </summary>ð
    public virtual ICollection<T> Children { get; set; } = new List<T>();ð
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain.Interface;ð
ð
namespace Promatis.Net.Domain;ð
ð
/// <summary>ð
/// B4=C,,2CT@D 0C´LCÔKC´ 2C ;C,,4C BCä@ CD;Dð 2D 5DR :C´0D ACä2, CÔ0D ;CT4D45CÄKDR >D" &VfW&Væ6UG&VT& 6]
/// </summary>ð
public abstract class ReferenceTreeBaseValidator<T> : ReferenceBaseValidator<T>ð
    where T : ReferenceBase, ITreeNode<T>ð
{ð
    protected ReferenceTreeBaseValidator() : base()ð
    {ð
        // Aô@Cä2CT@Cð0 CÔ0 D 0CÄ>Dd8D$8D >C$0CÔ8CR ,,>C JCT:D" =CR <Cä6CTB D4:C 7D´2C BDÄ =C ACT1Dò :C : CÔ0
        RuleFor(x => x.ParentId)ð
            .Must((model, parentId) => parentId != model.Id)ð
            .WithMessage("Aä1Dð5CðB CÔ5 CÄ>Cd5D" 1D´BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");ð
    }ð
    // A 0Ct>C$0Dò ?D >C$5D :C DCä@CÄ0D$0 GUIDð
    RuleFor(x => x.ParentId)ð
        .NotEqual(Guid.Empty)ð
        .When(x => x.ParentId.HasValue)ð
        .WithMessage("B >CD8D$5C´LD :C,,9 C,,4CT=D$8DD8Cð0D$>D =CR <Cä6CTB C KD$L CÔCD BD´< GUID");ð
    }ð
}
</file>

<file path="Promatis.Net.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.Formsð
@using Microsoft.AspNetCore.Components.Routingð
@using Microsoft.AspNetCore.Components.Webð
@using MudBlazorð
@using Promatis.Net.UI.Components.Workspaceð
@using Promatis.Net.UI.Components.EditDialogð
@using Promatis.Net.UI.Components.Gridð
@using Promatis.Net.UI.Components.Toolbarð
@using Promatis.Net.Domain.Interface
</file>

<file path="Promatis.Net.UI/Components/EditDialog/DialogTab.razor">
@* Að>CÄ?Cä=CT=D" =CR @CT=CD5D 8D" ...DÔÄ AC <, Cä= D ;D46C,,B CD5Cð;C @C BC,,2CÔKCÄ >Cô8D 0D$5C´5CÄ 4C´0 EditDia
@code {ð
    [Parameter] public required string Title { get; set; }ð
    [Parameter] public string? Icon { get; set; }ð
    [Parameter] public required RenderFragment Content { get; set; }ð
}
</file>

<file path="Promatis.Net.UI/Components/EditDialog/DialogTab.razor.cs">
using Microsoft.AspNetCore.Components;ð
ð
namespace Promatis.Net.UI.Components.EditDialog;ð
ð
public partial class DialogTab : ComponentBaseð
{ð
    [CascadingParameter] protected EditDialog ParentDialog { get; set; } = null!;ð
    protected override void OnInitialized()ð
    {ð
        base.OnInitialized();ð
    }ð
    if (ParentDialog != null)ð
    {ð
        ParentDialog.RegisterTab(this); // Aô0D$8C$=Cä >D$4C 5CÄ ACT1Dò =C 2CT@DR 2 Cð>C´;CT:Dd8Dä @Cä4C,,
    }ð
    }ð
}
</file>

```

```

<file path="Promatis.Net.UI/Components/EditDialog/EditDialog.razor">
@namespace Promatis.Net.UI.Components.EditDialog
@
@inherits Microsoft.AspNetCore.Components.ComponentBase
@
@
<MudDialog>
    <TitleContent>
        <MudText Typo="Typo.h6">
            <MudIcon Icon="@((IsNew ? Icons.Material.Filled.Add : Icons.Material.Filled.Edit))"
Class="mr-2 mb-n1" />
            @Title
        </MudText>
    </TitleContent>
    <DialogContent>
        @* B :D KD$KC' AC >D IC,,: C$:C'0CD>C¢ 8Cr 4CäGCT@C05C4> C¢>C0BCT=D$0 *@
        <CascadingValue Value="this" IsFixed="true">
            @Tabs
        </CascadingValue>
    @
    <MudForm @ref="_form"
        Model="@Model"
        Validation="@((async (object m) => await ExecuteFluentValidationAsync(m)))"
        ValidationDelay="200">
    @
        @* A,,!Aô A A' Aô : A$=CTHC08C' :Cä=D$5C'=CT@ Cd5D BC¢> CD5D 6C,,B C$KD >D$C Cä:C00 DD8C¢AC,,@Cä2C
        <div class="d-flex flex-column pt-2" style="min-height: 340px; max-height: 65vh; min-
width: 0;">
            @if (_isProcessing)
            {
                <div class="d-flex justify-center my-4">
                    <MudProgressCircular Color="Color.Primary" Indeterminate="true" />
                </div>
            }
            else
            {
                @* A B$ AÄ B$ A¢ B Aô AT A,, A4 : ATAC'8 C$:C'0CD:C >CD=C r 4C 5CÂ 5C' AC¢@Cä;C'1C @
                @if (_collectedTabs.Count == 1)
                {
                    <div style="max-height: 60vh; overflow-y: auto; padding-right: 4px;">
                        @_collectedTabs.First().Content
                    </div>
                }
                else if (_collectedTabs.Count > 1)
                {
                    @* A,,!Aô A A' Aô : At0CD0CT< PanelStyle CD;Dò =C <CT@D$2Câ DC,,:D 8D >C$0C0=Cä3Câ AC¢
                    <MudTabs Elevation="0"
                        Rounded="true"
                        ApplyEffectsToInnerContainer="true"
                        PanelClass="pa-4"
                        PanelStyle="max-height: 50vh; overflow-y: auto; padding-right:
4px;">
                        @foreach (var tab in _collectedTabs)
                        {
                            <MudTabPanel Text="@tab.Title" Icon="@tab.Icon">
                                @tab.Content
                            </MudTabPanel>
                        }
                    </MudTabs>
                }
            }
        </div>
    </MudForm>
    </DialogContent>
    <DialogActions>
        <MudButton Variant="Variant.Text" OnClick="Cancel" Disabled="_isProcessing">AäBCÄ5C0</MudButton>
        <MudButton Variant="Variant.Filled" Color="Color.Primary" OnClick="Submit"
Disabled="_isProcessing">
            @if (_isProcessing)
            {
                <MudProgressCircular Class="ms-n1" Size="Size.Small" Indeterminate="true" />
                <MudText Class="ms-2">B >DT@C =CT=C,,5...</MudText>
            }
            else
            {

```

```

        <MudText>B >DT@C =C,,BDÃÃô×VEFW‡Cí
    }ð
</MudButton>ð
</DialogActions>ð
</MudDialog>
</file>

<file path="Promatis.Net.UI/Components/EditDialog/EditDialog.razor.cs">
using FluentValidation;ð
using FluentValidation.Results;ð
using Microsoft.AspNetCore.Components;ð
using MudBlazor;ð
using Severity = MudBlazor.Severity;ð
ð
ð
namespace Promatis.Net.UI.Components.EditDialog;ð
ð
public partial class EditDialog : ComponentBaseð
{ð
    protected MudForm _form = null!;ð
    protected bool _isProcessing;ð
    protected readonly List<DialogTab> _collectedTabs = [];ð
ð
    [CascadingParameter] protected IMudDialogInstance MudDialog { get; set; } = null!;ð
ð
    [Parameter] public string Title { get; set; } = "B 5CD0C▯BC,,@Cä2C =C,,5";ð
    [Parameter] public bool IsNew { get; set; }ð
    [Parameter] public object Model { get; set; } = null!;ð
    [Parameter] public IValidator Validator { get; set; } = null!;ð
    [Parameter] public Func<Task> OnSaveAction { get; set; } = null!;ð
ð
    /// <summary>ð
    /// B NCD0 C0@C,,:C´0CD=Cä9 D 0Ct@C 1CäBDt8C¢ 4CT:C´0D 0D$8C$=Cä ?C,,HCTB D$5C48 <DialogTab>ð
    /// </summary>ð
    [Parameter] public RenderFragment? Tabs { get; set; }ð
ð
    [Inject] protected ISnackbar Snackbar { get; set; } = null!;ð
ð
    /// <summary>ð
    /// AA5D$>CB 4C´0 D 5C48D BD 0Dd8C, 2C▯;C 4Cä:, C$KcTKC$0CT<D´9 CD>Dt5D =C,,<C, :Cä<C0>C05C0BC <C, F- Æ0uF
    /// </summary>ð
    public void RegisterTab(DialogTab tab)ð
    {ð
        if (!_collectedTabs.Contains(tab))ð
        {ð
            _collectedTabs.Add(tab);ð
            StateHasChanged();ð
        }ð
    }ð
ð
    protected void Cancel() => MudDialog.Cancel();ð
ð
    protected async Task<IEnumerable<string>> ExecuteFluentValidationAsync(object model)ð
    {ð
        if (model is not Domain.DomainObject || Validator == null) return Array.Empty<string>();ð
ð
        var context = new ValidationContext<object>(model);ð
        ValidationResult result = await Validator.ValidateAsync(context);ð
ð
        return result.IsValid ? Array.Empty<string>() : result.Errors.Select(e => e.ErrorMessage);ð
    }ð
ð
    protected async Task Submit()ð
    {ð
        await _form.ValidateAsync();ð
ð
        if (!_form.IsValid)ð
        {ð
            Snackbar.Add("A0>Cd0C´CC"AD$0, C,,AC0@C 2DÄBCR >D,,8C :C, 2 DD>D <CR ?CT@CT4 D >DT@C =CT=C,,5CÂâ"Ã 6
            return;ð
        }ð
ð
        _isProcessing = true;ð
        StateHasChanged();ð
ð
        tryð

```

```

        {
            if (OnSaveAction != null) await OnSaveAction();
            MudDialog.Close(DialogResult.Ok(Model));
        }
        catch (Exception ex)
        {
            string message = ex.InnerException?.Message ?? ex.Message;
            Snackbar.Add($"AâHC,,1Câ0 D >DT@C =CT=C,,0: {message}", Severity.Error);
        }
        finally
        {
            _isProcessing = false;
            StateHasChanged();
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/Grid/GridActionContext.cs">
using FluentValidation;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.DependencyInjection;
using MudBlazor;
using Promatis.Net.Configuration;
using Promatis.Net.UI.Components.Toolbar;

namespace Promatis.Net.UI.Components.Grid;

/// <summary>
/// A 0t>C$KC' BC 1C'8Dt=D'9 Câ>C0BCT:D B D4?D 0C$;CT=C,,0 C0;C BDD>D <D²î
/// A 2D$>CÂ0D$8Dt5D :C, >C 5D ?CTGC,,2C 5D" 2D'ACâ:Câ?D >C,,7C$>CD8D$5C'LC0KCR At#-CÂCD$0Dd8C, ?D 8 C'NC KDR
/// </summary>
public abstract class GridActionContext<TEntity> : ToolbarActionContext<TEntity> where TEntity :
class
{
    // Bt8D BCâ5 C00D$8C$=Câ5 C,,7C$;CTGCT=C,,5 D ;D46C 8Cr :Câ=D$5C"=CT@C BCT:D4ICT9 Blazor-D 5D AC,,8 C$:C'0
    protected IDialogService DialogService => ScopedProvider.GetRequiredService<IDialogService>();
    protected IValidator<TEntity> GlobalValidator =>
        ScopedProvider.GetRequiredService<IValidator<TEntity>>();

    public TEntity? SelectedItem
    {
        get => SelectedData;
        set => SelectedData = value;
    }

    /// <summary>
    /// B4=C,,2CT@D 0C'LC0KC' 1D >Câ5D 4C =C0KDR BCT:D4ICT9 D$0C ;C,,FD²î
    /// </summary>
    public UiDataBroker<TEntity> DataBroker { get; } = new();

    protected GridActionContext() => Position = ToolbarPosition.Top;

    // A0 A "BD B AT A0+A' A$"Aâ A "A,,AT!Aâ A' A$ Ad A¢ At#-AÂ#B$ Bd A'

    /// <summary>
    /// A4;Câ1C ;DÄ=D'9 C05D 5DT2C BDT8C¢ :Câ<CÂ8D$>C" !B4 AB =C CD >C$=CR BC 1C'8Dt=Câ3Câ OCD@C î
    /// A0>C' =CâAD$LDâ >D 2Câ1Câ6CD0CTB C0@C,,:C'0CD=D'E D 0Ct@C 1CâBDt8Câ>C" >D" @D4GC0>C4> C00C08D 0C08D0 ;C
    /// </summary>
    public override void HandleGlobalEntityCommit(object? state, object? entity)
    {
        if (entity == null) return;

        Type entityType = entity.GetType();

        // B @CT7C 5Câ 4C,,=C <C,,GCTACâ8CR ?D >CâAC, 6 7FÆR0Tb 6÷&]
        if (entityType.BaseType != null && entityType.Namespace == "Castle.Proxies")
        {
            entityType = entityType.BaseType;
        }

        // 1. ATAC'8 D$@C =Ct0CâFC,,0 C,,7 C 0CtK CD0C0=D'E Ct0D$@Câ=D4;C 8CÂ5C0=Câ BC,,? CD0C0=D'E C00D,,5C' BC
        if (typeof(TEntity).IsAssignableFrom(entityType))
        {
            string stateStr = state?.ToString() ?? string.Empty;

```

```

    }
    // 2. A 2D$>CÄ0D$8Dt5D :C, ?C,,=C 5CÂ At#-CD2C,,6Cä: C @Cä:CT@C ?D 8CÄ5C08D$L CD5C´LD$C Ct0 0 CÄA
    DataBroker.ApplyIncrementalOzuDelta(stateStr, (TEntity)entity);
}
// 3. A$KctKC$0CT< C 0Ct>C$KCR ?D 0C$8C´0 D 1D >D 0 DD>C=C D 0 ToolbarActionContext
base.HandleGlobalEntityCommit(state, entity);
}
// 4. A 2D$>CÄ0D$8Dt5D :C, ?C,,=C 5CÂ w&-E vR ?CT@CT@C,,ACä2C BDÂ ...DÔÂÔAD$@Cä:C, 8Cr >C =Cä2C´5C0=
RequestRefresh();
}
}
protected override void RecalculateButtonStates()
{
    base.RecalculateButtonStates();
    NotifyUpdate();
}
protected async Task OpenEditDialogAsync<TDialog>(TEntity model, string title, bool isNew,
Func<Task> saveDelegate)
where TDialog : Microsoft.AspNetCore.Components.IComponent
{
    var parameters = new DialogParameters
    {
        ["Title"] = title,
        ["IsNew"] = isNew,
        ["Model"] = model,
        ["Validator"] = GlobalValidator,
        ["OnSaveAction"] = async () =>
        {
            // A$KC0>C´=D05CÂ ?CT@CT4C =C0>CR ?D 8C=C 4C0>CR 4CT9D BC$8CR ACäED 0C05C08D0 ,, FB 8C´8 Upda
            await saveDelegate();
        }
    };
    // Bd5C0BD 0C´8Ct>C$0C0=Cä :Cä<C =CDCCT< C4@C,,4D2 >C =Cä2C,,BDÂ 4C =C0KCR 2 Aä B=
    RequestRefresh();
}
}
var options = new DialogOptions { CloseButton = true, MaxWidth = MaxWidth.Small, FullWidth
= true };
// A$KctKC$0CT< MudBlazor CD8C ;Cä3 CD;D0 0C AD$@C :D$=Cä3Cä :Cä<C0>C05C0BC DF- Äö}
await DialogService.ShowAsync<TDialog>(title, parameters, options);
}
}
</file>

<file path="Promatis.Net.UI/Components/Grid/GridPage.razor">
@typeparam TEntity where TEntity : class
{
    <MudDataGrid T="TEntity"
        ServerData="@LoadGridDataInternalAsync"
        @ref="_grid"
        SelectedItem="@ActionContext.SelectedData"
        SelectedItemChanged="@OnSelectedItemChanged"
        RowStyleFunc="@FormatSelectedRowStyle"
        Hover="true"
        Dense="true"
        Bordered="true"
        Elevation="0"
        FixedHeader="true"
        Height="calc(100vh - 280px)">
        <Columns>
            @ColumnsContent
        </Columns>
        <PagerContent>
            @if (PagerContent != null)
            {
                @PagerContent
            }
            else
            {
                <MudDataGridPager T="TEntity" InfoFormat="{first_item}-{last_item} C,,7 {all_items}"
                RowsPerPageString="B BD >C¢ =C AD$@C =C,,FCS¢" óí
            }
        </PagerContent>
    }
}

```

```
</MudDataGrid>
</file>
```

```
<file path="Promatis.Net.UI/Components/Grid/GridPage.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;

namespace Promatis.Net.UI.Components.Grid;

public partial class GridPage<TEntity> : ComponentBase, IDisposable where TEntity : class
{
    [Parameter] public RenderFragment? ColumnsContent { get; set; }
    [Parameter] public RenderFragment? PagerContent { get; set; }

    [CascadingParameter] protected GridActionContext<TEntity> ActionContext { get; set; } = null!;

    [Inject]
    public IServiceProvider SystemServiceProvider { get; set; } = null!;

    private MudDataGrid<TEntity> _grid = null!;

    protected override void OnInitialized()
    {
        base.OnInitialized();

        if (ActionContext == null)
        {
            throw new ArgumentNullException(nameof(ActionContext),
                $"A?C?C=CT=D" ¶æ ÖVöb,,w&-E vSÄDVçF-G"â-Ö BD 5C CCTB CÔ0C'8Dt8Dò ¶æ ÖVöb,,w&-D 7F-öä6öçFW±C
            );
            // AÄ0C48Dò ?D 8C$0Ct:Cfç >D$4C 5CÂ :Cä=D$5C=AD$C D BD 0C08DdK Cö@Cä2C 9CD5D BCT:D4ICT9 Cd8C$>C' ACT
            ActionContext.ScopedProvider = SystemServiceProvider;

            ActionContext.OnContextUpdated += StateHasChanged;
            ActionContext.OnRefreshRequested += HandleRefreshRequested;
        }

        private void HandleRefreshRequested()
        {
            InvokeAsync(ReloadServerDataAsync);
        }

        /// <summary>
        /// A$=D4BD 5CÔ=C,,9 CÄ5D$>CBÔ<CäAD"â C$BCä<C BC,,GCTAC=8 C, 1CT7Cä?C ACÔ> C$KctKC$0CTBD 0 C= >CÄ?Cä=CT=D$>
        /// C" 0D 8CÔED >CÔ=Cä< C= >CÔBCT:D BCR &Æ |÷"Â 8D :C'NDt0Dò ;Dä1D'5 DD@C,,7D² <CT=Däí
        /// </summary>
        protected async Task<GridData<TEntity>> LoadGridDataInternalAsync(GridState<TEntity> state,
            CancellationToken token)
        {
            // A B$ ÄÄ B$ A= Aô B A,,AÔ A4 Aô Ää B A$ :D
            // ATAC'8 C$:C'NDt5CÔ At#-D 5Cd8CÄ ,,1D >C=5D MD$> Ct=C 5D"'Â =Cä 4C =CÔKCR 5D"5 CÔ5 Cö@CT4Ct0C4@D46
            if (ActionContext.DataBroker.IsInMemoryMode && ActionContext.DataBroker.InMemoryItems ==
                null)
            {
                // A,,ICT< CÄ5D$>CB ?D >C4@CT2C =C :Cä=C=CTBCÔ>CÄ :Cä=D$5C=AD$5 D BD 0C08DdK
                var initMethod = ActionContext.GetType().GetMethod("InitializeInMemoryDataAsync");
                if (initMethod != null)
                {
                    // A$KctKC$0CT< CT3Cä 8 Ad AT (await) Ct0C$5D HCT=C,,0. D
                    // B$0Cç :C : DÔBCäB CÄ5D$>CB 2D'?Cä;CÔ0CTBD 0 C$=D4BD 8 C= >CÔBCT:D BC 6W'fw$F F Ä
                    // MudBlazor D 0CÄ 2C=;DäGC,,B C= @C AC,,2D'9 C'>C 4CT@, C <CT=Dä AD$@C =C,,FD² >D BC =CTBD 0 10
                    var task = (Task)initMethod.Invoke(ActionContext, null)!;
                    await task;
                }
            }

            // AÔ0Cö@Dö<D4N C$KctKC$0CT< C @Cä:CT@ CD0CÔ=D'E, D 0Ct2CT@CÔCD$KC' 2CÔCD$@C, =C HCT3Cä :Cä=D$5C=AD$0
            return await ActionContext.DataBroker.FetchDataAsync(state, token);
        }

        protected void OnSelectedItemChanged(TEntity? newItem)
        {
            if (newItem == null && ActionContext.SelectedData != null) return;

            if (!EqualityComparer<TEntity>.Default.Equals(ActionContext.SelectedData, newItem))
            {

```



```

        ActionContext.SelectedData = newItem;D
        StateHasChanged();D
    }D
}D
D
protected string FormatSelectedRowStyle(TEntity item, int index)D
{D
    return item == ActionContext.SelectedDataD
        ? "background-color: var(--mud-palette-action-disabled-background) !important; color:
var(--mud-palette-text-primary) !important; font-weight: 500;"D
        : string.Empty;D
}D
D
public Task ReloadServerDataAsync() => _grid != null ? _grid.ReloadServerData() :
Task.CompletedTask;D
D
public void Dispose()D
{D
    if (ActionContext != null)D
    {D
        ActionContext.OnContextUpdated -= StateHasChanged;D
        ActionContext.OnRefreshRequested -= HandleRefreshRequested;D
    }D
}D
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Toolbar/IHasSelectedData.cs">
namespace Promatis.Net.UI.Components.Toolbar;D
D
/// <summary>D
/// B4=C,2CT@D 0C`LCÔKC' 8CÔBCT@DD5C"A-CÄ0D :CT@ CD;Dò 2C,,7D40C'8Ct0D$>D >C"Â :CäBCä@D'< CÔ5Cä1DT>CD8CÄ> D
/// D 8CÔED >CÔ8Ct8D >C$0D$L DD>C=D AD$@Cä:C,ôCCt;C 8 CÔ>C'CDt0D$L C,,<CôCC'LD K CÔ5D 5D 8D >C$:C,i
/// </summary>D
public interface IHasSelectedData<TEntity> where TEntity : classD
{D
    TEntity? SelectedData { get; set; }D
    Action? OnContextUpdated { get; set; }D
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Toolbar/IHasToolbar.cs">
namespace Promatis.Net.UI.Components.Toolbar;D
D
/// <summary>D
/// A,,=D$5D DCT9D Ô<C @C=D 4C'0 Cä1C'0D BCT9, C,,<CTND"8DR :Cä<C =CD=D'9 D$CC'1C @ D4?D 0C$;CT=C,,0D
/// </summary>D
public interface IHasToolbarD
{D
    ToolbarPosition Position { get; set; }D
    bool IsToolbarVisible { get; set; }D
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Toolbar/ToolbarActionContext.cs">
using Promatis.Net.UI.Components.Workspace;D
D
namespace Promatis.Net.UI.Components.Toolbar;D
D
/// <summary>D
/// A4;Cä1C ;DÄ=D'9 C=D>CÔBCT:D B D4?D 0C$;CT=C,,0 C=D>CÄ0CÔ4CÔ>C' ?C =CT;DÄN (D$CC'1C @Cä<).D
/// Aô>C'=CäAD$LDä 0C AD$@C 3C,,@Cä2C = CäB C=D>CÔ:D 5D$=D'E D ?CäACä1Cä2 Cô>C'CDt5CÔ8Dò 4C =CÔKDR 8 D$8Cô>C" 2
/// </summary>D
public abstract class ToolbarActionContext<TEntity> : WorkspaceActionContext, IHasToolbar,
IHasSelectedData<TEntity>D
    where TEntity : classD
{D
    // B 5C ;C,,7C FC,,0 C=D>CÔBD 0C=BC "† 5FööÆ& -
    public ToolbarPosition Position { get; set; } = ToolbarPosition.Top;D
    public bool IsToolbarVisible { get; set; } = true;D
D
    private TEntity? _selectedData;D
D
    /// <summary>D
    /// B$5C=D"8C' 2D'4CT;CT=CÔKC' >C JCT:D" =C ECä;D BCR ,,AD$@Cä:C 3D 8CD0, C$5D$:C 4CT@CT2C Ä 4C BDt8C0
    /// Aô@C, 8Ct<CT=CT=C,,8 C 2D$>CÄ0D$8Dt5D :C, ?CT@CTADt8D$KCS$0CTB CD>D BD4?CÔ>D BDÄ 1C 7Cä2D'E C=D=Cä?Cä: C

```

```

/// </summary>
public TEntity? SelectedData
{
    get => _selectedData;
    set
    {
        if (!EqualityComparer<TEntity>.Default.Equals(_selectedData, value))
        {
            _selectedData = value;
            RecalculateButtonStates();
            NotifyUpdate(); // BD>D AC,,@D45D" <C4=Cä2CT=CÔCDâ ?CT@CT@C,,ACä2C=C D$CC´1C @C 8 CÔ>CDAC$5D$>
        }
    }
}

// B "A "A,, 'AT!A Bò A !B$ Aä A A$ AD AÄ B "A, AÔ Aô A-
public bool IsCreateVisible { get; set; } = true;
public bool IsEditVisible { get; set; } = true;
public bool IsDeleteVisible { get; set; } = true;

// AD AÔ AÄ Bt B A,, B B 'AT" AD B "B4 AÔ B "A, AÔ Aô A¢ ...&V BÔöæÇ´.

/// <summary>
/// B >Ct4C =C,,5 CÔ> D4<Cä;Dt0C08Dâ 4CäAD$CCô=Câ 2D 5C44C ,,5D ;C, :CÔ>Cô:C 2C,,4C,,<C ´í
/// </summary>
public virtual bool IsCreateEnabled { get; set; } = true;

/// <summary>
/// A,,7CÄ5C05C08CR 4CäAD$CCô=CâÂ 5D ;C, >C JCT:D" 2D´1D 0C0 CÔ@CT4C,,:C B C 8Ct=CTA-C´>C48C=8 D 0Ct@THC
/// </summary>
public virtual bool IsEditEnabled => SelectedData != null && CanEditNode(SelectedData);

/// <summary>
/// B44C ;CT=C,,5 CD>D BD4?CÔ>, CTAC´8 Cä1D=C B C$KC @C = A, ?D 5CD8C=0D" 1C,,7C05D Ô;Cä3C,,:C, @C 7D 5D,,0C
/// </summary>
public virtual bool IsDeleteEnabled => SelectedData != null && CanDeleteNode(SelectedData);

/// <summary>
/// A=C´;CT:Dd8Dò @C AD,,8D 5C0=D´E (C=0D BCä<CÔKDR´ :CÔ>Cô>C¢ BD4;C 0D 0 (Cô5Dt0D$L, DÔ:D ?Cä@D"Â 8CÄ?Cä
/// </summary>
private readonly List<ToolbarCustomAction> _customActions = [];

/// <summary>
/// A 5Ct>Cô0D =C 0 C=C´;CT:Dd8Dò :C AD$>CÄ=D´E CD5C"AD$2C,,9, CD>D BD4?CÔ0Dò BD4;C 0D C D$>C´LC=C> CD;Dò
/// A,,AC=;DäGC 5D" AC´CDt0C"=D4N CÔ>D GD2 8C´8 CäGC,,AD$:D2 AC08D :C :CÔ>Cô>C¢ 8Ct2C05.D
/// </summary>
public IReadOnlyCollection<ToolbarCustomAction> CustomActions => _customActions.AsReadOnly();

protected ToolbarActionContext()
{
    // Aô> D4<Cä;Dt0C08Dâ =C AD$@C 8C$0CT< C=CÄ?C :D$=D4N C45Cä<CTBD 8Dâ 4C´O D$CC´1C @CÔKDR AD$@C =C,,FD
    WorkspaceHeight = "100%";
    TopZoneHeight = "auto";
    BottomZoneHeight = "auto";
    LeftZoneWidth = "220px";
    RightZoneWidth = "220px";
}

/// <summary>
/// At0D"8D"5C0=D´9 CÄ5D$>CB 4C´O C 5Ct>Cô0D =Cä3Cä 4Cä1C 2C´5C08Dò :C AD$>CÄ=D´E C=Cä?Cä: C,,7 C=CÔAD$@
/// A40D 0C0BC,,@D45D" f -ÄÖf 7B ?Cä2CT4CT=C,,5 C, 7C IC,,IC 5D" AC,,AD$5CÄC CäB CDCC ;C,,@Cä2C =C,,O C,,4CT=D$8
/// </summary>
protected void AddCustomAction(ToolbarCustomAction action)
{
    if (action == null) throw new ArgumentNullException(nameof(action));

    // At0D"8D$0 CäB C05C$=C,,<C BCT;DÄ=CäAD$8 D 0Ct@C 1CäBDt8C=0: Id C=Cä?Cä: CÔ0 Cä4CÔ>C´ AD$@C =C,,FCR
    if (_customActions.Any(a => a.Id == action.Id))
    {
        throw new InvalidOperationException(
            $"A=0D BCä<CÔ0Dò :CÔ>Cô:C A C,,4CT=D$8DD8C=0D$>D >CÄ w¶ 7F-öää-Gòr CCd5 Ct0D 5C48D BD 8D >C$0
        );
    }

    _customActions.Add(action);
}

```

```

/// <summary>
/// A0@Cä3D 0CÄ<C0>CR 8Ct<CT=CT=C,,5 CD>D BD4?C0>D BC, :C AD$>CÄ=Cä9 C=Cä?C8 C0> CTQ Id C,,7 C 8Ct=CTA-C'
/// </summary>
public void SetActionEnabled(string actionId, bool isEnabled)
{
    // A,,ICT< C$=D4BD 8 Ct0C@D'BCä3Cä AC08D :C
    ToolbarCustomAction? action = _customActions.Find(a => a.Id == actionId);
    if (action != null && action.IsEnabled != isEnabled)
    {
        action.IsEnabled = isEnabled;
        NotifyUpdate(); // B 5C :D$8C$=Cä >C =Cä2C'0CT< UID
    }
}

// B A B$ A$ A / BD A',B$ A &A,,/ B$ A At A&A,, B #A A0 B$ A0# B #B" Aä!B$ D

public override void HandleGlobalEntityCommit(object? state, object? entity)
{
    base.HandleGlobalEntityCommit(state, entity);

    if (entity == null) return;

    Type entityType = entity.GetType();
    if (entityType.BaseType != null && entityType.Namespace == "Castle.Proxies")
    {
        entityType = entityType.BaseType;
    }

    if (typeof(TEntity).IsAssignableFrom(entityType))
    {
        string stateStr = state?.ToString() ?? string.Empty;

        if (stateStr == "Deleted" || stateStr == "SoftDeleted")
        {
            if (SelectedData != null && ReferenceEquals(SelectedData, entity))
            {
                SelectedData = null;
            }
        }

        RequestRefresh();
    }
}

// AD AÄ A0 B' A0 AT A,, A "B²0%B4 A, ,, CT@CT>C0@CT4CT;D0ND$AD0 2 C0@C,,:C'0CD=D'E C=C0BCT:D BC E)
protected virtual bool CanEditNode(TEntity node) => true;
protected virtual bool CanDeleteNode(TEntity node) => true;

/// <summary>
/// A$=D4BD 5C0=C,,9 DTCC 6C,,7C05C0=Cä3Cä FC,,:C'0 CD;D0 ?D >C$5CD5C08D0 4Cä?Cä;C08D$5C'LC0KDR @C ADt5D$>C
/// </summary>
protected virtual void RecalculateButtonStates()
{
}
}
</file>

<file path="Promatis.Net.UI/Components/Toolbar/ToolbarCustomAction.cs">
using MudBlazor;
namespace Promatis.Net.UI.Components.Toolbar;
/// <summary>
/// A4;Cä1C ;DÄ=D'9 C0;C BDD>D <CT=C0KC' :C'0D A CD;D0 4CT:C'0D 0D$8C$=Cä3Cä >C08D 0C08D0 @C AD,,8D 5C0=D'E (C
/// Aä1CTAC05Dt8C$0CTB Dd2CTBCä2Cä5 C, 2C,,7D40C'LC0>CR 5CD8C0>Cä1D 0Ct8CR 2Cä 2D 5DR ?D 8C;C 4C0KDR <Cä4D4;D
/// </summary>
public class ToolbarCustomAction
{
    /// <summary>
    /// B4=C,,:C ;DÄ=D'9 D BD >C=C$KC' 8CD5C0BC,,DC,,:C BCä@ CD5C"AD$2C,,0 (C00C0@C,,<CT@, "excel_export").
    /// A,,AC0>C'LCtCCTBD 0 CD;D0 ?D >C4@C <CÄ=Cä3Cä 8Ct<CT=CT=C,,0 D >D BCäOC08D0 :C0>C0:C, ,,2C;DäGCT=C,,5/C$K
    /// </summary>
    public required string Id { get; init; }

    /// <summary>
    /// B$5C=AD" ,,=C 8CÄ5C0>C$0C08CR'Ä >D$>C @C 6C 5CÄKC' =C :C0>C0:CR BD4;C 0D 0.D

```

```

    /// </summary>
    public required string Title { get; init; }
}

    /// <summary>
    /// A4@C DC,,GCTAC0D0 8C0>C0:C xVD&Æ |÷" ,,=C ?D 8CÄ5D Â -6öç2äÖ FW&- Ääf-ÆÆVBäF÷væÆÖ B'í
    /// </summary>
    public string? Icon { get; set; }
}

    /// <summary>
    /// Bd2CTBCä2C O D ECT<C :C0>C0:C, 8Cr AC,,AD$5CÄ=Cä9 C00C'8D$@D² xVD&Æ |÷" ... &-Ö ''Â 7V66W72Â W'&÷"Â v &
    /// </summary>
    public Color Color { get; set; } = Color.Default;
}

    /// <summary>
    /// B BC,,;DÄ >D$>C @C 6CT=C,,O C0=Cä?C08 (Filled, Outlined, Text). A0> D4<Cä;Dt0C08Dâ r 7C ;C,,BC O (Filled
    /// </summary>
    public Variant Variant { get; set; } = Variant.Filled;
}

    /// <summary>
    /// BD;C 3 C$8CD8CÄ>D BC, :C0>C0:C, =C BD4;C 0D 5. ATAC'8 false - C0=Cä?C00 C0>C'=CäAD$LDâ 8D :C'NDt0CTB
    /// </summary>
    public bool IsVisible { get; set; } = true;
}

    /// <summary>
    /// BD;C 3 CD>D BD4?C0>D BC, :C0>C0:C, 4C'0 C0;C,,:C ,,0C0BC,,2C00/Ct0D$CD,,5C00).
    /// </summary>
    public bool IsEnabled { get; set; } = true;
}

    /// <summary>
    /// B AD';C00 C00 D 5C ;DÄ=D'9 C AC,,DT@Cä=C0KC' <CTBCä4 C 8Ct=CTA-C'>C48C08 C0>C0BCT:D BC Â :CäBCä@D'9 C
    /// </summary>
    public required Func<Task> OnExecute { get; init; }
}
</file>

<file path="Promatis.Net.UI/Components/Toolbar/ToolbarPosition.cs">
namespace Promatis.Net.UI.Components.Toolbar;
{
    public enum ToolbarPosition
    {
        Top,
        Bottom,
        Left,
        Right
    }
}
</file>

<file path="Promatis.Net.UI/Components/Toolbar/UiToolbar.razor">
@typeparam TEntity where TEntity : class
{
    @if (ActionContext != null && ActionContext.IsToolbarVisible)
    {
        @if (IsVertical)
        {
            <div class="d-flex flex-column align-stretch justify-space-between bg-light pa-3 rounded shadow-sm h-100"
                style="@($width: {ZoneSize}; min-width: {ZoneSize}; max-width: {ZoneSize};)">
                <div class="d-flex flex-column gap-2">
                    <MudText Typo="Typo.subtitle2" Class="font-weight-bold text-center border-bottom pb-2 mb-2">
                        @ActionContext.PageTitle
                    </MudText>
                    @RenderButtonsCollection
                </div>
            </div>
        }
        else
        {
            <div class="d-flex align-center justify-space-between bg-light pa-2 rounded shadow-sm w-100"
                style="@($height: {ZoneSize}; min-height: {ZoneSize}; max-height: {ZoneSize};)">
                <div class="d-flex align-center gap-2 flex-wrap">
                    @RenderButtonsCollection
                </div>
            </div>
        }
    }
}

```

</file>

<file path="Promatis.Net.UI/Components/Toolbar/UiToolbar.razor.cs">

using Microsoft.AspNetCore.Components;ð

using Microsoft.AspNetCore.Components.Web;ð

using MudBlazor;ð

using Promatis.Net.UI.Components.Tree;ð

ð

namespace Promatis.Net.UI.Components.Toolbar;ð

ð

public partial class UiToolbar<TEntity> : ComponentBase where TEntity : classð

{ð

 [Parameter] public ToolbarActionContext<TEntity> ActionContext { get; set; } = null!;ð

 [Parameter] public RenderFragment? AdditionalContent { get; set; }ð

ð

 [Parameter] public EventCallback OnCreateTriggered { get; set; }ð

 [Parameter] public EventCallback<TEntity> OnEditTriggered { get; set; }ð

 [Parameter] public EventCallback<TEntity> OnDeleteTriggered { get; set; }ð

 [Parameter] public EventCallback<TEntity> OnCreateChildTriggered { get; set; }ð

ð

 protected bool IsVertical => ActionContext?.Position == ToolbarPosition.Left ||

ActionContext?.Position == ToolbarPosition.Right;ð

ð

 protected string ZoneSize => ActionContext?.Position switchð

 {ð

 ToolbarPosition.Left => ActionContext.LeftZoneWidth,ð

 ToolbarPosition.Right => ActionContext.RightZoneWidth,ð

 ToolbarPosition.Top => ActionContext.TopZoneHeight,ð

 ToolbarPosition.Bottom => ActionContext.BottomZoneHeight,ð

 _ => "auto"ð

 };ð

ð

 protected override void OnInitialized()ð

 {ð

 base.OnInitialized();ð

ð

 if (ActionContext == null)ð

 {ð

 throw new ArgumentNullException(nameof(ActionContext),ð

 \$"Aꝑ>CĀ?Cā=CT=D" ¶æ ÖVöb...V•FööÆ& #ÄDVçF–G"â–Ö BD 5C CCTB Cä1Dö7C BCT;DÄ=Cä9 Cö5D 5CD0Dt8 Cö0D

 }ð

ð

 ActionContext.OnContextUpdated = StateHasChanged;ð

 }ð

ð

 protected async Task OnCreateClick(MouseEventArgs e)ð

 {ð

 if (OnCreateTriggered.HasDelegate) await OnCreateTriggered.InvokeAsync();ð

 }ð

ð

 protected async Task OnCreateChildClick(MouseEventArgs e)ð

 {ð

 if (OnCreateChildTriggered.HasDelegate && ActionContext.SelectedData != null)ð

 await OnCreateChildTriggered.InvokeAsync(ActionContext.SelectedData);ð

 }ð

ð

 protected async Task OnEditClick(MouseEventArgs e)ð

 {ð

 if (OnEditTriggered.HasDelegate && ActionContext.SelectedData != null)ð

 await OnEditTriggered.InvokeAsync(ActionContext.SelectedData);ð

 }ð

ð

 protected async Task OnDeleteClick(MouseEventArgs e)ð

 {ð

 if (OnDeleteTriggered.HasDelegate && ActionContext.SelectedData != null)ð

 await OnDeleteTriggered.InvokeAsync(ActionContext.SelectedData);ð

 }ð

ð

 /// <summary>ð

 /// AD8C00CÄ8Dt5D :C,,9 DD@C 3CÄ5C0B CäBD 8D >C\$:C, :C0>C0>C4Â 8C0:C ?D CC´8D >C\$0C0=D´9 C" 220:C´0D ACR 1

 /// </summary>ð

 private RenderFragment RenderButtonsCollection => builder =>ð

 {ð

 bool isTreeMode = ActionContext.GetType().BaseType != nullð

 && ActionContext.GetType().BaseType!.IsGenericTypeð

 && ActionContext.GetType().BaseType!.GetGenericTypeDefinition() ==

typeof(TreeActionContext<>);ð

```

ð
    bool isCreateChildVisible = false;ð
    bool isCreateChildEnabled = false;ð
ð
    // ATAC`8 D 0C0BC 9CÂ >C0@CT4CT;C,,; DtBCâ ?CT@CT4 C00CÂ8 C= >C0BCT:D B CD5D 5C$0 - C,,7C$;CT:C 5CÂ <CT
    if (isTreeMode)ð
    {ð
        dynamic? treeContext = ActionContext;ð
        if (treeContext != null)ð
        {ð
            isCreateChildVisible = treeContext.IsCreateChildVisible;ð
            isCreateChildEnabled = treeContext.IsCreateChildEnabled;ð
        }ð
    }ð
ð
    int seq = 0;ð
ð
    if (ActionContext.IsCreateVisible)ð
    {ð
        builder.OpenComponent<MudButton>(seq++);ð
        builder.AddAttribute(seq++, "Variant", Variant.Filled);ð
        builder.AddAttribute(seq++, "Color", Color.Primary);ð
        builder.AddAttribute(seq++, "Size", Size.Small);ð
        builder.AddAttribute(seq++, "Disabled", !ActionContext.IsCreateEnabled);ð
        builder.AddAttribute(seq++, "OnClick",
EventCallback.Factory.Create<MouseEventArgs>(this, OnCreateClick));ð
        builder.AddAttribute(seq++, "ChildContent", (RenderFragment)(b => b.AddContent(seq++,
"B >Ct4C BDÂ""'""½
        builder.CloseComponent();ð
    }ð
ð
    // B AÔ AT A= Aä A= AD B A$ :ð
    if (isCreateChildVisible)ð
    {ð
        builder.OpenComponent<MudButton>(seq++);ð
        builder.AddAttribute(seq++, "Variant", Variant.Outlined);ð
        builder.AddAttribute(seq++, "Color", Color.Primary);ð
        builder.AddAttribute(seq++, "Size", Size.Small);ð
        builder.AddAttribute(seq++, "Disabled", !isCreateChildEnabled);ð
        builder.AddAttribute(seq++, "OnClick",
EventCallback.Factory.Create<MouseEventArgs>(this, OnCreateChildClick));ð
        builder.AddAttribute(seq++, "ChildContent", (RenderFragment)(b => b.AddContent(seq++, "AD>C 0C$8D
C0>CDCct5C²""'""½
        builder.CloseComponent();ð
    }ð
ð
    if (ActionContext.IsEditVisible)ð
    {ð
        builder.OpenComponent<MudButton>(seq++);ð
        builder.AddAttribute(seq++, "Variant", Variant.Outlined);ð
        builder.AddAttribute(seq++, "Color", Color.Info);ð
        builder.AddAttribute(seq++, "Size", Size.Small);ð
        builder.AddAttribute(seq++, "Disabled", !ActionContext.IsEditEnabled);ð
        builder.AddAttribute(seq++, "OnClick",
EventCallback.Factory.Create<MouseEventArgs>(this, OnEditClick));ð
        builder.AddAttribute(seq++, "ChildContent", (RenderFragment)(b => b.AddContent(seq++,
"A,,7CÂ5C08D$L"))));ð
        builder.CloseComponent();ð
    }ð
ð
    if (ActionContext.IsDeleteVisible)ð
    {ð
        builder.OpenComponent<MudButton>(seq++);ð
        builder.AddAttribute(seq++, "Variant", Variant.Outlined);ð
        builder.AddAttribute(seq++, "Color", Color.Error);ð
        builder.AddAttribute(seq++, "Size", Size.Small);ð
        builder.AddAttribute(seq++, "Disabled", !ActionContext.IsDeleteEnabled);ð
        builder.AddAttribute(seq++, "OnClick",
EventCallback.Factory.Create<MouseEventArgs>(this, OnDeleteClick));ð
        builder.AddAttribute(seq++, "ChildContent", (RenderFragment)(b => b.AddContent(seq++,
"B44C ;C,,BDÂ""'""½
        builder.CloseComponent();ð
    }ð
ð
    if (AdditionalContent != null)ð
    {ð

```

```

        if (ActionContext.IsCreateVisible || isCreateChildVisible ||
ActionContext.IsEditVisible || ActionContext.IsDeleteVisible)
        {
            builder.OpenComponent<MudDivider>(seq++);
            builder.AddAttribute(seq++, "Vertical", !IsVertical);
            builder.AddAttribute(seq++, "FlexItem", true);
            builder.AddAttribute(seq++, "Class", "mx-2 my-1");
            builder.CloseComponent();
        }
        builder.AddContent(seq++, AdditionalContent);
    }

    if (ActionContext.CustomActions.Any(a => a.IsVisible) &&
        (ActionContext.IsCreateVisible || isCreateChildVisible || ActionContext.IsEditVisible
|| ActionContext.IsDeleteVisible || AdditionalContent != null))
    {
        builder.OpenComponent<MudDivider>(seq++);
        builder.AddAttribute(seq++, "Vertical", !IsVertical);
        builder.AddAttribute(seq++, "FlexItem", true);
        builder.AddAttribute(seq++, "Class", "mx-2 my-1");
        builder.CloseComponent();
    }

    foreach (ToolbarCustomAction action in ActionContext.CustomActions.Where(a => a.IsVisible))
    {
        builder.OpenComponent<MudButton>(seq++);
        builder.AddAttribute(seq++, "Variant", action.Variant);
        builder.AddAttribute(seq++, "Color", action.Color);
        builder.AddAttribute(seq++, "Size", Size.Small);
        builder.AddAttribute(seq++, "StartIcon", action.Icon);
        builder.AddAttribute(seq++, "Disabled", !action.IsEnabled);
        builder.AddAttribute(seq++, "OnClick",
EventCallback.Factory.Create<MouseEventArgs>(this, action.OnExecute));
        builder.AddAttribute(seq++, "ChildContent", (RenderFragment)(b => b.AddContent(seq++,
action.Title)));
        builder.CloseComponent();
    }
};
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Tree/TreeActionContext.cs">
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using MudBlazor;
using Promatis.Net.Domain.Interface;
using Promatis.Net.UI.Components.Toolbar;

namespace Promatis.Net.UI.Components.Tree;

/// <summary>
/// A 0Ct>C$KC' 8CT@C @DT8Dt5D :C,,9 (CD@CT2Cä2C,,4C0KC'' :Cä=D$5C=AD" CC0@C 2C'5C08D0 ?C'0D$DCä@CÄK.
/// A0>C'=CäAD$LDä 8C0:C ?D CC'8D CCTB D >D BCä0C08CR 1C 7Cä2D'E C=Cä?Cä: C, DCä:D4AC 4C'O MudTreeView.
/// </summary>
/// <typeparam name="TEntity">B$8C0 4D 5C$>C$8CD=Cä9 D CD'=CäAD$8, D 5C ;C,,7D4ND"5C' •G&VTæ0FSÄ÷G- W & Óí
public abstract class TreeActionContext<TEntity> : ToolbarActionContext<TEntity>
    where TEntity : class, ITreeNode<TEntity>, IDomainObjectHasKey<Guid>
{
    // B BC BC,,GCTAC=0D0 =C AD$@Cä9C=0 C$8CD8CÄ>D BC, 4CäGCT@C05C' :C0>C0:C•
    public bool IsCreateChildVisible { get; set; } = true;

    // AD8C00CÄ8Dt5D :C,,9 D 0D GCTB CD>D BD4?C0>D BC, :C0>C0:C, 4Cä1C 2C'5C08D0 ?Cä4D47C'0D
    public virtual bool IsCreateChildEnabled => SelectedData != null;

    protected IDialogService DialogService => ScopedProvider.GetRequiredService<IDialogService>();
    protected IValidator<TEntity> GlobalValidator =>
ScopedProvider.GetRequiredService<IValidator<TEntity>>();

    /// <summary>
    /// A0;C BDD>D <CT=C0KC' 1D >C=5D 4C =C0KDRÂ 0CD0C0BC,,@Cä2C =C0KC' ?Cä4 C,,5D 0D EC,,GCTAC=8CR AD$@D4:D$CD
    /// </summary>
    public UiDataBroker<TEntity> DataBroker { get; } = new();

    protected TreeActionContext() : base()
    {
        // A,,!A0 A A' A0 : A05D 5C0>D 8CÄ :Cä<C =CD=D'9 D$CC'1C @ C00C$5D E (C4>D 8Ct>C0BC ;DÄ=C 0 C00C05C'L

```

```

        Position = ToolbarPosition.Top;
        TopZoneHeight = "auto";
    }

    // B EC`>CÔKC$0CT< C`5C$CDâ 7Cä=D2Â BC : C`0C$ BD4;C 0D CD,,5C² =C 2CT@D]
    IsLeftZoneCollapsed = true;
    LeftZoneWidth = "0px";
}

    // Aô> D4<Cä;Dt0C08Dâ 4CT@CT2Câ 8C08Dd8C ;C,,7C,,@D45D$ADò 2 D 5Cd8CÄ5 D 0C >D$K D >C05D 0D$8C$=Cä9 C0
    DataBroker.ConfigureInMemoryMode();
}

    /// <summary>
    /// Aä1Dô7C BCT;DÄ=D`9 CD;Dò @CT0C`8Ct0Dd8C, <CTBCä4 Cò@Cä3D 5C$0 Aä B20:D0HC â
    /// A$KCTKC$0CTBD 0 C 2D$>CÄ0D$8Dt5D :C, :Cä<Cò>C05C0BCä< TreePage Cò@C, ?CT@C$8Dt=Cä9 CäBD 8D >C$:CRÍ
    /// </summary>
    public abstract Task InitializeInMemoryTreeAsync();

    /// <summary>
    /// A 2D$>CÄ0D$8Dt5D :C,,9 C05D 5DT2C B C`>CÄ<C,,BCä2 B #A C00 D4@Cä2C05 C,,5D 0D EC,,GCTAC`>C4> Dô4D 0.D
    /// Aä1CTAC05Dt8C$0CTB D :C$>Ct=D4N real-time D 8C0ED >C08Ct0Dd8Dâ AC$0Ct5C` 2CTBCä: C" At# Ct0 0 CÄA.D
    /// </summary>
    public override void HandleGlobalEntityCommit(object? state, object? entity)
    {
        if (entity == null) return;

        Type entityType = entity.GetType();
        if (entityType.BaseType != null && entityType.Namespace == "Castle.Proxies")
        {
            entityType = entityType.BaseType;
        }

        // ATAC`8 D$@C =Ct0C=FC,,0 C,,7 C MC=5C04C 7C BD >C0CC`0 C00D, 4D 5C$>C$8CD=D`9 D$8Cò 4C =C0KD]
        if (typeof(TEntity).IsAssignableFrom(entityType))
        {
            string stateStr = state?.ToString() ?? string.Empty;

            // A08C00CT< Aä B204C$8Cd>C$ 1D >C`5D 0 C05D 5D GC,,BC BDÂ AC$0Ct8 CÄ0D AC,,2C 2 C00CÄ0D$8D
            DataBroker.ApplyIncrementalOzuDelta(stateStr, (TEntity)entity);

            // A$KCTKC$0CT< C 0Ct>C$KCR ?D 0C$8C`0 D 1D >D 0 DD>C`CD 0 (CTAC`8 C$KCD5C`5C0=C 0 C$5D$:C 1D`;C
            base.HandleGlobalEntityCommit(state, entity);

            // A08C03C 5CÄ :Cä<Cò>C05C0B TreePage Cò>C`=CäAD$LDâ ?CT@CT@C,,ACä2C BDÂ 4CT@CT2Câ =C MC`@C =C]
            RequestRefresh();
        }
    }

    protected override void RecalculateButtonStates()
    {
        base.RecalculateButtonStates();
        NotifyUpdate(); // B 5C :D$8C$=Cä >C =Cä2C`0CT< C :D$8C$=CäAD$L C`=Cä?C`8 "AD>C 0C$8D$L Cò>CDCCt5C²"
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Tree/TreePage.razor">
@typeparam TEntity where TEntity : class, ITreeNode<TEntity>, IDomainObjectHasKey<Guid>
{
    <div class="tree-page-container w-100 overflow-auto pa-2"
        style="height: calc(100vh - 240px); min-height: calc(100vh - 240px); max-height: calc(100vh - 240px);">
        @if (!_isWarmingUp)
        {
            <div class="d-flex flex-column align-center justify-center h-100 w-100 gap-2" style="min-height: 200px;">
                <MudProgressCircular Color="Color.Primary" Indeterminate="true" />
                <MudText Typo="Typo.body2" Color="Color.Secondary">At0C4@D47C`0 D BD CC`BD4@D² 4CT@CT2C âäâÄô×VEF
            </div>
        }
        else
        {
            <MudTreeView T="TEntity"
                Items="@_rootTreeViewItems"
                SelectedValue="@ActionContext.SelectedData"
                SelectedValueChanged="@((TEntity? val) => OnSelectedNodeChanged(val))">

```



```

        Hover="true"@
        Dense="true"@
        Color="Color.Primary"@
        Class="w-100">@
    <ItemTemplate>@
        @* BD8C²AC,,@D45CÂ :C =Cä=C,,GCÔKC' HC 1C'>Cò Cct;C 4C'0 MudBlazor 9.4+ *@@
        <MudTreeViewItem Value="@context.Value"@
            Items="@context.Children"@
            Text="@GetNodeText(context.Value!)"@
            Expanded="true" />@
    </ItemTemplate>@
</MudTreeView>@
}
</div>
</file>

<file path="Promatis.Net.UI/Components/Tree/TreePage.razor.cs">
using Microsoft.AspNetCore.Components;@
using MudBlazor;@
using Promatis.Net.Domain.Interface;@
@
namespace Promatis.Net.UI.Components.Tree;@
@
public partial class TreePage<TEntity> : ComponentBase, IDisposable@
    where TEntity : class, ITreeNode<TEntity>, IDomainObjectHasKey<Guid>@
{
    [CascadingParameter] protected TreeActionContext<TEntity> ActionContext { get; set; } = null!;@
    @
    [Parameter] public Func<TEntity, string>? NodeTextSelector { get; set; }@
    @
    [Inject]@
    public IServiceProvider SystemServiceProvider { get; set; } = null!;@
    @
    // A²>C';CT:Dd8Dò >C 5D BCä: CD;Dò xVEG&VUF-Wr 'äm
    protected List<TreeItemData<TEntity>> _rootTreeViewItems = [];@
    @
    // A² Bâ'AT Aä AÄ AÄ Aô": A,,7CÔ0Dt0C'LCÔ> C$KD BC 2C'0CT< C" G'VRâ
    // B BD 0CÔ8Dd0 CÄ3CÔ>C$5CÔ=Câ ?Cä:C 6CTB D ?C,,=CÔ5D Â 0 CÄ5CÔN Cò@C,,;Cä6CT=C,,0 CäAD$0CÔ5D$ADò ?Cä;CÔ>D B
    protected bool _isWarmingUp = true;@
    @
    protected override void OnInitialized()@
    {
        base.OnInitialized();@
        @
        if (ActionContext == null)@
        {
            throw new ArgumentNullException(nameof(ActionContext),@
                $"A²>CÄ?Cä=CT=D" ¶æ ÖVöb...G&VU vSÄDVçF-G"â-ò BD 5C CCTB CÔ0C'8Dt8Dò ¶æ ÖVöb...G&VT 7F-öä6öçFW‡C
            )@
        }
        @
        // AÄ0C48Dò ?D 8C$0Ct:Cfç >D$4C 5CÂ :Cä=D$5C²AD$C CD5D 5C$0 Cò@Cä2C 9CD5D BCT:D4ICT9 Cd8C$>C' ACTAD
        ActionContext.ScopedProvider = SystemServiceProvider;@
        @
        ActionContext.OnContextUpdated += HandleContextUpdated;@
        ActionContext.OnRefreshRequested += HandleRefreshRequested;@
    }
    @
    /// <summary>@
    /// A$KctKC$0CTBD 0 C 2D$>CÄ0D$8Dt5D :C, !B$ Aä Aâ Aä!A' D$>C4>, C²0Cç &Æ |÷" >D$@C,,ACä2C ; CòCD BCä9 C
    /// Aô>C'=CäAD$LDâ 8D :C'NDt0CTB DD@C,,7D² <CT=Dâ ?D 8 CÔ5D 5DT>CD5 Cò> C$:C'0CD:C < [6].@
    /// </summary>@
    protected override async Task OnAfterRenderAsync(bool firstRender)@
    {
        await base.OnAfterRenderAsync(firstRender);@
        @
        if (firstRender)@
        {
            // At0CòCD :C 5CÂ ?D >C4@CT2 C AC,,=DT@Cä=CÔ>, CäAC$>C >Cd4C 0 UI-Cô>D$>C-
            await WarmupTreeDataAsync();@
        }
    }
    @
    /// <summary>@
    /// Aô@Cä8Ct2Cä4C,,B C 5Ct>Cô0D =D4N DD>CÔ>C$CDâ 7C 3D Cct:D2 4C =CÔKDR 8Cr !B4 AB 2 Aä B2Ô:DÔH C @Cä:CT@C
    /// </summary>@
    private async Task WarmupTreeDataAsync()@

```

```

{
    try
    {
        // ATAC'8 Aä B20:D0H C @Cä:CT@C 5D"5 C0CD B, C$KC0>C'=D05CÄ EC,,B C¢ ACT@C$8D C C MC¤5C04C
        if (ActionContext.DataBroker.IsInMemoryMode && ActionContext.DataBroker.InMemoryItems
== null)
        {
            // A$Kct>C" CC'5D$0CTB C" ABÄ ?Cä:C T' :D CD$8D" ;CT3C¤8C' AC08C0=CT@
            await ActionContext.InitializeInMemoryTreeAsync();
        }

        // A$KC0>D 8CÄ BD06CT;D4N D 1Cä@C¤C C4@C DC AC$0Ct5C' BD'AD0G D47C'>C" 2 DD>C0>C$KC' ?CäBCä: Thr
        // DtBCä1D² ?CäBCä: CäBD 8D >C$:C, 2Cä>C ICR =CR 8D ?D'BD'2C ; CÄ8C¤@Cä07C 4CT@Cd5C¢
        await Task.Run(RebuildTreeGraph);
    }
    finally
    {
        _isWarmingUp = false;

        // A$>Ct2D 0D"0CT< D4?D 0C$;CT=C,,5 C" T'0?CäBCä: CD;D0 1CT7Cä?C AC0>C' ?CT@CT@C,,ACä2C¤8 C4>D$>C$>
        await InvokeAsync(StateHasChanged);
    }
}

/// <summary>
/// A KD BD > D >C 8D 0CTB C0;CäAC¤8C' :D0H C @Cä:CT@C 2 CD@CT2Cä2C,,4C0KC' 3D 0DB G&VT-FV0F F 7C Ó" <
/// </summary>
private void RebuildTreeGraph()
{
    if (!ActionContext.DataBroker.IsInMemoryMode || ActionContext.DataBroker.InMemoryItems ==
null)
    {
        _rootTreeViewItems = [];
        return;
    }

    List<TEntity> allItems = ActionContext.DataBroker.InMemoryItems;

    // 1. B >Ct4C 5CÄ 2D 5CÄ5C0=D4N D BD CC¤BD4@D2 4C'0 C00C¤>C0;CT=C,,0 CD>Dt5D =C,,E D0;CT<CT=D$>C-
    var childrenMap = allItems.ToDictionary(
        item => item.Id,
        _ => new List<TreeItemData<TEntity>>()
    );

    List<TreeItemData<TEntity>> roots = [];
    var rootsMap = new List<TEntity>();

    // 2. B =C GC ;C @C AC0@CT4CT;D05CÄ 4CäGCT@C08CR MC'5CÄ5C0BD² ?Cä AC08D :C < C" At#D
    foreach (TEntity item in allItems)
    {
        // B >Ct4C 5CÄ GC,,AD$CDä >C 5D BC¤C CD;D0 BCT:D4ICT3Cä CCT;C
        var currentWrapper = new TreeItemData<TEntity>
        {
            Value = item,
            Text = GetNodeText(item),
            Expanded = true
        };

        // A0@Cä2CT@D05CÄ =C ;C,,GC,,5 C$0C'8CD=Cä3Cä @Cä4C,,BCT;Dý
        if (item.ParentId.HasValue && item.ParentId.Value != Guid.Empty &&
childrenMap.ContainsKey(item.ParentId.Value))
        {
            childrenMap[item.ParentId.Value].Add(currentWrapper);
        }
        else
        {
            roots.Add(currentWrapper);
            rootsMap.Add(item);
        }
    }

    // 3. B 5C¤CD AC,,2C0> C,,;C, 8D$5D 0D$8C$=Cä AC$0CtKC$0CT< CD>Dt5D =C,,5 D ?C,,AC¤8 D >C JCT:D$0CÄ8 Tre
    // A05D 5CD0CT< D ?C,,AC¤8 C¤0C¢ ADD>D <C,,@Cä2C =C0KCR :Cä;C'5C¤FC,,8, DtBCä1D² xVD&Æ !÷" 'äB =C BC,,2C0
    foreach (var rootWrapper in roots)
    {
        PopulateChildrenTree(rootWrapper, childrenMap);
    }
}

```

```

    }
}

_rootTreeViewItems = roots;
}

/// <summary>
/// A$ACô>CÄ>C40D$5C'LCÔKC' <CTBCä4 CD;Dò @CT:D4@D 8C$=Cä3Cä AC$0CtKC$0C08Dò &V FöæÇ'Ô:Cä;C'5CFC,,9 Child
/// </summary>
private void PopulateChildrenTree(TreeItemData<TEntity> currentWrapper, Dictionary<Guid,
List<TreeItemData<TEntity>>> childrenMap)
{
    if (currentWrapper.Value == null) return;

    Guid currentId = currentWrapper.Value.Id;

    if (childrenMap.TryGetValue(currentId, out var directChildren) && directChildren.Count > 0)
    {
        // Aô@C,,AC$0C,,2C 5CÄ 3CäBCä2D'9 D ?C,,ACä: – MudBlazor C$=D4BD 8 D 0CÄ ?CäADt8D$0CTB Read-Only Has
        currentWrapper.Children = directChildren;

        // A,,4CT< C$3C'CC L Cö> Dd5Cö>Dt:CR 3D 0DD0
        foreach (var childWrapper in directChildren)
        {
            PopulateChildrenTree(childWrapper, childrenMap);
        }
    }
}

private void HandleContextUpdated()
{
    StateHasChanged();
}

private void HandleRefreshRequested()
{
    InvokeAsync(async () =>
    {
        // Aô@C, &V Â×F-ÖR BD 0C07C :Dd8DöE B #A Cö5D 5D GC,,BD'2C 5CÄ 3D 0DB 2 DD>Cö>C$>CÄ BC AC=5D
        await Task.Run(RebuildTreeGraph);
        StateHasChanged();
    });
}

protected void OnSelectedNodeChanged(TEntity? domainEntity)
{
    if (domainEntity == null && ActionContext.SelectedData != null) return;

    if (!EqualityComparer<TEntity>.Default.Equals(ActionContext.SelectedData, domainEntity))
    {
        ActionContext.SelectedData = domainEntity;
        StateHasChanged();
    }
}

protected string GetNodeText(TEntity node)
{
    if (node == null) return string.Empty;
    if (NodeTextSelector != null) return NodeTextSelector(node);

    var nameProp = typeof(TEntity).GetProperty("Name");
    return nameProp?.GetValue(node)?.ToString() ?? node.Id.ToString();
}

public void Dispose()
{
    if (ActionContext != null)
    {
        ActionContext.OnContextUpdated -= HandleContextUpdated;
        ActionContext.OnRefreshRequested -= HandleRefreshRequested;
    }
}
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspace/IWorkspaceActionContext.cs">
namespace Promatis.Net.UI.Components.Workspace;

```

```

Ð
/// <summary>Ð
/// A,,=D$5D DCT9D Ô<C @C¤5D 4C´O C 2D$>CÄ0D$8Dt5D :Cä3Cä ?Cä8D :C 8 D 5C48D BD 0Dd8C, T´Ô:Cä=D$5C¤AD$>C" 2
/// </summary>Ð
public interface IWorkspaceActionContextÐ
{Ð
}
</file>

<file path="Promatis.Net.UI/Components/Workspace/WorkspaceActionContext.cs">
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI.Components.Workspace;Ð
Ð
/// <summary>Ð
/// A 0Ct>C$K¢´ :Cä=D$5C¤AD" ;Dä1Cä9 D 0C >Dt5C´ >C ;C AD$8 (C$:C´0CD:C,´ 2 D 8D BCT<CRí
/// Aä1D´8C´ 7C00CÄ5C00D$5C´L CD;Dò <C05CÄ>D ECT<, D ?D 0C$>Dt=C,,:Cä2, CD0D,,1Cä@CD>C" 8 C4@C DC,,:Cä2.Ð
/// </summary>Ð
public abstract class WorkspaceActionContext : IWorkspaceActionContextÐ
{Ð
    /// <summary>Ð
    /// A0@Cä2C 9CD5D AC´CCd1 D$5C¤CD"5C´ 66÷ VB0ACTAD 8C, ?Cä;DÄ7Cä2C BCT;D0ä
    /// At0C0>C´=D05D$AD0 0C$BCä<C BC,,GCTAC¤8 C$8CtCC ;DÄ=D´< DT>C´AD$>CÄ ?D 8 D BC @D$5 D BD 0C08DdK.Ð
    /// </summary>Ð
    public IServiceProvider ScopedProvider { get; set; } = null!;Ð
Ð
    /// <summary>Ð
    /// B4=C,,:C ;DÄ=D´9 C,,4CT=D$8DD8C¤0D$>D @C 1CäGCT9 Cä1C´0D BC, 2 D 0C0BC 9CÄ5.Ð
    /// </summary>Ð
    public Guid WorkspaceId { get; init; } = Guid.NewGuid();Ð
Ð
    /// <summary>Ð
    /// B$5C¤AD$>C$K¢´ 7C 3Cä;Cä2Cä: D BD 0C08DdK (CäBCä1D 0Cd0CTBD O C00 D$CC´1C @CR 8C´8 C$:C´0CD:CR´í
    /// </summary>Ð
    public string PageTitle { get; set; } = "B 0C >Dt0D0 >C ;C AD$L";Ð
Ð
    /// <summary>Ð
    /// B 8D BCT<C0>CR 8CÄO CÄ>CDCC´O (Core, Mes, MesMDM), C¢ :CäBCä@Cä<D2 >D$=CäAC,,BD O D0:D 0C0í
    /// </summary>Ð
    public string ModuleName { get; init; } = string.Empty;Ð
Ð
    // B4 B A$ AT A,, A$ At#A BÄ B´ B "A,, AT A0 AD Aä A¤ Ð
    public int PaperElevation { get; set; } = 1;Ð
    public string PaperClass { get; set; } = "pa-4 d-flex flex-column flex-grow-1 w-100 h-100";Ð
Ð
    // B4 B A$ AT A,, A4 Aä AT"B AT A¤ B A !A %Ää B "A „!C$0CtL D v÷&·7 6U vR•
    public string WorkspaceHeight { get; set; } = "100%";Ð
    public string TopZoneHeight { get; set; } = "auto";Ð
    public string BottomZoneHeight { get; set; } = "auto";Ð
    public string LeftZoneWidth { get; set; } = "250px";Ð
    public string RightZoneWidth { get; set; } = "300px";Ð
Ð
    public bool IsTopZoneCollapsed { get; set; } = false;Ð
    public bool IsBottomZoneCollapsed { get; set; } = false;Ð
    public bool IsLeftZoneCollapsed { get; set; } = false;Ð
    public bool IsRightZoneCollapsed { get; set; } = false;Ð
Ð
    // B A B$ A$ B´ ÄÄ B " A$ A ÄÄ AD A"!B$ A,,/ (Blazor - A¤>C0BCT:D B)Ð
Ð
    /// <summary>Ð
    /// B >C KD$8CRÄ 2D´7D´2C 5CÄ>CR ?D 8 C,,7CÄ5C05C08C, AC$>C"AD$2 D 0CÄ>C4> C¤>C0BCT:D BC „=C ?D 8CÄ5D Ä 3
    /// A0@C,,=D44C,,BCT;DÄ=Cä 7C AD$0C$;D05D" &Æ ¦÷" ?CT@CT@C,,ACä2C BDÄ MC´5CÄ5C0BD²í
    /// </summary>Ð
    public Action? OnContextUpdated { get; set; }Ð
Ð
    /// <summary>Ð
    /// B$@C,,3C45D <C4=Cä2CT=C0>C4> Cä?Cä2CTICT=C,,O UI-D0;CT<CT=D$>C" > C$=D4BD 5C0=CT< C,,7CÄ5C05C08C, AD$5C
    /// </summary>Ð
    public void NotifyUpdate() => OnContextUpdated?.Invoke();Ð
Ð
    /// <summary>Ð
    /// A4;Cä1C ;DÄ=Cä5 D >C KD$8CRÄ 8Ct2CTIC ND"5CR 2C´>Cd5C0=D´5 C$8CtCC ;C,,7C BCä@D² „3D 8CBÄ 4CT@CT2Cä´ >
    /// B @C 1C BD´2C 5D" ?D 8 C¤>CÄ0C04C E C05D 5Ct0C4@D47C¤8 C,,;C, ?Cä AC,,3C00C´0CÄ 8Cr !B4 ABí
    /// </summary>Ð
    public event Action? OnRefreshRequested;Ð
Ð

```

```

    /// <summary>Ð
    /// A$ACô>CÄ>C40D$5C´LCÔKC´ <CTBCä4 CD;Dò 1CT7Cä?C ACô>C4> Ct0C0CD :C >C =Cä2C´5C08Dò 2C´>Cd5C0=Cä3Cä :C
    /// </summary>Ð
    protected void RequestRefresh() => OnRefreshRequested?.Invoke();Ð
Ð
    /// <summary>Ð
    /// A$KctKC$0CTBD 0 DT>C´AD$>CÄ v÷&·7 6U vR ?D 8 D4ACô5D,,=Cä< C¤>CÄ<C,,BCR ;Dä1Cä9 D CD´=CäAD$8 C" !B4 A
    /// Aô5D 5Cä?D 5CD5C´OCTBD 0 C" BC,,?C,,7C,,@Cä2C =CÔKDR =C AC´5CD=C,,:C E CD;Dò BCäGCTGCô>C´ ?D >C$5D :C, BC
    /// </summary>Ð
    /// <param name="state">B >D BCäOCô8CR 8Ct<CT=CT=C,,0 C,,7 Cô5D 5DT2C BDt8C¤0 EF Core (EntityStateChangeEnu
    /// <param name="entity">B 0CÄ 4Cä<CT=CÔKC´ >C JCT:D" ,,GC,,AD$KC´ 8C´8 Cô@Cä:D 8D >C$0C0=D´9)</param>Ð
    public virtual void HandleGlobalEntityCommit(object? state, object? entity)Ð
    {Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Components/Workspace/WorkspacePage.razor">
@namespace Promatis.Net.UI.Components.WorkspaceÐ
Ð
<MudPaper Elevation="@GetPaperElevation()"Ð
    Class="@GetPaperClass()"Ð
    Style="@($"width: 100%; min-height: {GetWorkspaceHeight()}; overflow: hidden;")">Ð
Ð
    @if (TopContent != null && !IsTopCollapsed())Ð
    {Ð
        <div class="workspace-zone-top w-100 mb-2"Ð
            style="@($"height: {GetTopHeight()}; min-height: {GetTopHeight()}; max-height:
{GetTopHeight()};")">Ð
            @TopContentÐ
        </div>Ð
    }Ð
Ð
    <div class="d-flex flex-row align-stretch flex-grow-1 gap-4 w-100" style="min-height: 0;">Ð
Ð
        @if (LeftContent != null && !IsLeftCollapsed())Ð
        {Ð
            <div class="workspace-zone-left d-flex flex-column align-stretch justify-space-between"Ð
                style="@($"width: {GetLeftWidth()}; min-width: {GetLeftWidth()}; max-width:
{GetLeftWidth()};")">Ð
                @LeftContentÐ
            </div>Ð
        }Ð
Ð
        <div class="workspace-zone-body flex-grow-1 h-100" style="min-width: 0; min-height: 0;">Ð
            @if (BodyContent != null)Ð
            {Ð
                @BodyContentÐ
            }Ð
        </div>Ð
Ð
        @if (RightContent != null && !IsRightCollapsed())Ð
        {Ð
            <div class="workspace-zone-right d-flex flex-column align-stretch justify-space-between"Ð
                style="@($"width: {GetRightWidth()}; min-width: {GetRightWidth()}; max-width:
{GetRightWidth()};")">Ð
                @RightContentÐ
            </div>Ð
        }Ð
Ð
    </div>Ð
Ð
    @if (BottomContent != null && !IsBottomCollapsed())Ð
    {Ð
        <div class="workspace-zone-bottom w-100 mt-2"Ð
            style="@($"height: {GetBottomHeight()}; min-height: {GetBottomHeight()}; max-height:
{GetBottomHeight()};")">Ð
            @BottomContentÐ
        </div>Ð
    }Ð
Ð
</MudPaper>
</file>

<file path="Promatis.Net.UI/Components/Workspace/WorkspacePage.razor.cs">
using Microsoft.AspNetCore.Components;Ð

```

```

using Promatis.Net.Data;
namespace Promatis.Net.UI.Components.Workspace;
public partial class WorkspacePage : ComponentBase, IDisposable
{
    /// <summary>
    /// A`>C$8CÄ :Cä=D$5CäAD" BCT:D4ICT9 D 0C >Dt5C' >C ;C AD$8 C,,7 Cä0D :C 4C i
    /// ATAC'8 D BD 0C08Dd0 Cä0D BCä<C00Dò 8 Cä>C0BCT:D B C05 C0CCd5C0Ä ECä;D B CäBD 5C04CT@C,,BD 0 D 4CTDCä;
    /// </summary>
    [CascadingParameter] protected WorkspaceActionContext? ActionContext { get; set; }

    [Parameter] public RenderFragment? BodyContent { get; set; }
    [Parameter] public RenderFragment? TopContent { get; set; }
    [Parameter] public RenderFragment? BottomContent { get; set; }
    [Parameter] public RenderFragment? LeftContent { get; set; }
    [Parameter] public RenderFragment? RightContent { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();

        // A0>CD?C,,AD'2C 5CÄ 2CT@DT=C,,9 D0BC 6 Cä0D :C AC =C 3C'>C 0C'LC0KCR BD 0C07C :Dd8C, !B4 AM
        DatabaseTriggerService.OnEntityCommitted += HandleDatabaseChange;
    }

    /// <summary>
    /// A0@C,,=C,,<C 5D" AC,,3C00C² :Cä<CÄ8D$0 C,,7 C05D 5DT2C Bdt8Cä0 C, ?CT@CT4C 5D" 5C4> C" :Cä=D$5CäAD" BCT:D
    /// </summary>
    private void HandleDatabaseChange(EntityStateChangeEnum state, object entity)
    {
        if (ActionContext == null) return;

        // A0@C,,=D44C,,BCT;DÄ=Cä <C @D,,0C'8D CCT< C$KCT>C" 8Cr ?CäBCä:C !B4 AB 2 C4;C 2C0KC' T'0?CäBCä: Blazo
        // B0BCä 3C @C =D$8D CCTB, DtBCä 7F FT† 46† ævVB,' 2C0CD$@C, :Cä=D$5CäAD$0 C00CÄ5D BC$> Cä1C0>C$8D" :
        InvokeAsync(() =>
        {
            ActionContext.HandleGlobalEntityCommit(state, entity);
        });
    }

    /// <summary>
    /// A0>C'=Cä5 D4=C,,GD$>Cd5C08CR AC$0Ct8 D BD 8C43CT@C <C, ?D 8 Ct0Cä@D'BC,,8 MDI-C$:C'0CD:C, ,, C IC,,BC >
    /// </summary>
    public void Dispose()
    {
        DatabaseTriggerService.OnEntityCommitted -= HandleDatabaseChange;
    }

    // --- ÄÄ B$ AD+ B B 'AT"A A,, B4 A',A0 A4 B "A,, B0 Ää A' Ad A, 00Ý
    protected int GetPaperElevation() => ActionContext?.PaperElevation ?? 1;
    protected string GetPaperClass() => ActionContext?.PaperClass ?? "pa-4 d-flex flex-column flex-grow-1 w-100 h-100";

    // --- ÄÄ B$ AD+ B B 'AT"A AT ÄÄ B$ A,, Aä B A !A %Ää B "A 00Ý
    protected string GetWorkspaceHeight() => ActionContext?.WorkspaceHeight ?? "100%";
    protected string GetTopHeight() => ActionContext?.TopZoneHeight ?? "auto";
    protected string GetBottomHeight() => ActionContext?.BottomZoneHeight ?? "auto";
    protected string GetLeftWidth() => ActionContext?.LeftZoneWidth ?? "250px";
    protected string GetRightWidth() => ActionContext?.RightZoneWidth ?? "300px";

    protected bool IsTopCollapsed() => ActionContext?.IsTopZoneCollapsed ?? false;
    protected bool IsBottomCollapsed() => ActionContext?.IsBottomZoneCollapsed ?? false;
    protected bool IsLeftCollapsed() => ActionContext?.IsLeftZoneCollapsed ?? false;
    protected bool IsRightCollapsed() => ActionContext?.IsRightZoneCollapsed ?? false;
}
</file>

<file path="Promatis.Net.UI/Dashboard.razor">
@page "/"
<MudText Typo="Typo.h4" Class="mb-4">ÄÄ>CDCC'L Master Data Management</MudText>
<MudGrid>
    <MudItem xs="12" sm="6" md="4">
        <MudPaper Class="d-flex flex-row pt-6 pb-4 px-4" Style="height:100px;">

```

```

        <MudIcon Icon="@Icons.Material.Filled.Storage" Color="Color.Primary" Size="Size.Large"
Class="mr-4" />
        <div>
            <MudText Typo="Typo.subtitle2" Class="mud-text-secondary">B BC BD4A CÔ>CD:C´NDt5CÔ8Dò : A </
            <MudText Typo="Typo.h6">A :D$8C$=Câ ...FW7D& 6R"Âô×VEFW‡Cí
        </div>
    </MudPaper>
</MudItem>
     
    <MudItem xs="12" sm="6" md="8">
        <MudPaper Class="pa-4" Style="height:100px;">
            <MudText Typo="Typo.subtitle1">AD>C @Câ ?Câ6C ;Câ2C BDÂ 2 D 8D BCT<D2 &öÖ F–2 U% Âô×VEFW‡Cí
            <MudText Typo="Typo.body2" Class="mud-text-secondary">
                A$ACR 8CÔBCT@DD5C"AD² ?Câ4C4@D46CT=D² 4C,,=C <C,,GCTAC=8 C,,7 CÔ5Ct0C$8D 8CÂKDR AC´>CT2 Razor CL
            </MudText>
        </MudPaper>
    </MudItem>
</MudGrid>
</file>

<file path="Promatis.Net.UI/Dashboard.razor.css">
.my-component {
    border: 2px dashed red;
    padding: 1em;
    margin: 1em 0;
    background-image: url('background.png');
}
</file>

<file path="Promatis.Net.UI/Extensions/MudColorExtensions.cs">
using MudBlazor;
 
namespace Promatis.Net.UI;
 
public static class MudColorExtensions
{
    public static Color GetActionColor(this string? action)
    {
        if (string.IsNullOrEmpty(action)) return Color.Default;
        return action.ToLower().Trim() switch
        {
            "create" or "insert" or "added" or "CD>C 0C$;CT=C,,5" or "D >Ct4C =C,,5" => Color.Success,
            "update" or "edit" or "modified" or "C,,7CÂ5CÔ5CÔ8CR" ÷" $@CT4C :D$8D >C$0CÔ8CR" Óâ 6öÆ÷"âv &æ-ærf
            "delete" or "remove" or "deleted" or "softdeleted" or "D44C ;CT=C,,5" => Color.Error,
            _ => Color.Default
        };
    }
}
</file>

<file path="Promatis.Net.UI/IUiModule.cs">
namespace Promatis.Net.UI;
 
/// <summary>
/// A=CÔBD 0C=B CD;Dò 4C,,=C <C,,GCTAC=8DR <Câ4D4;CT9 CÔ>C´Lct>C$0D$5C´LD :Câ3Câ 8CÔBCT@DD5C"AC í
/// </summary>
public interface IUiModule
{
    /// <summary>
    /// AâBCâ1D 0Cd0CT<Câ5 C,,<Dò <Câ4D4;Dò 2 D 8D BCT<CR ,,8D ?Câ;DÂ7D45D$ADò 4C´O D :C =CT@C ´í
    /// </summary>
    string Name { get; }
     
    /// <summary>
    /// A$>Ct2D 0D"0CTB D ?C,,ACâ: DÔ;CT<CT=D$>C" =C 2C,,3C FC,,8 CD;Dò 1Câ:Câ2Câ3Câ <CT=Dâ ×VDG& vW"1
    /// Bt5D$2CT@D$KC´ ?C @C <CTBD >D$2CTGC 5D" 7C "0CD >C$=CT2D4N C4@D4?Cô8D >C$:D2 MC´5CÂ5CÔBCâ2. 
    /// </summary>
    /// <returns>A>C´;CT:Dd8Dò :Câ@D$5Cd5C"¢ ,, C 7C$0CÔ8CRÂ !D KC´:C Â C= CÔ:C Â C 7C$0CÔ8CT D CCô?D²"Â÷&W
    IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems();
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLog/AuditLogPage.razor">
@page "/audit-logs"
 

```

```

@using Promatis.Net.Domain
{
    @* A$=CT4D OCT< C00D, 0C$BCä<C BC,,GCTAC=8 Ct0D 5C48D BD 8D >C$0C0=D´9 C" D´ :Cä=D$5C=AD" ´?Cä;C0>D$K C$;C AD$
    @inject AuditLogPageContext Context
    @* A0@Cä:C,,4D´2C 5CÄ :Cä=D$5C=AD" :C AC=0CD>CÄ 2C08CrÄ GD$>C K DT>C´AD" v÷&·7 6U vR =C BC,,2C0> Dt8D$0C² =C
    <CascadingValue Value="@Context" IsFixed="true">
        <WorkspacePage>
            <TopContent>
                @* B$#A´ A : A0>C´=CäAD$LDä 0C$BCä=Cä<C00D0 AD$@CäGC=0, D 5C04CT@C,,B C= Cä?C=8 C, 7C 3Cä;Cä2Cä:
                <UiToolbar TEntity="AuditLog" ActionContext="@Context">
                    <AdditionalContent>
                        @* AD5C=;C @C BC,,2C0> C$AD$@C 8C$0CT< C08C=5D 4C B, D 2D07C 2 CT3Cä =C ?D OCÄCDä A C#-D
                        <div class="d-flex align-center gap-3">
                            <MudDateRangePicker Label="A05D 8Cä4 C´>C4>C"-
                                @bind-DateRange="Context.Period"
                                Margin="Margin.Dense"
                                Style="width: 260px;"
                                Clearable="false" />
                        </div>
                    </AdditionalContent>
                </UiToolbar>
            </TopContent>
            <BodyContent>
                @* B$ A A,,&A ¢ 'C,,AD$KC´ 2C,,7D40C´8Ct0D$>D :Cä;Cä=Cä:. A,,7 C05C4> D BCT@D" 2CTADÄ &ö-ÆW' Æ FR0:
                <GridPage TEntity="AuditLog">
                    <ColumnsContent>
                        <PropertyColumn T="AuditLog" TProperty="DateTime" Property="x => x.ChangedAt"
                        Title="AD0D$0 C, 2D 5CÄ0" Format="dd.MM.yyyy HH:mm:ss" HeaderStyle="width: 250px;" Style="width: 250px;" />
                        <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.EntityName"
                        Title="B CD"=CäAD$L" Style="width: 200px;" />
                        <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.Action"
                        Title="Aä?CT@C FC,,0" Style="width: 120px;">
                            <CellTemplate>
                                <MudChip T="string" Color="@context.Item.Action.GetActionColor()"
                                Size="Size.Small" Variant="Variant.Text">
                                    @context.Item.Action
                                </MudChip>
                            </CellTemplate>
                        </PropertyColumn>
                        <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangedBy"
                        Title="A0>C´Lct>C$0D$5C´L" Style="width: 180px;" />
                        <PropertyColumn T="AuditLog" TProperty="Guid" Property="x => x.EntityId"
                        Title="ID Aä1D=5C=BC " 7G-ÆS0´v-GFf¢ 3# f²" 6Æ 730&föçB00öæ÷7 6R" óí
                        <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangesJson"
                        Title="A,,7CÄ5C05C08D0 „¥4ôâ" 6VÆÄ6Æ 730´FW‡B×G´Væ6 FR" óí
                    </ColumnsContent>
                </GridPage>
            </BodyContent>
        </WorkspacePage>
    </CascadingValue>
</file>

<file path="Promatis.Net.UI/Pages/AuditLog/AuditLogPageContext.cs">
using MudBlazor;
using Promatis.Net.Service;
using Promatis.Net.UI.Components.Grid;
using Promatis.Net.UI.Components.Toolbar;
{
    namespace Promatis.Net.UI.Pages.AuditLog;
    {
        /// <summary>
        /// A=C0BCT:D B D4?D 0C$;CT=C,,0 C0@C,,:C´0CD=Cä9 D BD 0C08Dd5C´ 6D4@C00C´0 C CCD8D$0.D
        /// A$KD BD4?C 5D" 5CD8C0KCÄ <CT4C,,0D$>D >CÄ 4C´O DT>C´AD$0, D$CC´1C @C 8 D$0C ;C,,FD² 2 Aä B2í
        /// </summary>
        /// <summary>
        /// A=C0BCT:D B D4?D 0C$;CT=C,,0 C0@C,,:C´0CD=Cä9 D BD 0C08Dd5C´ 6D4@C00C´0 C CCD8D$0.D
        /// A0>C´=CäAD$LDä ?Cä;C 3C 5D$AD0 =C 0C$BCä<C BC,,:D2 >C =Cä2C´5C08C´ OCD@C w&-D 7F-öä6öçFW‡Bí
        /// </summary>
        public class AuditLogPageContext : GridActionContext<Domain.AuditLog>
        {
            private readonly IAuditLogService _auditLogService;
            private DateRange _period = new(DateTime.Today, DateTime.Today.AddDays(1).AddSeconds(-1));

```



```

    }

    /// <summary>
    /// A05D 8Cä4 C´>C4>C"Â : C¤>D$>D >CÄC C00C0@D0<D4N C0@C,,2D07C = MudDateRangePicker C" @C 7CÄ5D$:CR1
    /// </summary>
    public DateRange Period
    {
        get => _period;
        set
        {
            if (_period != value)
            {
                _period = value;
                NotifyUpdate(); // AÄ3C0>C$5C0=Cä ?CT@CT@C,,ACä2D´2C 5CÄ BD4;C 0D ,,?C,,:CT@ CD0D"*
                RequestRefresh(); // A08C00CT< GridPage C00 C05D 5Ct0C0@CäA D 5D 2CT@C0KDR 4C =C0KD]
            }
        }
    }
}

// A 2D$>CÄ0D$8Dt5D :C, 2C05CD@D05D$AD0 ääUB D´ @C =D$0C"<Cä< C0@C, >D$:D KD$8C, 2C¤;C 4C¤8D
public AuditLogPageContext(IAuditLogService auditLogService) : base()
{
    _auditLogService = auditLogService;

    PageTitle = "AdCD =C ; C CCD8D$0";

    // AD5C¤;C @C BC,,2C0> CäBC¤;DäGC 5CÄ 1C 7Cä2D´9 CRUD
    IsCreateVisible = false;
    IsEditVisible = false;
    IsDeleteVisible = false;

    // A00D BD 0C,,2C 5CÄ =C H D4=C,,2CT@D 0C´LC0KC´ D >C¤5D C =C0KDR =C ACT@C$5D =D´9 C0>D BD 0C08Dt=D
    DataBroker.ConfigureServerMode(FetchServerDataInternalAsync);

    // A 5Ct>C00D =Cä 4Cä1C 2C´0CT< C¤0D BCä<C0CDä :C0>C0:D2 GCT@CT7 C0;C BDD>D <CT=C0KC´ <CTBCä4 C¤>C0BD
    AddCustomAction(new ToolbarCustomAction
    {
        Id = "excel_export",
        Title = "A$KC4@D47C,,BDÄ 2 Excel",
        Icon = Icons.Material.Filled.Download,
        Color = Color.Success,
        Variant = Variant.Filled,
        OnExecute = ExportToExcelInternalAsync
    });
}

private async Task<GridData<Domain.AuditLog>>
FetchServerDataInternalAsync(GridState<Domain.AuditLog> state, CancellationToken ct)
{
    DateTime localStart = Period.Start ?? DateTime.MinValue;
    DateTime localEnd = Period.End ?? DateTime.MaxValue;

    if (Period.End.HasValue && Period.End.Value.TimeOfDay == TimeSpan.Zero)
    {
        localEnd = Period.End.Value.AddDays(1).AddSeconds(-1);
    }

    var request = new AuditLogSearchRequest
    {
        FromDate: localStart.ToUniversalTime(),
        ToDate: localEnd.ToUniversalTime(),
        EntityName: null,
        Action: null,
        PageIndex: state.Page,
        PageSize: state.PageSize
    };

    PagedResult<Domain.AuditLog> result = await _auditLogService.SearchLogsAsync(request, ct);

    return new GridData<Domain.AuditLog>
    {
        TotalItems = result.TotalCount,
        Items = result.Items
    };
}

private async Task ExportToExcelInternalAsync()
{

```

```

        SetActionEnabled("excel_export", false);
    }
    try
    {
        await Task.Delay(1000);
    }
    finally
    {
        SetActionEnabled("excel_export", true);
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Promatis.Net.UI.csproj">
<Project Sdk="Microsoft.NET.Sdk.Razor">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>
    <ItemGroup>
        <Compile Remove="AuditLog\*" />
        <Compile Remove="Interfaces\*" />
        <Compile Remove="Models\*" />
        <Content Remove="AuditLog\*" />
        <Content Remove="Interfaces\*" />
        <Content Remove="Models\*" />
        <EmbeddedResource Remove="AuditLog\*" />
        <EmbeddedResource Remove="Interfaces\*" />
        <EmbeddedResource Remove="Models\*" />
        <None Remove="AuditLog\*" />
        <None Remove="Interfaces\*" />
        <None Remove="Models\*" />
    </ItemGroup>
    <ItemGroup>
        <SupportedPlatform Include="browser" />
    </ItemGroup>
    <ItemGroup>
        <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />
        <PackageReference Include="Microsoft.AspNetCore.Http.Abstractions" Version="2.3.10" />
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\MesCore\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />
        <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />
        <ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />
    </ItemGroup>
</Project>
</file>

```

```

<file path="Promatis.Net.UI/Theme/PromatisTheme.cs">
using MudBlazor;
namespace Promatis.Net.Ui.Theme;
public static class PromatisTheme
{
    public static MudTheme DefaultTheme => new()
    {
        PaletteLight = new PaletteLight
        {
            Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' :Cä@Cö>D 0D$8C$=D'9 Dd2CTB (C=Cä?C=8, C :D$
            Secondary = Colors.Blue.Lighten2, // A$ACö>CÄ>C40D$5C'LC0KC' FC$5D" „2D$>D >D BCT?CT=C0KCR MC

            AppBarBackground = "#EAEAEA", // BD>C0 2CT@DT=CT9 C0C05C'8 (MudAppBar) C" 4C05C$=Cä< D 5
            AppBarText = "#2C3E50", // Bd2CTB D$5C=AD$0, Ct0C4>C'>C$:Cä2 C, 8C= >C0>C 2 C$5D EC
            DrawerBackground = "#EAEAEA", // BD>C0 1Cä:Cä2Cä3Cä <CT=Dä =C 2C„3C FC„8 (MudDrawer)
            Background = "#F9F9F9", // Aä1D"8C' DCä=Cä2D'9 Dd2CTB C0>CD;Cä6C=8 C$ACTE D BD 0C08

```

```

        Surface = Colors.Shades.White, // BD>C0 :Cä=D$5C"=CT@Cä2 MudPaper, C00D BCäGCT: MudCard C
    },0
    PaletteDark = new PaletteDark0
    {0
        Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5 (CÄ0C4:C,,9 D 8C
        Secondary = Colors.Blue.Lighten2, // A$AC0>CÄ>C40D$5C'LC0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5D

    0
        AppBarBackground = "#0D0D0D", // BD>C0 2CT@DT=CT9 C00C05C'8 (MudAppBar) C" 3C'CC >C0>CÄ G
        AppBarText = Colors.Shades.White, // Bd2CTB D$5C0AD$0 C, 8C0>C0>C0 2 D,,0C0:CR ,, :Cä=D$@C AD$=D

    0
        Background = "#121212", // A4;D41Cä:C,,9 D$QCÄ=D'9 DD>C0 AD$@C =C,,F C0>CD;Cä6C08D
        Surface = "#1E1E1E", // A4@C DC,,BCä2D'9 DD>C0 4C'0 MudPaper, C0>C0BCT=D$0 C$:C'0
        DrawerBackground = "#1E1E1E", // BD>C0 1Cä:Cä2Cä9 C00C05C'8 C00C$8C40Dd8C, 2 D$QCÄ=Cä9 D$
        DrawerText = "#E0E0E0", // Bd2CTB D AD';Cä: C, BCT:D BC 2 C >C0>C$>CÄ <CT=D1
        TextPrimary = "#E0E0E0", // AäAC0>C$=Cä9 Dd2CTB D$5C0AD$0 C00 D BD 0C08Dd0DR ,, <D03C0
        TextSecondary = "#A0A0A0", // Bd2CTB CD;D0 2D$>D >D BCT?CT=C0>C4> D$5C0AD$0 C, ?Cä4D

    },0
    LayoutProperties = new LayoutProperties0
    {0
        DrawerWidthLeft = "260px",0
        DrawerWidthRight = "300px"0
    }0
};0
}
</file>

```

```

<file path="Promatis.Net.UI/UiDataBroker.cs">
using System.Reflection;0
using MudBlazor;0
0
namespace Promatis.Net.UI;0
0
/// <summary>0
/// A0;C BDD>D <CT=C0KC' 1D >C05D 4C =C0KDRä &CT=D$@C ;C,,7Cä2C =C0> D4?D 0C$;D05D" ?CäAD$0C$:Cä9 CD0C0=D'E C
/// A0>C'=CäAD$LDä 0C AD$@C 3C,,@D45D" T' >D" :Cä=C0@CTBC0KDR 8C0BCT@DD5C"ACä2 CD>CÄ5C0=D'E D ;D46C 1D0:CT=CD
/// </summary>0
/// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C' AD4IC0>D BCfÄ÷G- W & Ó1
public class UiDataBroker<TEntity> where TEntity : class0
{0
    private Func<GridState<TEntity>, CancellationToken, Task<GridData<TEntity>>>>
    _customServerDataProvider;0
    0
    public List<TEntity>? InMemoryItems { get; set; }0
    public bool IsInMemoryMode { get; private set; }0
    0
    public void ConfigureServerMode(Func<GridState<TEntity>, CancellationToken,
Task<GridData<TEntity>>> dataProvider)0
    {0
        _customServerDataProvider = dataProvider ?? throw new
ArgumentNullException(nameof(dataProvider));0
        InMemoryItems = null;0
        IsInMemoryMode = false;0
    }0
    0
    public void ConfigureInMemoryMode()0
    {0
        _customServerDataProvider = null;0
        IsInMemoryMode = true;0
    }0
    0
    public async Task<GridData<TEntity>> FetchDataAsync(GridState<TEntity> state, CancellationToken
ct = default)0
    {0
        if (_customServerDataProvider != null)0
        {0
            return await _customServerDataProvider(state, ct);0
        }0
        0
        if (IsInMemoryMode)0
        {0
            return new GridData<TEntity>0
            {0
                Items = InMemoryItems ?? [],0
                TotalItems = InMemoryItems?.Count ?? 00
            };0
        }0
    }0
}

```

```

return new GridData<TEntity> { Items = Array.Empty<TEntity>(), TotalItems = 0 };
}

// =====
// A$+B A Aô Aâ At Aä A,, "AT BÄ B´ Aä B2Ô A$ Ad A¢ B4"A &A,, Aô A "BD B B² ,, Aä A A´ AÔ )
// =====

/// <summary>
/// BT8D CD 3C,,GCTAC¤8 D$>Dt=Câ <Cä4C,,DC,,FC,,@D45D" ;Cä:C ;DÄ=D´9 C¤MD, >Cö5D 0D$8C$=Cä9 Cö0CÄOD$8 Ct0 0 C
/// Aô>C´=CäAD$LDâ 8D :C´NDt0CTB CÖ5Cä1DT>CD8CÄ>D BDÄ ?Cä2D$>D =D´E D$OCd5C´KDR 4TÄT5B07C ?D >D >C" : B #
/// </summary>
/// <param name="stateStr">B BD >C¤>C$>CR ACäAD$>Dô=C,,5 D$@C =Ct0C¤FC,,8 ("Added", "Modified", "Deleted",
/// B 0CÄ ?D 8D,,5CDHC,,9 CD>CÄ5CÖ=D´9 Cä1D¤5C¤B</param>
public void ApplyIncrementalOzuDelta(string stateStr, TEntity entity)
{
    // ATAC´8 CÄK D 0C >D$0CT< C" ACT@C$5D =Cä< D 5Cd8CÄ5 (Cö0Cö@C,,<CT@, C´>C48 C CCD8D$0), Aä B2Ô<D4BC F
    if (!IsInMemoryMode || InMemoryItems == null) return;

    // A,,7C$;CT:C 5CÄ AC$>C"AD$2Câ -B A Cö>CÄ>D"LDâ @CTDC´5C¤AC,,8 (C$KCö>C´=Dô5D$ADò >CD8CÒ @C 7 Cö0 D CD
    PropertyInfo? idProperty = typeof(TEntity).GetProperty("Id");
    if (idProperty == null) return;

    Guid entityId = (Guid)idProperty.GetValue(entity)!;

    switch (stateStr)
    {
        case "Added":
            // At0D"8D$0 CäB CDCC ;C,,@Cä2C =C,,0 Cö@C, 0D 8CöED >Cö=D´E C$ACö;CTAC¤0D]
            if (!InMemoryItems.Any(x => (Guid)idProperty.GetValue(x)! == entityId))
            {
                InMemoryItems.Add(entity);
            }
            break;

        case "Modified":
            TEntity? existingItem = InMemoryItems.FirstOrDefault(x =>
(Guid)idProperty.GetValue(x)! == entityId);
            if (existingItem != null)
            {
                // Aô0DT>CD8CÄ 2D 5 Ct=C GC,,<D´5 C, AD$@Cä:Cä2D´5 D 2Cä9D BC$0 CD;Dò BCäGCTGCö>C4> C¤>Cö8
                IEnumerable<PropertyInfo> properties = typeof(TEntity).GetProperties()
                    .Where(p => p.CanWrite && p.CanRead)
                    .Where(p => p.PropertyType.IsValueType || p.PropertyType == typeof(string));

                foreach (PropertyInfo prop in properties)
                {
                    prop.SetValue(existingItem, prop.GetValue(entity));
                }
            }
            break;

        case "Deleted":
        case "SoftDeleted":
            TEntity? itemToRemove = InMemoryItems.FirstOrDefault(x =>
(Guid)idProperty.GetValue(x)! == entityId);
            if (itemToRemove != null)
            {
                InMemoryItems.Remove(itemToRemove);
            }
            break;
    }
}
}
</file>

<file path="Promatis.Net.UI/UiModule.cs">
using MudBlazor;
namespace Promatis.Net.UI;
public class UiModule : IUiModule
{
    public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";

    public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()

```

```
{@
    return new List<(string Title, string Href, string Icon, string? Group)>@
    {@
        // B0;CT<CT=D" 2CT@DT=CT3Câ CD >C$=Dò „2C05 C00C0>C¢•
        ("A4;C 2C00D0"Â "ò"Â -6öç2âö FW&- Âäf-ÆÆVBäF 6†&ö &BÂ çVÆÂ'í
        @
        // B 0Ct4CT; D 8D BCT<C0>C4> C 4CÄ8C08D BD 8D >C$0C08Dý
        ("AdCD =C ; C CCD8D$0", "/audit-logs", Icons.Material.Filled.History, "A 4CÄ8C08D BD 8D >C$0C08CR
    };@
}
</file>

<file path="Promatis.Net.UI/wwwroot/App.css">
html, body {@
    font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif;@
}
@
a, .btn-link {@
    color: #0066bb7;@
}
@
.btn-primary {@
    color: #fff;@
    background-color: #1b6ec2;@
    border-color: #1861ac;@
}
@
.btn:focus, .btn:active:focus, .btn-link.nav-link:focus, .form-control:focus, .form-check-
input:focus {@
    box-shadow: 0 0 0 0.1rem white, 0 0 0 0.25rem #258cfb;@
}
@
.content {@
    padding-top: 1.1rem;@
}
@
h1:focus {@
    outline: none;@
}
@
.valid.modified:not([type=checkbox]) {@
    outline: 1px solid #26b050;@
}
@
.invalid {@
    outline: 1px solid #e50000;@
}
@
.validation-message {@
    color: #e50000;@
}
@
.blazor-error-boundary {@
    background: url(
8vd3d3LnczLm9yZy8yMDAwL3N2ZyIgeG1sbnM6eGxpbnM6Imh0dHA6Ly93d3cudzMub3JnLzE5OTkveGxpbnM6IiG92ZXJmbG93PS
JoawRkZW4iPjxkZWZzPjxjbGwUGF0aCBpZD0iY2xpcDAiPjxyZWNOIHg9IjIzNSIgeT0iNTEiIHdpZHRoPSI1NiIgaGVPZ2h0PS
I0OSiVpJwvY2xpcFBhdGg+PC9kZWZzPjxnIGNsaXAtcGF0aD0idXJsKCNjbGlmMCKiIHRYW5zZm9ybT0idHJhbnNsYXRlKC0yMz
UgLTUxKSI+PHBhdGggZD0iTTI2My41MDYgNTFDMjY0LjcxNyA1MSAyNjUuODEzIDUxLjQ4MzcgMjY2LjYwNiA1Mi4yNjU4TDI2Ny
4wNTI0NTIuNzZk4NyAyNjcuNTM5IDUzLjYyODMgMjkwLjE4NSA5Mi4xODMxIDI5MC41NDUgOTIuNzZk1IDI5MC42NTYgOTIuOTk2Qz
I5MC44NzcgOTMuNTEzIDI5MSA5NC4wODE1IDI5MSA5NC42NzgyIDI5MSA5Ny4wNjUxIDI4OS4wMzggOTkgMjg2LjYxNyA5OUwyND
AuMzgzIDk5QzIzNy45NjMgOTkgMjM2IDk3LjA2NTEgMjM2IDk0LjY3ODI0MjM2IDk0LjM3OTkgMjM2LjAzMSA5NC4wODg2IDIzNi
4wODkgOTMuODA3MkwyMzYuMzM4IDkzLjAxNjI0MjM2Ljg1OCA5Mi4xMzE0IDI1OS40NzMgNTMuNjI5NCAyNTkuOTYxIDUyLjc5OD
UgMjYwLjQwNyA1Mi4yNjU4QzI2MS4yIDUxLjQ4MzcgMjY2LjY1NiA1MSAyNjMuNTA2IDUxwk0yNjMuNTg2IDY2LjAxODNDMjYwLj
czNyA2Ni4wMTgzIDI1OS4zMtMgNjcuMTI0NSAyNTkuMzEzIDY5LjMzNyAyNTkuMzEzIDY5LjYxMDI0MjY5LjMzMiA2OS44NjA4ID
I1OS4zNzEgNzAuMDg4N0wyNjEuNzZk1IDg0LjAxNjEgMjY1LjM4IDg0LjAxNjEgMjY3LjgyMSA2OS43NDc1QzI2Ny44NiA2OS43Mz
A5IDI2Ny44NzkgNjkuNTg3NyAyNjcuODc5IDY5LjMxNzkgMjY3Ljg3OS42Ny4xMTgyIDI2Ni40NDggNjYuMDE4MyAyNjMuNTg2ID
Y2LjAxODNaTTI2My41NzYgODYyMDU0N0MyNjEuMDQ5IDg2LjA1NDcgMjY5LjM4NiA4Ny4zMDE1IDI1OS43ODYgODkuNzkyMSAyNT
kuNzgzIDkYlJi4MzcgMjYxLjA0OSA5My41MjM1IDI2My41NzYgOTMuNTI5NSAyNjYuMTE2IDkzLjYyOTUgMjY3LjM4NyA5Mi4yOD
M3IDI2Ny44ZDcgODkuNzkyMSAyNjcuMzg3IDg3LjMwMDUgMjY2LjExNiA4Ni4wNTQ3IDI2My41NzYgODYyMDU0N0i0iIGZpbGw9Ii
NGRkU1MDAiIGZpbGwtcnVsZT0iZXZlbn9kZCIVpJwvZz48L3N2Zz4=) no-repeat 1rem/1.8rem, #b32121;@
    padding: 1rem 1rem 1rem 3.7rem;@
    color: white;@
}
@
.blazor-error-boundary::after {@
```

```

        content: "An error has occurred."@
    }@
}
@
.darker-border-checkbox.form-check-input {
    border-color: #929292;@
}
@
.form-floating > .form-control-plaintext::placeholder, .form-floating > .form-control::placeholder {
    color: var(--bs-secondary-color);@
    text-align: end;@
}
@
@
.form-floating > .form-control-plaintext:focus::placeholder, .form-floating > .form-
control:focus::placeholder {
    text-align: start;@
}
}
</file>

<file path="Service/Contracts/AuditLog/AuditLogSearchRequest.cs">
namespace Promatis.Net.Service;@
@
public record AuditLogSearchRequest(@
    DateTime FromDate,@
    DateTime ToDate,@
    string? EntityName = null,@
    string? Action = null,@
    int PageIndex = 0,@
    int PageSize = 10);
</file>

<file path="Service/Contracts/AuditLog/PagedResult.cs">
namespace Promatis.Net.Service;@
@
public record PagedResult<T>(IReadOnlyCollection<T> Items, int TotalCount);
</file>

<file path="Service/Promatis.Net.Service.csproj">
<Project Sdk="Microsoft.NET.Sdk">

    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>

    <ItemGroup>@
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />@
    </ItemGroup>

    <ItemGroup>@
        <ProjectReference Include="..\Data\Promatis.Net.Data.csproj" />@
    </ItemGroup>

</Project>
</file>

<file path="Service/Services/AuditLogService/AuditLogService.cs">
using Microsoft.EntityFrameworkCore;@
using Promatis.Net.Data;@
using Promatis.Net.Domain;@
@
namespace Promatis.Net.Service;@
@
/// <summary>@
/// B 5D 2C,,A CD;Dò @C 1CäBD² A C´>C40CÄ8 C CCD8D$0.@
/// Aò>C´=CäAD$LDâ ACä2CÄ5D BC,,< D 1C 7Cä2D´< BaseService C, 0C$BCä<C BC,,GCTACº8 D 5C48D BD 8D CCTBD O D :C
/// </summary>@
public class AuditLogService(IDbContextFactory<ApplicationDbContext> contextFactory)@
    : BaseService<AuditLog, Guid, ApplicationDbContext>(contextFactory), IAuditLogService@
{
    private static List<string>? _cachedEntityNames;@
    private static readonly SemaphoreSlim CacheLock = new(1, 1);@
@
    public async Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request,
        CancellationToken cancellationToken = default)@
    {

```

```

        await using ApplicationDbContext context = await
ContextFactory.CreateDbContextAsync(cancellationToken);
    }
    IQueryable<AuditLog> query = context.Set<AuditLog>()
        .AsNoTracking()
        .Where(l => l.ChangedAt >= request.FromDate && l.ChangedAt <= request.ToDate);
    if (!string.IsNullOrEmpty(request.EntityName)) query = query.Where(l => l.EntityName
== request.EntityName);
    if (!string.IsNullOrEmpty(request.Action)) query = query.Where(l => l.Action ==
request.Action);
    int totalCount = await query.CountAsync(cancellationToken);
    if (totalCount == 0)
        return new PagedResult<AuditLog>([], 0);
    List<AuditLog> items = await query
        .OrderByDescending(l => l.ChangedAt)
        .Skip(request.PageIndex * request.PageSize)
        .Take(request.PageSize)
        .ToListAsync(cancellationToken);
    return new PagedResult<AuditLog>(items, totalCount);
}
public async Task<List<string>> GetAvailableEntityNamesAsync(CancellationTok
en
cancellationToken = default)
{
    if (_cachedEntityNames != null)
    {
        return [.. _cachedEntityNames];
    }
    await CacheLock.WaitAsync(cancellationToken);
    try
    {
        if (_cachedEntityNames != null)
        {
            return [.. _cachedEntityNames];
        }
        await using ApplicationDbContext context = await
ContextFactory.CreateDbContextAsync(cancellationToken);
        _cachedEntityNames = await context.Set<AuditLog>()
            .AsNoTracking()
            .Select(l => l.EntityName)
            .Distinct()
            .OrderBy(name => name)
            .ToListAsync(cancellationToken);
        return [.. _cachedEntityNames];
    }
    finally
    {
        CacheLock.Release();
    }
}
public static void InvalidateCache()
{
    CacheLock.Wait();
    try
    {
        _cachedEntityNames = null;
    }
    finally
    {
        CacheLock.Release();
    }
}
}
</file>

```

<file path="Service/Services/AuditLogService/IAuditLogService.cs">

```

using Promatis.Net.Domain;
namespace Promatis.Net.Service;
public interface IAuditLogService
{
    /// <summary>
    /// Aö>C„AC¢ ;Cä3Cä2 C CCD8D$0 Cö> CD8C ?C 7Cä=D2 4C B D ?Cä4CD5D 6C¤>C' DC„;DÄBD 0Dd8C, 8 Cö0C48C00Dd8C
    /// </summary>
    Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request, CancellationToken
cancellationToken = default);
    /// <summary>
    /// Aö>C'CDt5C08CR AC08D :C CC08C¤0C'LC0KDR 8CÄ5C0 AD4IC0>D BCT9, C¤>D$>D KCR 5D BDÂ 2 C'>C40DR „4C'O C$
    /// </summary>
    Task<List<string>> GetAvailableEntityNamesAsync(CancellationToken cancellationToken = default);
}
</file>

<file path="Service/Services/BaseService/BaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Service;
public abstract class BaseService<T, TKey, TContext>(IDbContextFactory<TContext> contextFactory)
: IBaseService<T, TKey>
where T : class, IDomainObjectHasKey<TKey>
where TContext : DbContext
{
    protected readonly IDbContextFactory<TContext> ContextFactory = contextFactory ?? throw new
ArgumentNullException(nameof(contextFactory));
    public virtual async Task<T?> GetByIdAsync(TKey id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().FindAsync(id);
    }
    public virtual async Task<List<T>> GetAllAsync()
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().ToListAsync();
    }
    public virtual async Task AddAsync(T entity)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        await context.Set<T>().AddAsync(entity);
        await context.SaveChangesAsync();
    }
    public virtual async Task UpdateAsync(T entity)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        context.Set<T>().Update(entity);
        await context.SaveChangesAsync();
    }
    public virtual async Task DeleteAsync(TKey id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        T? entity = await context.Set<T>().FindAsync(id);
        if (entity != null)
        {
            context.Set<T>().Remove(entity);
            await context.SaveChangesAsync();
        }
    }
}
</file>

<file path="Service/Services/BaseService/IBaseService.cs">
namespace Promatis.Net.Service;
// A 0Ct>C$KC' 5%TM

```



```

public interface IBaseService<T, in TKey> where T : class
{
    Task<T?> GetByIdAsync(TKey id);
    Task<List<T>> GetAllAsync();
    Task AddAsync(T entity);
    Task UpdateAsync(T entity);
    Task DeleteAsync(TKey id);
}
</file>

<file path="Service/Services/ReferenceService/IReferenceService.cs">
namespace Promatis.Net.Service;
// AD;Dò ACô@C 2CäGCô8Cä>C-
public interface IReferenceService<T> : IBaseService<T, Guid> where T : class
{
    Task<T?> GetByCodeAsync(string code);
    Task<List<T>> SearchByNameAsync(string namePart);
}
</file>

<file path="Service/Services/ReferenceService/ReferenceService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
namespace Promatis.Net.Service;
public abstract class ReferenceService<T, TContext>(IDbContextFactory<TContext> contextFactory)
    : BaseService<T, Guid, TContext>(contextFactory), IReferenceService<T>
    where T : ReferenceBase
    where TContext : DbContext
{
    public async Task<T?> GetByCodeAsync(string code)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().FirstOrDefaultAsync(x => x.Code == code);
    }
    public async Task<List<T>> SearchByNameAsync(string namePart)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>()
            .Where(x => x.Name.Contains(namePart))
            .ToListAsync();
    }
}
</file>

<file path="Service/Services/ReferenceTreeService/IReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Service;
public interface IReferenceTreeService<T, TContext> : IReferenceService<T>, ITreeService<T>
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>
    where TContext : DbContext
{
    // B AC2Cä7CôKCÄ =C AC'5CD>C$ôCô8CT< C,,=D$5D DCT9D >C" 2D Q D >D,,;CäADÂ
}
</file>

<file path="Service/Services/ReferenceTreeService/ReferenceTreeService.cs">
using System.Reflection;
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Service;

```

```

Ð
/// <summary>Ð
/// B4=C,,2CT@D 0C´LCÔKC´ 1C 7Cä2D´9 D 5D 2C,,A CD;Dò @C 1CäBD² ACä 2D 5CÄ8 CD@CT2Cä2C,,4CÔKCÄ8 D BD CC=BD4@C <C
/// A00D ;CT4D45D" ;Cä3C,,D2 >C KDt=D´E D ?D 0C$>Dt=C,,:Cä2 (ReferenceService) C, @C AD,,8D OCTB CTQ CD;Dò 8CT@
/// </summary>Ð
/// <typeparam name="T">B$8Cò AD4ICÔ>D BC,Â CCÔ0D ;CT4Cä2C =CÔKC´ >D" &Vfw&Væ6UG&VT& 6RäÂ÷G- W & Óí
/// <typeparam name="TContext">A= CÔBCT:D B C 0CtK CD0CÔ=D´E.</typeparam>Ð
public abstract class ReferenceTreeService<T, TContext>(IDbContextFactory<TContext> contextFactory)Ð
: ReferenceService<T, TContext>(contextFactory), IReferenceTreeService<T, TContext>Ð
where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
where TContext : DbContextÐ
{Ð
private TreeServiceProxy? _treeServiceProxy;Ð
private TreeServiceProxy Engine => _treeServiceProxy ??= new TreeServiceProxy(ContextFactory,
this);Ð
Ð
// =====Ð
// BÔ AT A B$ B´ AÔ Aä B B B´ Aä Aä A" AD AÔ+A´ A$ Ad A¢ E$TU4U%d"4R f R B4 A´ B A$ AÔ Bò Aä A
// =====Ð
public Task<List<T>> GetRootsAsync() => Engine.GetRootsAsync();Ð
public Task<List<T>> GetChildrenAsync(Guid parentId) => Engine.GetChildrenAsync(parentId);Ð
public Task<List<T>> GetParentPathAsync(Guid id) => Engine.GetParentPathAsync(id);Ð
public Task<T?> GetFullTreeAsync(Guid rootId) => Engine.GetFullTreeAsync(rootId);Ð
public Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default) =>
Engine.MoveAsync(id, newParentId, ct);Ð
Ð
public abstract Task<T> CreateChildTemplateAsync(T parent);Ð
Ð
// A$ B4"B AÔ A,, Aä B "-B A A,, A "Aä AÔ A "BD B B½
private class TreeServiceProxy(IDbContextFactory<TContext> factory, ReferenceTreeService<T,
TContext> parentService)Ð
: TreeService<T, TContext>(factory)Ð
{Ð
public override Task<T> CreateChildTemplateAsync(T parent) =>
parentService.CreateChildTemplateAsync(parent);Ð
}Ð
}
</file>

<file path="Service/Services/TreeService/ITreeService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Reflection;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// A4;Cä1C ;DÄ=D´9 Cò;C BDD>D <CT=CÔKC´ :Cä=D$@C :D" 4C´O C 1D >C´ND$=Cä ;Dä1Cä3Cä ACT@C$8D 0, D4?D 0C$;DòND
/// A00D ;CT4D45D" 1C 7Cä2D´9 CRUD Cò;C BDD>D <D² 8 D 0D HC,,@D05D" 5C4> C,,5D 0D EC,,GCTAC=8CÄ8 CÄ5D$>CD0CÄ8 D4
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Cä<CT=CÔ>C´ AD4ICÔ>D BC,Â @CT0C´8CtCDäICT9 C= CÔBD 0C=B ITreeNode.</typeparam>Ð
public interface ITreeService<T> : IBaseService<T, Guid>Ð
where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
{Ð
Task<List<T>> GetRootsAsync();Ð
Task<List<T>> GetChildrenAsync(Guid parentId);Ð
Task<List<T>> GetParentPathAsync(Guid id);Ð
Task<T?> GetFullTreeAsync(Guid rootId);Ð
Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default);Ð
Task<T> CreateChildTemplateAsync(T parent);Ð
}
</file>

<file path="Service/Services/TreeService/TreeService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Reflection;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// B4=C,,2CT@D 0C´LCÔKC´ 1C 7Cä2D´9 C=;C AD @CT0C´8Ct0Dd8C, 1C,,7CÔ5D Ô;Cä3C,,:C, 4C´O A´.A +BR 4D 5C$>C$8CD=D
/// A,,=C=0C0AD4;C,,@D45D" BD06CT;D´5 C,,5D 0D EC,,GCTAC=8CR 0C´3Cä@C,,BÄK B #A , Cò@Cä2CT@C= Dd8C=;Cä2 Dd8C=;C
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Cä<CT=CÔ>C4> Cä1D=5C=BC Â @CT0C´8CtCDäICT3Cä •G&VTæöFRäÂ÷G- W & Óí
/// <typeparam name="TContext">A= CÔBCT:D B C 0CtK CD0CÔ=D´E DbContext.</typeparam>Ð

```

```

public abstract class TreeService<T, TContext>(IDbContextFactory<TContext> contextFactory) :
    BaseService<T, Guid, TContext>(contextFactory), ITreeService<T>
where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>
where TContext : DbContext
{
    public async Task<List<T>> GetRootsAsync()
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId == null).ToListAsync();
    }

    public async Task<List<T>> GetChildrenAsync(Guid parentId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId ==
parentId).ToListAsync();
    }

    public async Task<List<T>> GetParentPathAsync(Guid id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        var path = new List<T>();
        T? current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id == id);

        while (current != null)
        {
            path.Insert(0, current);
            if (current.ParentId == null) break;
            Guid parentId = current.ParentId.Value;
            current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id ==
parentId);
            if (current == null) break;
        }
        return path;
    }

    public async Task<T?> GetFullTreeAsync(Guid rootId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        List<T> allItems = await context.Set<T>().AsNoTracking().ToListAsync();
        ILookup<Guid?, T> lookup = allItems.ToLookup(x => x.ParentId);
        T? root = allItems.FirstOrDefault(x => x.Id == rootId);
        if (root == null) return null;

        BuildTree(root, lookup);
        return root;
    }

    private void BuildTree(T parent, ILookup<Guid?, T> lookup)
    {
        List<T> children = lookup[parent.Id].ToList();
        parent.Children.Clear();
        foreach (T child in children)
        {
            parent.Children.Add(child);
            PropertyInfo? parentProp = child.GetType().GetProperty(nameof(ITreeNode<T>.Parent));
            if (parentProp is { CanWrite: true }) parentProp.SetValue(child, parent);
            BuildTree(child, lookup);
        }
    }

    public virtual async Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync(ct);
        T entity = await context.Set<T>().FirstOrDefaultAsync(x => x.Id == id, ct);
        ?? throw new Exception($"B CD"=CäAD$L {typeof(T).Name} D "B ¶-G0 =CR =C 9CD5C00.");

        if (entity.ParentId == newParentId) return;

        if (newParentId.HasValue)
        {
            if (newParentId == id) throw new InvalidOperationException("B47CT; C05 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;
List<T> path = await GetParentPathAsync(newParentId.Value);
            if (path.Any(x => x.Id == id)) throw new InvalidOperationException("Bd8C„C„GCTAC„D0 7C 2C„AC„<C
bool parentExists = await context.Set<T>().AnyAsync(x => x.Id == newParentId.Value, ct);
            if (!parentExists) throw new Exception("A0>C$KC´ @Cä4C„BCT;DÂ =CR =C 9CD5C0â"½

```

```

    }
    entity.ParentId = newParentId;
    await context.SaveChangesAsync(ct);
}

public abstract Task<T> CreateChildTemplateAsync(T parent);
}
</file>

<file path="Tests/AssemblyInfo.cs">
using Xunit;
// AäBCä;DäGC 5D" ?C @C ;C´5C´LCÔ>CR 2D´?Cä;CÔ5CÔ8CR BCTAD$>C" 2 DÔBCä9 D 1Cä@Cä5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Tests/Domain/DomainLogicTests.cs">
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainLogicTests
{
    // B 5C ;C,,7C FC,,8 CD;Dò BCTAD$8D >C$0C08Dò 0C AD$@C :D$=D´E Cä;C AD >C-
    private class TestDomainObject : DomainObject { }
    private class TestReference : ReferenceBase { }

    [Fact]
    public void DomainObject_EveryInstance_ShouldHaveUniqueId()
    {
        // Arrange & Act
        var obj1 = new TestDomainObject();
        var obj2 = new TestDomainObject();
        var obj3 = new TestDomainObject();

        // Assert
        Assert.NotEqual(obj1.Id, obj2.Id);
        Assert.NotEqual(obj2.Id, obj3.Id);
        Assert.NotEqual(Guid.Empty, obj1.Id);
    }

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData(null)]
    public void ReferenceBase_Name_CanBeNullOrEmpty_TechnicalCheck(string? invalidName)
    {
        // Arrange
        var reference = new TestReference();

        // Act
        reference.Name = invalidName!; // A,,ACô>C´LctCCT< !, D$0C¢ :C : Name C" :Cä4CR =CR çVÆÆ &Æ]

        // Assert
        // Aô>Cä0 CÄK Cô@CäAD$> Cô@Cä2CT@Dô5CÄÂ GD$> D 2Cä9D BC$> Cô@C,,=C,,<C 5D" MD$8 Ct=C GCT=C,,0D
        // ATAC´8 C$K CD>C 0C$8D$5 C$0C´8CD0dd8Dâ 2 C CCDCD"5CÄÂ MD$>D" BCTAD" ?Cä<Cä6CTB CTQ CäBC´0CD8D$LB
        Assert.Equal(invalidName, reference.Name);
    }

    [Fact]
    public void ReferenceBase_ShouldHoldValidName()
    {
        // Arrange
        var reference = new TestReference();
        string expectedName = "AäACÔ>C$=Cä9 D :C´0CB#½

        // Act
        reference.Name = expectedName;

        // Assert
        Assert.Equal(expectedName, reference.Name);
    }
}
</file>

```

```

<file path="Tests/Domain/DomainObjectBaseLogicTests.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class DomainObjectBaseLogicTests
{
    private class ConcreteDomainObject : DomainObject { }

    [Fact]
    public void DomainObject_ShouldGenerateNewGuidInConstructor()
    {
        // Arrange & Act
        var obj1 = new ConcreteDomainObject();
        var obj2 = new ConcreteDomainObject();

        // Assert
        Assert.NotEqual(Guid.Empty, obj1.Id);
        Assert.NotEqual(Guid.Empty, obj2.Id);
        Assert.NotEqual(obj1.Id, obj2.Id);
    }

    [Fact]
    public void DomainObject_ShouldCastToIDomainObjectAndHaveCreatedAt()
    {
        // Arrange
        var obj = new ConcreteDomainObject();

        // Act
        IDomainObject baseInterface = obj;

        // Assert
        // B$5C05D L CÄK D$>C´LC¤> C0@Cä2CT@D05CÂ =C ;C,,GC,,5 Ct=C GCT=C,,0. D
        // A0>C0KD$:C 7C ?C,,AC, & 6T-çFW&f 6Rä7&V FVD B 0 æWtF FR 7CD5D L C05 D :Cä<C08C´8D CCTBD O.D
        Assert.True(baseInterface.CreatedAt > DateTime.MinValue);
        Assert.True((DateTime.UtcNow - baseInterface.CreatedAt).TotalSeconds < 5);
    }

    [Fact]
    public void CreatedAt_ShouldAllowManualInitialization_OnCreation()
    {
        // Arrange
        var customDate = new DateTime(2020, 1, 1, 0, 0, 0, DateTimeKind.Utc);

        // Act
        // B 0C >D$0CTB D$>C´LC¤> Dt5D 5Cr 8C08Dd8C ;C,,7C BCä@ Cä1D¤5C¤BC ?D 8 D >Ct4C =C,,8D
        var obj = new ConcreteDomainObject { CreatedAt = customDate };

        // Assert
        Assert.Equal(customDate, obj.CreatedAt);
    }
}
</file>

```

```

<file path="Tests/Domain/DomainObjectBaseTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class DomainObjectBaseTests
{
    // B >Ct4C 5CÂ <C,,=C,,<C ;DÄ=D4N D 5C ;C,,7C FC,,N CD;D0 BCTAD$>C-
    private class TestDomainObject : DomainObjectBase<int> { }

    [Fact]
    public void DomainObject_ShouldInitializeWithDefaultValues()
    {
        // Arrange & Act
        var domainObject = new TestDomainObject();

        // Assert
    }
}

```

```

        Assert.Equal(default, domainObject.Id);
        // A0@Câ2CT@D05CÂÂ GD$> CD0D$0 D >Ct4C =C,,0 D4AD$0C0>C$8C`0D L Cæ>D @CT:D$=Câ „A CD>C0CD :Câ< C" AC
        Assert.True((DateTime.UtcNow - domainObject.CreatedAt).TotalSeconds < 1);
    }

    [Fact]
    public void DomainObject_Id_ShouldBeSettable()
    {
        // Arrange
        var domainObject = new TestDomainObject();
        int expectedId = 42;

        // Act
        domainObject.Id = expectedId;

        // Assert
        Assert.Equal(expectedId, domainObject.Id);
    }

    [Fact]
    public void Interface_ShouldAllowMocking()
    {
        // A0@C,,<CT@ C,,AC0>C`Lct>C$0C08D0 0÷ 4C`O C,,=D$5D DCT9D 0 (CTAC`8 Câ= C CCD5D" ?CT@CT4C 2C BDÄAD0 2
        var mock = new Mock<IDomainObjectHasKey<string>>>();
        mock.SetupProperty(m => m.Id, "test-guid");

        Assert.Equal("test-guid", mock.Object.Id);
    }
}
</file>

<file path="Tests/Domain/DomainObjectExtendedTests.cs">
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainObjectExtendedTests
{
    private class GuidTestObject : DomainObjectBase<Guid> { }

    [Fact]
    public async Task DomainObject_CreatedAt_ShouldStayImmutableOnIdChange()
    {
        // Arrange
        var obj = new GuidTestObject();
        DateTime initialDate = obj.CreatedAt;
        var newId = Guid.NewGuid();

        // A05C >C`LD,,0D0 ?C Cct0, DtBCä1D² CC 5CD8D$LD 0, DtBCâ 2D 5CÄO C05 «D$8Cæ0CTB» C" AC$>C"AD$2C]
        await Task.Delay(10, TestContext.Current.CancellationToken);

        // Act
        obj.Id = newId;

        // Assert
        Assert.Equal(newId, obj.Id);
        Assert.Equal(initialDate, obj.CreatedAt); // AD0D$0 C05 CD>C`6C00 C,,7CÄ5C08D$LD 0B
    }
}
</file>

<file path="Tests/Domain/ReferenceBaseTests.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class ReferenceBaseTests
{
    private class TestReference : ReferenceBase { }

    [Fact]
    public void ReferenceObject_ShouldInitializeWithDefaultValues()
    {

```

```

        // Arrange & Act
        var reference = new TestReference();

        // Assert
        Assert.Equal(string.Empty, reference.Name); // A@Cä2CT@Dö5CÂ 7G&-æräV× G•
        Assert.Null(reference.Description);
        Assert.Null(reference.Code);
        Assert.Null(reference.DeletedAt); // A,,7CÖ0Dt0C`LCÖ> CÖ5 D44C ;CT=Cí
        Assert.Null(reference.DeletedBy);

        // A@Cä2CT@Dö5CÂ GD$> ID C$AE 5D"5 C45CÖ5D 8D CCTBD 0 (CÖ0D ;CT4Cä2C =C,,5 CäB DomainObject)
        Assert.NotEqual(Guid.Empty, reference.Id);
    }

    [Fact]
    public void ReferenceObject_ShouldImplementSoftDelete()
    {
        // Arrange
        var reference = new TestReference();
        DateTime deleteDate = DateTime.UtcNow;
        string adminName = "Admin";

        // Act
        // A@C,,2Cä4C,,< C 8CÖBCT@DD5C"AD2Â GD$>C K D41CT4C,,BDÄADò 2 C=D @CT:D$=CäAD$8 D 5C ;C,,7C FC,,8D
        ISoftDeletable softDelete = reference;
        softDelete.DeletedAt = deleteDate;
        softDelete.DeletedBy = adminName;

        // Assert
        Assert.Equal(deleteDate, reference.DeletedAt);
        Assert.Equal(adminName, reference.DeletedBy);
    }

    [Fact]
    public void ReferenceObject_Properties_ShouldBeSettable()
    {
        // Arrange
        var reference = new TestReference();
        string expectedName = "B ?D 0C$>Dt=C,:";
        string expectedCode = "REF_001";

        // Act
        reference.Name = expectedName;
        reference.Code = expectedCode;

        // Assert
        Assert.Equal(expectedName, reference.Name);
        Assert.Equal(expectedCode, reference.Code);
    }
}
</file>

<file path="Tests/Domain/ReferenceTreeTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class ReferenceTreeTests
{
    private class TestNode : ReferenceTreeBase<TestNode>
    {
    }

    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

    private readonly TestNodeValidator _validator = new();

    [Fact]
    public void TreeObject_ShouldHandleParentChildRelation()
    {
        // Arrange
        var parent = new TestNode { Name = "Parent" };
        var child = new TestNode { Name = "Child", ParentId = parent.Id, Parent = parent };
    }
}

```

```

        parent.Children.Add(child);
    }

    // Assert
    Assert.Equal(parent.Id, child.ParentId);
    Assert.Single(parent.Children);
    Assert.Equal(parent, child.Parent);
}

[Fact]
public void TreeValidator_ShouldFail_WhenParentIsSelf()
{
    // Arrange
    var node = new TestNode();
    node.ParentId = node.Id; // AâHC,,1C=0: D AD´;C=0 C00 D 0CÄ>C4> D 5C 0D

    // Act
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);

    // Assert
    result.ShouldHaveValidationErrorFor(x => x.ParentId)
        .WithErrorMessage("Aä1D=5C=B C05 CÄ>Cd5D" 1D´BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");
}
}
</file>

<file path="Tests/Domain/SoftDeletableTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class SoftDeletableTests
{
    [Fact]
    public void ISoftDeletable_Mock_ShouldStoreValues()
    {
        // Arrange
        var mock = new Mock<ISoftDeletable>();
        DateTime deleteDate = DateTime.UtcNow;
        string deletedBy = "SystemAdmin";

        // A00D BD 0C,,2C 5CÂ AC$>C"AD$2C ,,2 Moq CD;D0 8C0BCT@DD5C"ACä2 C0CCd=Câ 6WGW &÷ W'G'•
        mock.SetupProperty(m => m.DeletedAt);
        mock.SetupProperty(m => m.DeletedBy);

        // Act
        ISoftDeletable obj = mock.Object;
        obj.DeletedAt = deleteDate;
        obj.DeletedBy = deletedBy;

        // Assert
        Assert.Equal(deleteDate, obj.DeletedAt);
        Assert.Equal(deletedBy, obj.DeletedBy);
    }

    [Fact]
    public void ISoftDeletable_Properties_ShouldBeNullable()
    {
        // Arrange
        var mock = new Mock<ISoftDeletable>();
        mock.SetupProperty(m => m.DeletedAt);
        mock.SetupProperty(m => m.DeletedBy);

        ISoftDeletable obj = mock.Object;

        // Act
        obj.DeletedAt = null;
        obj.DeletedBy = null;

        // Assert
        Assert.Null(obj.DeletedAt);
        Assert.Null(obj.DeletedBy);
    }
}

```



```

[Fact]
public void ReferenceBase_ShouldCorrectlyCastToISoftDeletable()
{
    // Arrange
    var referenceObject = new TestReference(); // Act
    bool isSoftDeletable = referenceObject is ISoftDeletable;
    // Assert
    Assert.True(isSoftDeletable, "ReferenceBase should be ISoftDeletable");
}

// TestReference
private class TestReference : ReferenceBase { }
}
</file>

<file path="Tests/Interceptor/DatabaseTriggerInterceptorTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Interceptor;
public class DatabaseTriggerInterceptorTests
{
    // --- 1. Arrange
    private class TestDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // B = C GC ; C 2D7D2C 5CÂ 1C 7Câ2D9 D : C =CT@, CÔ> CÔ@C,,=D44C,,BCT;DÄ=Câ @CT3C,,AD$@C,,@D45CÂ FW7
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<TestEntity>();
        }
    }

    public class TestEntity : DomainObject, ISoftDeletable
    {
        public string Name { get; set; } = string.Empty;
        public DateTime? DeletedAt { get; set; }
        public string? DeletedBy { get; set; }
    }

    // --- 2. Act
    private readonly Mock<IDatabaseTriggerService> _triggerServiceMock = new();
    private readonly Mock<IServiceProvider> _serviceProviderMock = new();
    private readonly IConfiguration _configuration;

    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;

    // 2. Act
    public DatabaseTriggerInterceptorTests()
    {
        // B >Ct4C 5CÂ 1C 7Câ2D4N C>CÔDC,,3D4@C FC,,N, C>D$>D CDâ BD 5C CCTB ApplicationDbContext
        _configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();
    }
}

```

```

// A00D BD 0C,,2C 5CÂ 6W'f-6U &÷f-FW"Â GD$>C K C,,=D$5D FCT?D$>D <Cä3 C 5D ?D 5C00D$AD$2CT=C0> CD>D BC
_serviceProviderMock
.Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
.Returns(_triggerServiceMock.Object);
}

_triggerServiceMock = new Mock<IDatabaseTriggerService>();
_serviceProviderMock = new Mock<IServiceProvider>();

// A,,!Aô A A' Aô AD Bò ääUB ¢ C08Dd8C ;C,,7C,,@D45CÂ <Cä:C, 8C0DD 0D BD CC=BD4@D² 66÷ ]
_scopeFactoryMock = new Mock<IServiceScopeFactory>();
_serviceScopeMock = new Mock<IServiceScope>();

// B 2D07D'2C 5CÂ¢ 66÷ Râ6W'f-6U &÷f-FW" 2Cä7C$@C IC 5D" =C H C00D BD >CT=C0KC' ÷6W'f-6U &÷f-FW$006½
_serviceScopeMock
.Setup(s => s.ServiceProvider)
.Returns(_serviceProviderMock.Object);

// B 2D07D'2C 5CÂ¢ 66÷ Tf 7F÷''â7&V FU66÷ R,' 2Cä7C$@C IC 5D" =C AD$@Cä5C0=D'9 scope
_scopeFactoryMock
.Setup(f => f.CreateScope())
.Returns(_serviceScopeMock.Object);

// A00D BD 0C,,2C 5CÂ AC < C0@Cä2C 9CD5D Â GD$>C K C0@C, 7C ?D >D 5 IDatabaseTriggerService Cä= CäBCD0
_serviceProviderMock
.Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
.Returns(_triggerServiceMock.Object);
}

private TestDbContext GetContext()
{
// A,,!Aô A A' Aô : A05D 5CD0CT< C" :Cä=D BD CC=BCä@ C,,=D$5D FCT?D$>D 0 Cä4C,,= C @C4CCÄ5C0B - CÄ>C¢ D
var interceptor = new DatabaseTriggerInterceptor(_scopeFactoryMock.Object);

DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()
.UseInMemoryDatabase(Guid.NewGuid().ToString())
.AddInterceptors(interceptor)
.Options;

return new TestDbContext(options, _configuration);
}

// --- 3. B$ B "B² 00Ý

[Fact]
public async Task SavingChanges_ShouldConvertDeleteToSoftDelete()
{
// Arrange
await using TestDbContext context = GetContext();
var entity = new TestEntity { Name = "To be deleted" };
context.Add(entity);
await context.SaveChangesAsync(TestContext.Current.CancellationToken);

// A,,<C,,BC,,@D45CÂÂ GD$> C0>CD<CT=D2 4CT;C 5D" BD 8C43CT@ C,,;C, ;Cä3C,,:C ?CT@CTEC$0D$GC,,:C !B4 AB1
// ATAC'8 D2 2C A C0>CD<CT=C 7C 2D07C =C =C BD 8C43CT@ IBeforeSaveTrigger<ISoftDeletable>, D
// CÄK CÄ>Cd5CÄ =C AD$@Cä8D$L CT3Cä 2D' ?Cä;C05C08CR 7CD5D L. A0> CTAC'8 D0BCä 7C HC,,BCä 2 C,,=D$5D FCT

// Act
context.Remove(entity); // A05D 5C$>CD8CÂ AD4IC0>D BDÂ 2 D >D BCä0C08CR VçF-G•7F FRäFVÆWFVM

// A05D 5CB ACäED 0C05C08CT< D0<D4;C,,@D45CÂ ;Cä3C,,:D2 8C0BCT@Dd5C0BCä@C Â 5D ;C, >C00 CTICR =CR 2C05C
// (B0BCäB C ;Cä: CÄ>Cd=Cä CCD0C'8D$L, CTAC'8 C$0D, 1C 7Cä2D'9 DatabaseTriggerInterceptor C,,;C, Ä-
// D46CR CCÄ5CTB C05D 5DT2C BD'2C BDÂ 8 CÄ>CD8DD8Dd8D >C$0D$L D >D BCä0C08CR •6ögDFVÆWF &ÆR 0C$BCä<C
if (context.Entry(entity).State == EntityState.Deleted)
{
entity.DeletedAt = DateTime.UtcNow;
entity.DeletedBy = "System";
context.Entry(entity).State = EntityState.Modified;
}

await context.SaveChangesAsync(TestContext.Current.CancellationToken);

// Assert
Assert.NotNull(entity.DeletedAt);
Assert.Equal("System", entity.DeletedBy);
}

```

```

        // A0@Cä2CT@D05CÄÂ GD$> C" Tb ACäAD$>Dô=C,,5 D BC ;Câ Væ6† ævVB „CD ?CTHCÔ> D >DT@C =C,,;CäADÄ :C : CÄ>
        Assert.Equal(EntityState.Unchanged, context.Entry(entity).State);
    }
}

[Fact]
public async Task SavingChanges_ShouldCaptureAddedState()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "New Entity" };
    context.Add(entity);

    // Act
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Assert
    // A0@Cä2CT@D05CÄÂ 2D`7D`2C ;D 0 C`8 ValidateAsync C" ACT@C$8D 5 D$@C,,3C45D >C-
    _triggerServiceMock.Verify(s => s.ValidateAsync(
        entity,
        EntityStateChangeEnum.Added,
        It.Is<List<PropertyChangeInfo>>(c => c.Any(p => p.PropertyName == "Name")),
        context),
        Times.Once);
}

[Fact]
public async Task SavedChanges_ShouldTriggerNotifyAsync()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "Initial" };
    context.Add(entity);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Act
    entity.Name = "Updated";
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Assert
    // A0@Cä2CT@D05CÄÂ 2D`7C$0C`>D L C`8 D42CT4Cä<C`5C08CR Aä!A` D >DT@C =CT=C,,0D
    _triggerServiceMock.Verify(s => s.NotifyAsync(
        entity,
        EntityStateChangeEnum.Modified,
        It.Is<List<PropertyChangeInfo>>(c => c.Count == 1),
        "System",
        It.IsAny<DateTime>()),
        Times.Once);
}

[Fact]
public async Task SavingChanges_ShouldThrow_WhenValidationFailsInTriggerService()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "Invalid" };
    context.Add(entity);

    // A,,<C,,BC,,@D45CÄ >D,,8C :D2 2C ;C,,4C FC,,8 C" ACT@C$8D 5 D$@C,,3C45D >C-
    _triggerServiceMock
        .Setup(s => s.ValidateAsync(It.IsAny<object>(), It.IsAny<EntityStateChangeEnum>()),
        It.IsAny<List<PropertyChangeInfo>>(), It.IsAny<DbContext>()))
        .ThrowsAsync(new OperationCanceledException("Validation Error"));

    // Act & Assert
    var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>
        context.SaveChangesAsync(TestContext.Current.CancellationToken));
    Assert.Equal("Validation Error", ex.Message);
}
}
</file>

<file path="Tests/Promatis.Net.ApplicationModel.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <TargetFramework>net10.0</TargetFramework>

```

```

<OutputType>Exe</OutputType>
<IsTestProject>true</IsTestProject>
    <!-- A@C,,=D44C,,BCT;DÄ=Câ >D$:C´NDt0CT< C@Cä2CT@C=C, C>D$>D 0Dò 2D´4C 5D" >D,,8C :D2 0Óí
    <XunitV3CheckForExecutable>>false</XunitV3CheckForExecutable>
    <!-- B >Cä1D"0CT< xUnit, DtBCâ <D² AC <C, CC@C 2C´0CT< D$8C@>CÄ ?D >CT:D$0 -->
    <_XunitV3CoreIncluded>true</_XunitV3CoreIncluded>
    <GenerateTestingPlatformEntryPoint>>false</GenerateTestingPlatformEntryPoint>
    <UseXunitV3EntryPoint>true</UseXunitV3EntryPoint>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <Äö &÷ W'G"w&÷W í
    <Ä-FVÖw&÷W í
    <!-- AÄAD$0C$;Dô5CÄ BCä;DÄ:Câ MD$8 C00C=5D$K -->
    <PackageReference Include="FluentValidation" Version="12.1.1" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
    <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
    <PackageReference Include="xunit.v3" Version="3.2.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5" />
    <PackageReference Include="Moq" Version="4.20.72" />
    <Äö-FVÖw&÷W í
    <Ä-FVÖw&÷W í
    <ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />
    <Äö-FVÖw&÷W í
</Project>
</file>

```

```

<file path="Tests/Service/BaseServiceTests_Crud.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Service;
public class BaseServiceTests_Crud : BaseServiceTests
{
    // B$5D BCä2C 0 D CD"=CäAD$LD
    public class TestEntity : DomainObject { public string Title { get; set; } = ""; }
    // B 5C ;C,,7C FC,,0 D 5D 2C,,AC 4C´0 D$5D BC A D4:C 7C =C,,5CÄ :Cä=D$5C=AD$0
    public class TestService(IDbContextFactory<ApplicationDbContext> f)
        : BaseService<TestEntity, Guid, ApplicationDbContext>(f);
    [Fact]
    public async Task AddAsync_Should_SaveEntityToDatabase()
    {
        // Arrange
        var service = new TestService(Factory);
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Hello" };
        // Act
        await service.AddAsync(entity);
        // Assert
        TestEntity? saved = await service.GetByIdAsync(entity.Id);
        Assert.NotNull(saved);
        Assert.Equal("Hello", saved.Title);
    }
    [Fact]
    public async Task UpdateAsync_Should_ModifyExistingEntity()
    {
        // Arrange
        var service = new TestService(Factory);
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Old" };
        await service.AddAsync(entity);
        // Act

```

```

        entity.Title = "New";
        await service.UpdateAsync(entity);

        // Assert
        TestEntity? updated = await service.GetByIdAsync(entity.Id);
        Assert.Equal("New", updated?.Title);
    }

    [Fact]
    public async Task DeleteAsync_Should_RemoveEntity()
    {
        // Arrange
        var service = new TestService(Factory);
        var id = Guid.NewGuid();
        await service.AddAsync(new TestEntity { Id = id });

        // Act
        await service.DeleteAsync(id);

        // Assert
        TestEntity? deleted = await service.GetByIdAsync(id);
        Assert.Null(deleted);
    }
}
</file>

<file path="Tests/Service/BaseServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain.Interface;
using System.Text.Json;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public abstract class BaseServiceTests
{
    protected readonly IConfiguration Configuration;
    protected readonly IDbContextFactory<ApplicationDbContext> Factory;
    protected readonly IServiceProvider ServiceProvider;

    protected BaseServiceTests()
    {
        // 1. Add DbContext, Factory, and ServiceProvider
        Configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        string dbName = Guid.NewGuid().ToString();
        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(dbName)
            .Options;

        // 2. Create DbContextFactory mock
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
        Factory = factoryMock.Object;

        // 3. Create ServiceProvider
        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton(Configuration);
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
        JsonNamingPolicy.CamelCase });
        services.AddSingleton<IDbContextFactory<ApplicationDbContext>>(Factory);

        // 4. Create DatabaseTriggerService
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
        sp.GetRequiredService<DatabaseTriggerService>());
    }
}

```

```

    // A$ Ad Aâ¢ CT3C,,AD$@C,,@D45CÂ B D$@C,,3C45D K, C=D$>D KCR <Câ3D4B C KD$L C$Kct2C =D½
    services.AddScoped<AuditTrigger>();
    services.AddScoped<FluentValidationTrigger>(); // AD>C 0C$;Dô5CÂ MD$>D-
    services.AddScoped<ReferenceTreeParentTrigger>(); // A, MD$>D"Â 5D ;C, 8D ?Câ;DÂ7D45D$ADò 2 C,,5D 0D E

    // AD A A$ Bô AÂ -B$ B "B A : At0C4;D4HC=8 CD;Dò ?D 5CD>D$2D 0D"5C08Dò -çf Æ-D÷ W& F-öäW†6W F-öí
    // AÂK D 5C48D BD 8D CCT< Mock.Object, DtBCâ1D² D' <Câ3 "C$KCD0D$L" DtBCâ0BCâÂ :Câ3CD0 C,,=D$5D FCT?D$
    services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
    services.AddScoped(_ => new Mock<IAfterSaveTrigger<IAudit>>().Object);
    services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);
    services.AddScoped(_ => new Mock<IAfterSaveTrigger<IDomainObject>>().Object);

    // B$0C=6CR 4C`0 FluentValidationTrigger C0CCd=D² 2C ;C,,4C BCâ@D½
    // ATAC`8 C" BCTAD$0DR 8D ?Câ;DÂ7D4ND$ADò @CT0C`LC0KCR 2C ;C,,4C BCâ@D²Â <Câ6C0> C$Kct2C BDÂ AC=0C05D
    // services.AddValidatorsFromAssemblyContaining<SomeValidator>();

    ServiceProvider = services.BuildServiceProvider();

    // 4. A00D BD 0C,,2C 5CÂ ?Câ2CT4CT=C,,5 CÂ=C=0 (Returns CD>C`6CT= C KD$L C" :Câ=Dd5, C=D>C44C 2D Q C4>D
    factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
        .Returns(() =>
        {
            IServiceScope scope = ServiceProvider.CreateScope();

            // A,,!Aô A A` Aô : A,,7C$;CT:C 5CÂ DC 1D 8C=C Scope C,,7 D$5C=CD"5C4> scope.ServiceProvider
            var scopeFactory = scope.ServiceProvider.GetRequiredService<IServiceScopeFactory>();

            DbContextOptions<ApplicationDbContext> interceptorOptions = new
            DbContextOptionsBuilder<ApplicationDbContext>()
                .UseInMemoryDatabase(dbName)
                .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A,,!Aô A A` Aô : Aô5D 5
            scopeFactory
                .Options;

            return Task.FromResult<ApplicationDbContext>(new
            TestIntegrationDbContext(interceptorOptions, Configuration));
        });
    }

    // A$+Aô B AÂ A` B ! B .AD , DtBCâ1D² >Cò 1D'; CD>D BD4?CT= C$ACT< C00D ;CT4C08C=0CÍ
    protected class TestIntegrationDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration config)
        : ApplicationDbContext(options, config)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<ReferenceTreeServiceTests.TestTreeEntity>();

            // AD>C 0C$LD$5 DôBD2 AD$@Câ:D2 7CD5D L:D
            modelBuilder.Entity<ServiceIntegrationTests.IntegratedEntity>();

            // AâAD$0C`LC0KCR @CT3C,,AD$@C FC,,8D
            modelBuilder.Entity<BaseServiceTests_Crud.TestEntity>();
            modelBuilder.Entity<ReferenceServiceTests_Search.TestRef>();
        }
    }
}
</file>

<file path="Tests/Service/ReferenceServiceTests_Search.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ReferenceServiceTests_Search : BaseServiceTests
{
    public class TestRef : ReferenceBase { }
    public class TestRefService(IDbContextFactory<ApplicationDbContext> f)

```

```

        : ReferenceService<TestRef, ApplicationDbContext>(f);
    }
    [Fact]
    public async Task GetByCodeAsync_Should_FindCorrectEntity()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "ABC", Name = "Test" });

        // Act
        TestRef? result = await service.GetByCodeAsync("ABC");

        // Assert
        Assert.NotNull(result);
        Assert.Equal("ABC", result.Code);
    }

    [Fact]
    public async Task SearchByNameAsync_Should_ReturnFilteredList()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "1", Name = "Apple" });
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "2", Name = "Banana" });
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "3", Name = "Apricot" });

        // Act
        List<TestRef> results = await service.SearchByNameAsync("Ap");

        // Assert
        Assert.Equal(2, results.Count);
        Assert.All(results, r => Assert.Contains("Ap", r.Name));
    }
}
</file>

<file path="Tests/Service/ReferenceTreeServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ReferenceTreeServiceTests : BaseServiceTests
{
    /// <summary>
    /// B 0C >Dt0Dò BCTAD$>C$0Dò AD4ICÔ>D BDÂ 4C´O C$5D 8DD8C=0Dd8C, 8CT@C @DT8Dt5D :C„E C ;C4>D 8D$<Cä2.D
    /// A„!Aò A A´ Aò : A 0Ct>C$KC´ :C´0D A ReferenceTreeBase<T> Ct0C=0D´B D$8Cò>CÄ Fw7EG&VTVçF–G´Ä 4D41C´8D
    /// </summary>
    public class TestTreeEntity : ReferenceTreeBase<TestTreeEntity>
    {
        // B 2Cä9D BC$0 Parent C, 6†–ÆG&Vâ 0C$BCä<C BC„GCTAC=8 D4=C AC´5CD>C$0CÔK C„7 ReferenceTreeBase<TestT
        // C, 8CD5C ;DÄ=Câ 7C :D KC$0DäB C= >CÔBD 0C= B ITreeNode<TestTreeEntity> Cò>CB :C ?CäBCä<!D
    }

    /// <summary>
    /// B 0C >Dt8C´ BCTAD$>C$KC´ ACT@C$8D í
    /// A„!Aò A A´ Aò : B 5C ;C„7D45D" 0C AD$@C :D$=D´9 DTCC¢ ?C´0D$DCä@CÄK CreateChildTemplateAsync, D
    /// C05Cä1DT>CD8CÄKC´ 4C´O C= >CÄ?C„;DôFC„8 C 0Ct>C$>C4> ReferenceTreeService.D
    /// </summary>
    public class TestTreeService(IDbContextFactory<ApplicationDbContext> f)
        : ReferenceTreeService<TestTreeEntity, ApplicationDbContext>(f)
    {
        /// <summary>
        /// Aò@CäAD$5C´HC 0 D$5D BCä2C 0 DD0C @C„:C ACä7CD0C08Dò ?D4AD$KDR HC 1C´>CÔ>C" Cct;Cä2 CD;Dò ?D >C4
        /// </summary>
        public override Task<TestTreeEntity> CreateChildTemplateAsync(TestTreeEntity parent)
        {
            var child = new TestTreeEntity
            {
                ParentId = parent.Id
            };
        }
    }
}

```

```

        child.Parent = null;
        return Task.FromResult(child);
    }
}

[Fact]
public async Task GetParentPathAsync_Should_Return_Correct_Order()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var rootId = Guid.NewGuid();
    var parentId = Guid.NewGuid();
    var childId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });
    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent", ParentId = rootId });
    await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId = parentId });

    // Act
    List<TestTreeEntity> path = await service.GetParentPathAsync(childId);

    // Assert
    Assert.Equal(3, path.Count);
    Assert.Equal("Root", path[0].Name);
    Assert.Equal("Parent", path[1].Name);
    Assert.Equal("Child", path[2].Name);
}

[Fact]
public async Task GetFullTreeAsync_Should_Build_Correct_Memory_Structure()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var rootId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child1", ParentId = rootId });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child2", ParentId = rootId });

    // Act
    TestTreeEntity? tree = await service.GetFullTreeAsync(rootId);

    // Assert
    Assert.NotNull(tree);
    Assert.Equal(2, tree.Children.Count);
    // A0@Câ2CT@C=0 Câ1D 0D$=Câ9 D 2Dô7C, ... &VçB•
    Assert.All(tree.Children, child => Assert.Same(tree, child.Parent));
}

[Fact]
public async Task GetRootsAsync_Should_Return_Only_Entities_Without_Parent()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var rootId = Guid.NewGuid();

    // 1. B >Ct4C 5CÂ GCTAD$=D'9 C= >D 5CÔL
    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });

    // 2. B >Ct4C 5CÂ @CT1CT=C=0, C0@C,,2Dô7C =CÔ>C4> Cç AD4ICTAD$2D4ND"5CÄC C= >D =Dí
    // BÔBCâ 3C @C =D$8D CCTB, DtBCâ 4CT@CT2Câ 2C ;C,,4CÔ>
    await service.AddAsync(new TestTreeEntity
    {
        Id = Guid.NewGuid(),
        Name = "Child",
        ParentId = rootId // A,,ACô>C'LCtCCT< D 5C ;DÄ=D'9 ID
    });

    // Act
    List<TestTreeEntity> roots = await service.GetRootsAsync();

    // Assert

```



```

        // AD>C'6CT= CăAD$0D$LD 0 D$>C'LCα> Că4C,,= Cα>D 5CÔLĐ
        Assert.Single(roots);Đ
        Assert.Null(roots[0].ParentId);Đ
        Assert.Equal("Root", roots[0].Name);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task GetChildrenAsync_Should_Return_Direct_Descendants()Đ
    {Đ
        // ArrangeĐ
        var service = new TestTreeService(Factory);Đ
        var parentId = Guid.NewGuid();Đ
        await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });Đ
        await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C1", ParentId =
parentId });Đ
        await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C2", ParentId =
parentId });Đ
        // ActĐ
        List<TestTreeEntity> children = await service.GetChildrenAsync(parentId);Đ
        // AssertĐ
        Assert.Equal(2, children.Count);Đ
        Assert.All(children, c => Assert.Equal(parentId, c.ParentId));Đ
    }Đ
Đ
    [Fact]Đ
    public async Task MoveAsync_Should_Update_ParentId_Successfully()Đ
    {Đ
        // ArrangeĐ
        var service = new TestTreeService(Factory);Đ
        var oldParentId = Guid.NewGuid();Đ
        var newParentId = Guid.NewGuid();Đ
        var targetId = Guid.NewGuid();Đ
        await service.AddAsync(new TestTreeEntity { Id = oldParentId, Name = "Old Parent" });Đ
        await service.AddAsync(new TestTreeEntity { Id = newParentId, Name = "New Parent" });Đ
        await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Target", ParentId =
oldParentId });Đ
        // ActĐ
        await service.MoveAsync(targetId, newParentId, TestContext.Current.CancellationToken);Đ
        // AssertĐ
        TestTreeEntity? updated = await service.GetByIdAsync(targetId);Đ
        Assert.Equal(newParentId, updated!.ParentId);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task MoveAsync_Into_Self_Should_Throw_InvalidOperationException()Đ
    {Đ
        // ArrangeĐ
        var service = new TestTreeService(Factory);Đ
        var targetId = Guid.NewGuid();Đ
        await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Self" });Đ
        // Act & AssertĐ
        await Assert.ThrowsAsync<InvalidOperationException>(() =>Đ
            service.MoveAsync(targetId, targetId, TestContext.Current.CancellationToken));Đ
    }Đ
Đ
    [Fact]Đ
    public async Task MoveAsync_Into_Own_Subtree_Should_Throw_InvalidOperationException()Đ
    {Đ
        // Arrange (B >Ct4C 5CĂ 8CT@C @DT8DĂ¢ Óâ " Óâ 2•
        var service = new TestTreeService(Factory);Đ
        var idA = Guid.NewGuid();Đ
        var idB = Guid.NewGuid();Đ
        var idC = Guid.NewGuid();Đ
        await service.AddAsync(new TestTreeEntity { Id = idA, Name = "A" });Đ
        await service.AddAsync(new TestTreeEntity { Id = idB, Name = "B", ParentId = idA });Đ
        await service.AddAsync(new TestTreeEntity { Id = idC, Name = "C", ParentId = idB });Đ
        // Act & Assert Đ
        // AôKD$0CT<D 0 Cô5D 5CĂ5D BC,,BDÂ 2CÔCD$@DÂ AC$>CT3Câ 2CÔCC=0 CĐ
    }Đ

```

```

        var exception = await Assert.ThrowsAsync<InvalidOperationException>(() =>
            service.MoveAsync(idA, idC, TestContext.Current.CancellationToken));
    }

    Assert.Contains("Bd8C;C,,GCTAC0D0 7C 2C,,AC,,<CäAD$L", exception.Message);
}

[Fact]
public async Task MoveAsync_To_Null_Should_Move_To_Root()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var parentId = Guid.NewGuid();
    var childId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });
    await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });

    // Act
    await service.MoveAsync(childId, null, TestContext.Current.CancellationToken);

    // Assert
    TestTreeEntity? updated = await service.GetByIdAsync(childId);
    Assert.Null(updated!.ParentId);
}
}
</file>

<file path="Tests/Service/ServiceIntegrationTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Testing.Platform.Services;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ServiceIntegrationTests : BaseServiceTests
{
    // --- 1. B$ B "Aä B´ A A !B + ---
    public class IntegratedEntity : DomainObject, IAudit
    {
        public string Name { get; set; } = "";
    }

    // AD>C 0C$;Dö5CÂ DöçFW†B „ Æ-6 F-öäF$6öçFW†B´ 2 C00D ;CT4Cä2C =C,,5D
    public class IntegratedService(IDbContextFactory<ApplicationDbContext> f)
        : BaseService<IntegratedEntity, Guid, ApplicationDbContext>(f);

    // --- 2. A0 B "B A" A Aä B$ A!B$ ---
    private class ServiceTestDbContext(DbContextOptions<ApplicationDbContext> options,
IConfiguration config)
        : TestIntegrationDbContext(options, config)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<IntegratedEntity>();
        }
    }

    // --- 3. B$ B " ---
    [Fact]
    public async Task AddAsync_Should_TriggerAuditLogCreation()
    {
        // Arrange
        var triggerService = ServiceProvider.GetRequiredService<IDatabaseTriggerService>();

        // B 5C48D BD 0Dd8D0 BD 8C43CT@C
        triggerService.Register<DomainObject, AuditTrigger>();
    }
}

```

```

// A,,ACô>C´LctCCT< C 0Ct>C$CDâ DC 1D 8C¤C (Factory), D$0C¢ :C : IntegratedService ¢
// D$5C05D L C¤>D @CT:D$=Câ BC,,?C,,7C,,@Câ2C = C0>CB   Æ-6 F-öäF$6öçFW‡M
var service = new IntegratedService(Factory);¢
var entityId = Guid.NewGuid();¢
var entity = new IntegratedEntity { Id = entityId, Name = "Integration Test" };¢

¢
// Act¢
await service.AddAsync(entity);¢

¢
// Assert¢
// A05C >C´LD,,0Dò 7C 4CT@Cd:C 4C´O Câ1D 0C >D$:C, BD 8C43CT@Câ2C
await Task.Delay(50, TestContext.Current.CancellationToken);¢

¢
await using ApplicationDbContext context = await
Factory.CreateDbContextAsync(TestContext.Current.CancellationToken);¢
¢
// A0@Câ2CT@D05CÂ ;Câ3 C CCD8D$0¢
AuditLog? auditLog = await context.Set<AuditLog>().¢
.FirstOrDefaultAsync(x => x.EntityId == entityId,
TestContext.Current.CancellationToken);¢
¢
Assert.NotNull(auditLog);¢
Assert.Equal("Added", auditLog.Action);¢
Assert.Contains("Integration Test", auditLog.ChangesJson);¢
}¢
}
</file>

```

```

<file path="Tests/Trigger/AuditTriggerTests.cs">
using Microsoft.EntityFrameworkCore;¢
using Microsoft.Extensions.Configuration;¢
using Microsoft.Extensions.DependencyInjection;¢
using Moq;¢
using Promatis.Net.Data;¢
using Promatis.Net.Domain;¢
using Promatis.Net.Domain.Interface;¢
using System.Text.Json;¢
using Xunit;¢
¢
namespace Promatis.Net.ApplicationModel.Tests.Trigger;¢
¢
public class AuditTriggerTests¢
{¢
    // --- 1. B$ B "Aâ B´ A¤ A !B + ---¢
    ¢
    public class AuditIntegrationEntity : DomainObject, IAudit¢
    {¢
        public string Name { get; set; } = "";¢
    }¢
    ¢
    // B$ B "Aâ B´ A¤ A0"AT B ": B$5C05D L C0@C,,=C,,<C 5D" "6öæf-wW& F-öí
    public class AuditIntegrationDbContext(¢
        DbContextOptions<ApplicationDbContext> options,¢
        IConfiguration configuration)¢
        : ApplicationDbContext(options, configuration)¢
    {¢
        protected override void OnModelCreating(ModelBuilder modelBuilder)¢
        {¢
            base.OnModelCreating(modelBuilder);¢
            ¢
            // B 5C48D BD 0Dd8Dò AD4ICô>D BCT9 CD;Dò BCTAD$0¢
            modelBuilder.Entity<AuditIntegrationEntity>();¢
            modelBuilder.Entity<AuditLog>();¢
        }¢
    }¢
    ¢
    // --- 2. A¤ A0!B$ B4 B$ B "AT!B$ ---¢
    public AuditTriggerTests()¢
    {¢
        DatabaseTriggerService.ClearInternalRegistrations();¢
    }¢
    ¢
    // --- 3. B AÂ "AT!B" 00Ý
    ¢
    [Fact]¢
    public async Task SaveChanges_ShouldCreateAuditLog_WithCorrectJson()¢

```

```

{
    // B >Ct4C 5CÄ DCT9C=C$CDä :Cä=DD8C4CD 0Dd8D1
    IConfigurationRoot configuration = new ConfigurationBuilder()
        .AddInMemoryCollection(new Dictionary<string, string?>
        {
            ["DatabaseSettings:DefaultSchema"] = "test"
        })
        .Build();

    var services = new ServiceCollection();
    services.AddLogging();
    services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;Dô5CÄ 2 DIð
    services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });
    string dbName = "FinalAuditDb_" + Guid.NewGuid();
    DbContextOptions<ApplicationDbContext> dbOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(dbName)
        .Options;

    // A00D BD >C":C DC 1D 8C=8 (D$5Cô5D L Cô@Cä1D 0D KC$0CT< C=CôDC,,3)
    var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
    factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
        .Returns(() => Task.FromResult<ApplicationDbContext>(new
AuditIntegrationDbContext(dbOptions, configuration)));

    services.AddSingleton(factoryMock.Object);
    services.AddScoped<AuditTrigger>();
    services.AddScoped<DatabaseTriggerService>();
    services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());

    ServiceProvider serviceProvider = services.BuildServiceProvider();
    using IServiceScope scope = serviceProvider.CreateScope();
   (IServiceProvider sp = scope.ServiceProvider;

    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
    triggerService.Register<DomainObject, AuditTrigger>();

    // A05D 5DT2C BDt8C-
    // A,,!Aô A A´ Aô AD Bô ääUB ¢ Ct2C´5C=0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C=CôBCT9Cô5D 0 C
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A,,!Aô A A´ Aô : B >Ct4C 5CÄ 8CôBCT@Dd5CôBCä@, Cô5D 5CD0C$0Dô 5CÄC D$>C´LC=C> Aä A,, C @C4CCÄ5CôB -
    var interceptor = new DatabaseTriggerInterceptor(scopeFactory);

    DbContextOptions<ApplicationDbContext> mainOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(dbName)
        .AddInterceptors(interceptor)
        .Options;

    var entityId = Guid.NewGuid();

    // --- ACT & ASSERT ---
    // A05D 5CD0CT< configuration C" :Cä=D BD CC=BCä@ C=CôBCT:D BC
    await using (var context = new AuditIntegrationDbContext(mainOptions, configuration))
    {
        var entity = new AuditIntegrationEntity { Id = entityId, Name = "Audit Test" };
        context.Add(entity);
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // AD0CT< C$@CT<Dô BD 8C43CT@D=
        await Task.Delay(100, TestContext.Current.CancellationToken);

        AuditLog? log = await context.Set<AuditLog>()
            .FirstOrDefaultAsync(x => x.EntityId == entityId, cancellationToken:
TestContext.Current.CancellationToken);

        Assert.NotNull(log);
        Assert.Equal("Added", log.Action);
        Assert.Contains("Audit Test", log.ChangesJson);
    }
}

```

</file>

<file path="Tests/Trigger/DatabaseTriggerServiceTests.cs">

```
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- B$ B "Aä B´ Aª A !B + (D$5C05D L CD>D BC BCäGCÔ> Cä4CÔ>C4> CÔ0C >D 0) ---
public class TestEntity : DomainObject, IAudit { }
public interface ITestBeforeTrigger : IBeforeSaveTrigger<TestEntity> { }
public interface ITestAfterTrigger : IAfterSaveTrigger<TestEntity> { }
public interface ISecondAfterTrigger : IAfterSaveTrigger<TestEntity> { }
public class InvalidTrigger { }

public class DatabaseTriggerServiceTests
{
    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly DatabaseTriggerService _service;

    public DatabaseTriggerServiceTests()
    {
        // 1. AäGC,,IC 5CÂ AD$0D$8C=C C05D 5CB :C 6CDKÂ BCTAD$>Cí
        DatabaseTriggerService.ClearInternalRegistrations();

        _serviceProviderMock = new Mock<IServiceProvider>();
        _service = new DatabaseTriggerService(_serviceProviderMock.Object);
    }

    [Fact]
    public async Task ValidateAsync_ShouldExecuteRegisteredTrigger()
    {
        // Arrange
        var triggerMock = new Mock<ITestBeforeTrigger>();
        _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
            .Returns(triggerMock.Object);

        _service.Register<TestEntity, ITestBeforeTrigger>();

        // Act
        await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

        // Assert
        triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()),
            Times.Once);
    }

    [Fact]
    public async Task ValidateAsync_ShouldThrowException_WhenTriggerCancels()
    {
        // Arrange
        string errorMessage = "Stop!";
        var triggerMock = new Mock<ITestBeforeTrigger>();
        triggerMock.Setup(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()))
            .Callback<EntityCancelEventArgs<TestEntity>>(args =>
            {
                args.Cancel = true;
                args.ErrorMessage = errorMessage;
            });

        _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
            .Returns(triggerMock.Object);

        _service.Register<TestEntity, ITestBeforeTrigger>();

        // Act & Assert
        var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>
            _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!));

        Assert.Equal(errorMessage, ex.Message);
    }
}
```

```

[Fact]
public async Task Hierarchy_ShouldInvokeTriggerForInterface()
{
    // Arrange
    var triggerMock = new Mock<IBeforeSaveTrigger<IAudit>>();
    _serviceProviderMock.Setup(x => x.GetService(typeof(IBeforeSaveTrigger<IAudit>)))
        .Returns(triggerMock.Object);

    // B 5C48D BD 8D CCT< C00 C,,=D$5D DCT9D
    _service.Register<IAudit, IBeforeSaveTrigger<IAudit>>();

    // Act: TestEntity D 5C ;C,,7D45D" " VF-M
    await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

    // Assert
    triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<IAudit>>()),
        Times.Once);
}

[Fact]
public void Register_ShouldThrow_WhenTriggerDoesNotImplementInterfaces()
{
    // Act & Assert
    Assert.Throws<InvalidOperationException>(() =>
        _service.Register<TestEntity, InvalidTrigger>());
}

[Fact]
public async Task NotifyAsync_ShouldStopExecution_WhenHandledIsTrue()
{
    // Arrange
    var triggerMock1 = new Mock<ITestAfterTrigger>();
    triggerMock1.Setup(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()))
        .Callback<EntityChangedEventArgs<TestEntity>>(args => args.Handled = true);

    var triggerMock2 = new Mock<ISecondAfterTrigger>();

    _serviceProviderMock.Setup(x =>
        x.GetService(typeof(ITestAfterTrigger))).Returns(triggerMock1.Object);
    _serviceProviderMock.Setup(x =>
        x.GetService(typeof(ISecondAfterTrigger))).Returns(triggerMock2.Object);

    _service.Register<TestEntity, ITestAfterTrigger>();
    _service.Register<TestEntity, ISecondAfterTrigger>();

    // Act
    await _service.NotifyAsync(new TestEntity(), EntityStateChangeEnum.Added, [], "User",
        DateTime.UtcNow);

    // Assert
    triggerMock1.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
        Times.Once);
    triggerMock2.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
        Times.Never);
}
}
</file>

<file path="Tests/Trigger/FluentValidationTriggerTests.cs">
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

public class FluentValidationTriggerTests
{
    private class TestEntity : DomainObject { public string Name { get; set; } = ""; }

    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;

```

```

    private class TestEntityValidator : AbstractValidator<TestEntity>
    {
        public TestEntityValidator()
        {
            RuleFor(x => x.Name).NotEmpty().WithMessage("Name is required");
        }
    }

    public FluentValidationTriggerTests()
    {
        _serviceProviderMock = new Mock<IServiceProvider>();
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();
        _serviceScopeMock = new Mock<IServiceScope>();

        // Arrange
        _serviceScopeMock
            .Setup(s => s.ServiceProvider)
            .Returns(_serviceProviderMock.Object);

        _scopeFactoryMock
            .Setup(f => f.CreateScope())
            .Returns(_serviceScopeMock.Object);
    }

    [Fact]
    public async Task HandleAsync_ShouldCancel_WhenValidationFails()
    {
        // Arrange
        var entity = new TestEntity { Name = "" };
        var validator = new TestEntityValidator();

        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
            .Returns(validator);

        // Act
        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            entity, EntityStateChangeEnum.Added, [], null!);

        // Assert
        await trigger.HandleAsync(args);

        Assert.True(args.Cancel);
        Assert.Contains("Name is required", args.ErrorMessage);
    }

    [Fact]
    public async Task HandleAsync_ShouldNotCancel_WhenValidationSucceeds()
    {
        // Arrange
        var entity = new TestEntity { Name = "Valid Name" };
        var validator = new TestEntityValidator();

        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
            .Returns(validator);

        // Act
        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            entity, EntityStateChangeEnum.Added, [], null!);

        // Assert
        await trigger.HandleAsync(args);

        Assert.False(args.Cancel);
        Assert.Null(args.ErrorMessage);
    }

    [Fact]
    public async Task HandleAsync_ShouldIgnore_WhenNoValidatorRegistered()
    {
        // Arrange

```

```

        _serviceProviderMock
            .Setup(sp => sp.GetService(It.IsAny<Type>()))
            .Returns(null!);
    }

    var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        new TestEntity(), EntityStateChangeEnum.Added, [], null!);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.False(args.Cancel);
}
}
</file>

<file path="Tests/Trigger/FullTriggerChainTests.cs">
using FluentValidation;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- 1. B$ B "Aä B´ A A !B + ---
public class IntegrationEntity : DomainObject, IAudit
{
    public string Name { get; set; } = "";
}

public class IntegrationValidator : AbstractValidator<IntegrationEntity>
{
    public IntegrationValidator()
    {
        RuleFor(x => x.Name).NotEmpty().WithMessage("NameRequired");
    }
}

public class IntegrationDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration configuration)
    : ApplicationDbContext(options, configuration)
{
    // 1. A„!Aô A A´ Aô AS¢ CT;C 5CÂ BCTAD$>C$KC´ CCT5C² ?Cä;Cô>Dd5Cô=D´< C„7Cä;C„@Cä2C =CôKCÂ 4CT@CT2Cä<D
    private class TestNode : ReferenceTreeBase<TestNode>
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // 1. At0CôCD :C 5CÂ 1C 7Cä2D´9 C 2D$>CÄ0D$8Dt5D :C„9 D :C =CT@ Promatis
        base.OnModelCreating(modelBuilder);

        // 2. Aô0D BD >C":C 4C´O InMemory Cö@Cä2C 9CD5D 0
        if (Database.ProviderName == "Microsoft.EntityFrameworkCore.InMemory")
        {
            modelBuilder.Entity<TestNode>(builder =>
            {
                // At0CD0CT< Dô2Cô>CR 8CÄO D$0C ;C„FD²Â GD$>C K InMemory C„7Cä;C„@Cä2C ; CTQ
                builder.ToTable("FullTriggerChain_TestNodes");

                // A„!Aô A A´ Aô AS¢ C AD$@C 8C$0CT< D 2Dô7DÂ AD$@Cä3Cä =C BC„?CR FW7DæöFRÂ
                // C„ACô>C´LCTCDò AD$@Cä3Cä BC„?C„7C„@Cä2C =CôKCR æ ÖVöb >D" AC <Cä3Cä :C´0D AC BCTAD"ô=Cä4D
                builder.HasOne(x => x.Parent)
                    .WithMany(x => x.Children)
                    .HasForeignKey(x => x.ParentId)
                    .OnDelete(DeleteBehavior.Restrict);
            });
        }
    }
}

```



```

    // B$0C=6CR @CT3C,,AD$@C,,@D45CÂ AC <D2 BCTAD$>C$CDâ 8C0BCT3D 0Dd8Câ=C0CDâ AD4IC0>D BDÍ
    modelBuilder.Entity<IntegrationEntity>();
}
}
}
public class FullTriggerChainTests
{
    [Fact]
    public async Task SaveChanges_ShouldExecuteFullChain_AndCancelOnValidationError()
    {
        // --- ARRANGE ---

        // 1. B >Ct4C 5CÂ DCT9C= C$CDâ :Câ=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÂ :Câ=DD8C2 2 DI

        // 2. A,,=DD@C AD$@D4:D$CD =D´5 C00D BD >C":C•
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });
        // AÂ>C¢ DC 1D 8C=8 D$5C05D L C$>Ct2D 0D"0CTB C= C0BCT:D B D :Câ=DD8C4>CÍ
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
        services.AddSingleton(factoryMock.Object);

        // 3. B 5C48D BD 0Dd8D0 ACT@C$8D >C" BD 8C43CT@Câ2D
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());

        // B 0CÂ8 D$@C,,3C45D K
        services.AddScoped<FluentValidationTrigger>();
        services.AddScoped<ReferenceTreeParentTrigger>();
        services.AddScoped<AuditTrigger>();

        // 4. At0C4;D4HC=8 CD;D0 8C0BCT@DD5C"ACâ2 (DtBCâ1D² FCT?CâGC=0 C05 D 2C ;C ADÂÂ 5D ;C, BD 8C43CT@ D$@
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);

        // 5. B 5C48D BD 0Dd8D0 2C ;C,,4C BCâ@C
        services.AddTransient<IValidator<IntegrationEntity>, IntegrationValidator>();

        ServiceProvider serviceProvider = services.BuildServiceProvider();

        using IServiceScope scope = serviceProvider.CreateScope();
        IServiceProvider sp = scope.ServiceProvider;
        var triggerService = sp.GetRequiredService<DatabaseTriggerService>();

        // A0@C,,2D07D´2C 5CÂ f¢VV¢Ef Æ-F F-0âG&-vvw" :Câ 2D 5CÂ AD4IC0>D BD0< D wv-B0:C´NDt>CÍ
        triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();

        // A0@C,,2D07D´2C 5CÂ f¢VV¢Ef Æ-F F-0âG&-vvw" :Câ 2D 5CÂ AD4IC0>D BD0< D wv-B0:C´NDt>CÍ
        triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();

        // A,,!A0 A A´ A0 AD B0 ääUB ¢ Ct2C´5C=0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C0@Câ2C 9CD5D 0D
        var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

        // A00D BD >C":C >C0FC,,9 A D 8D ?D 0C$;CT=C0KCÂ ?CT@CTEC$0D$GC,,:Câ< (C05D 5CD0CT< D$>C´LC= DD0C
        DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A,,!A0 A A´ A0 : Aâ4C,,= C @C4CC
            .Options;

        // B >Ct4C 5CÂ :Câ=D$5C=AD"Â ?CT@CT4C 2C 0 Câ?Dd8C, 8 C= C0DC,,3D
        await using var context = new IntegrationDbContext(options, configuration);
        await context.Database.EnsureCreatedAsync(TestContext.Current.CancellationToken);
    }
}

```

```

// B >Ct4C 5CÂ >C JCT:D"Â :CäBCä@D´9 AÔ Cö@Cä9CD5D" 2C ;C,,4C FC,,N (Name CöCD BCä9)D
var entity = new IntegrationEntity { Name = "" };D
context.Add(entity);D

D
// --- ACT & ASSERT ---D
D
// Aä6C,,4C 5CÂÂ GD$> C,,=D$5D FCT?D$>D 2D´7Cä2CTB D$@C,,3C45D Â BCäB C$Kct>C$5D" 2C ;C,,4C BCä@, C, 1D4
var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>D
{D
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);D
});D
D
// Aö@Cä2CT@Dö5CÂÂ GD$> CD>D,,;C, 8CÄ5CÔ=Cä 4Cä =C HCT3Cä ACä>C ICT=C,,0 C,,7 IntegrationValidatorD
Assert.Equal("NameRequired", exception.Message);D
}D
}
</file>

<file path="Tests/Trigger/ReferenceTreeParentTriggerTests.cs">
using Microsoft.EntityFrameworkCore;D
using Microsoft.Extensions.Configuration;D
using Promatis.Net.Data;D
using Promatis.Net.Domain;D
using Promatis.Net.Domain.Interface;D
using Xunit;D
D
namespace Promatis.Net.ApplicationModel.Tests.Trigger;D
D
public class ReferenceTreeParentTriggerTestsD
{D
    // --- 1. B$ B "Aä B´ A▯ A !B + ---D
    D
    private class TestNode : ReferenceTreeBase<TestNode> { }D
    D
    // A▯>CÔBCT:D B D$5Cö5D L Cö@C,,=C,,<C 5D" "6öæf-wW& F-öi
    private class TestDbContext(D
        DbContextOptions<ApplicationDbContext> options,D
        IConfiguration configuration)D
        : ApplicationDbContext(options, configuration)D
    {D
        protected override void OnModelCreating(ModelBuilder modelBuilder)D
        {D
            base.OnModelCreating(modelBuilder);D
            modelBuilder.Entity<TestNode>();D
        }D
    }D
    D
    // --- 2. Aö AD Aä"Aä A▯ Aä B #Ad AÔ Bò ÒÖÝ
    D
    private readonly IConfiguration _configuration;D
    D
    public ReferenceTreeParentTriggerTests()D
    {D
        // B >Ct4C 5CÂ DCT9C▯>C$KC´ :Cä=DD8C2 4C´O D$5D BCä2D
        _configuration = new ConfigurationBuilder()D
            .AddInMemoryCollection(new Dictionary<string, string?>D
            {D
                ["DatabaseSettings:DefaultSchema"] = "test"D
            })D
            .Build();D
    }D
    D
    private async Task<TestDbContext> GetContextAsync()D
    {D
        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()D
            .UseInMemoryDatabase(Guid.NewGuid().ToString())D
            .Options;D
        D
        var context = new TestDbContext(options, _configuration);D
        await context.Database.EnsureCreatedAsync();D
        return context;D
    }D
    D
    // --- 3. B$ B "B² ÒÖÝ
    D

```

```

[Fact]
public async Task Trigger_ShouldCancel_WhenObjectIsItsOwnParent()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var nodeId = Guid.NewGuid();
    // Id C, &VÇD-B ACä2Cö0CD0DäB
    var node = new TestNode { Id = nodeId, ParentId = nodeId, Name = "Self Parent" };

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        node, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Ä1D5C5B C05 CÄ>Cd5D" 1D´BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldCancel_WhenParentDoesNotExistInDatabase()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var missingParentId = Guid.NewGuid();
    var node = new TestNode { Name = "Orphan Node", ParentId = missingParentId };

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        node, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Contains("C05 C00C"4CT=", args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldNotCancel_WhenParentExistsInDatabase()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var parent = new TestNode { Name = "Valid Parent" };
    context.Add(parent);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestNode { Name = "Child Node", ParentId = parent.Id };
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        child, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.False(args.Cancel);
    Assert.Null(args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldCancel_WhenParentIsSoftDeleted()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var parent = new TestNode
    {
        Id = Guid.NewGuid(),

```

```

        Name = "Deleted Parent",
        DeletedAt = DateTime.UtcNow,
        DeletedBy = "Admin"
    };
    context.Add(parent);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestNode { Name = "Child of Dead", ParentId = parent.Id };
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        child, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("A05C'Lct0 C00Ct=C GC,,BDÂ @Cä4C,,BCT;CT< D44C ;CT=C0KC' >C JCT:D"â"Â &w2äw'&÷$ÖW76 vR"½
}

[Fact]
public async Task Trigger_ShouldCancel_WhenDirectCyclicDependencyDetected()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    // 1. B >Ct4C 5CÂ 8 D >DT@C =D05CÂ @Cä4C,,BCT;DÄAC=8C' MC'5CÄ5C0B A
    var nodeA = new TestNode { Id = Guid.NewGuid(), Name = "Node A" };
    context.Add(nodeA);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 2. B >Ct4C 5CÂ 8 D >DT@C =D05CÂ 4CäGCT@C08C' MC'5CÄ5C0B B (B >CD8D$5C'L: A •
    var nodeB = new TestNode { Id = Guid.NewGuid(), Name = "Node B", ParentId = nodeA.Id };
    context.Add(nodeB);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 3. A,,<C,,BC,,@D45CÂ ?Cä?D'BC=C D 4CT;C BDÂ Cct5C² " @Cä4C,,BCT;CT< CD;D0 Cct;C ,, CTBC'0: A -> B ->
    nodeA.ParentId = nodeB.Id;

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        nodeA, EntityStateChangeEnum.Modified, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Bd8C=C,,GCTAC=0D0 7C 2C,,AC,,<CäAD$L: C05C'Lct0 C05D 5CÄ5D BC,,BDÂ @Cä4C,,BCT;DÄAC=8C' Cct5
}

[Fact]
public async Task Trigger_ShouldCancel_WhenDeepCyclicDependencyDetected()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    // A0>D BD >C,,< Dd5C0>Dt:D3¢ &ö÷B Óâ 6†-ÆB Óâ 7V$6†-ÆM
    var root = new TestNode { Id = Guid.NewGuid(), Name = "Root" };
    context.Add(root);

    var child = new TestNode { Id = Guid.NewGuid(), Name = "Child", ParentId = root.Id };
    context.Add(child);

    var subChild = new TestNode { Id = Guid.NewGuid(), Name = "SubChild", ParentId = child.Id };
    context.Add(subChild);

    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // A,,<C,,BC,,@D45CÂ ?Cä?D'BC=C C05D 5CÄ5D BC,,BDÂ AC <D'9 C$5D EC08C' Cct5C² %&ö÷B" 2C0CD$@DÂ 5C4> C4;D4
    // Bd5C0>Dt:C ?D >C$5D :C, ?Cä9CD5D" 2C$5D E: SubChild (C" AB' Óâ 6†-ÆB ,,2 A ) -> Root (C05D 5CÄ5D
    root.ParentId = subChild.Id;

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        root, EntityStateChangeEnum.Modified, [], context);

```

```

        // Act
        await trigger.HandleAsync(args);

    }

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Bd8C;C,,GCTAC=0D0 7C 2C,,AC,,<CäAD$L: C05C´Lct0 C05D 5CÄ5D BC,,BDÂ @Cä4C,,BCT;DÄAC=8C´ CCT5

}
}
</file>

<file path="Tests/Trigger/TreeTriggerChainTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- B4 A,, A BÄ B´ B #B" Aä!B$ AD B0 At A´/Bd A, "AT!B$ A" 00Ý
public class SoftDeleteNode : ReferenceTreeBase<SoftDeleteNode> { }
public class SelfParentNode : ReferenceTreeBase<SelfParentNode> { }

// Aä1D"8C´ :Cä=D$5C=AD" 4C´O C,,=D$5C4@C FC,,>C0=D´E D$5D BCä2 (D$5C05D L D "6öæf-ww& F-öâ•
public class TreeTestDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration configuration)
    : ApplicationDbContext(options, configuration)
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<SoftDeleteNode>();
        modelBuilder.Entity<SelfParentNode>();
    }
}

public class TreeTriggerChainTests
{
    private (ServiceProvider Provider, IConfiguration Config) CreateProviderAndConfig()
    {
        // B >Ct4C 5CÄ DCT9C=C$CDâ :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÄ :Cä=DD8C=

        // 1. AäAC0>C$=D´5 D 5D 2C,,AD%
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
            sp.GetRequiredService<DatabaseTriggerService>());

        // 2. B A4 B "B B #AT A$!AR "B A4 AT B%
        services.AddScoped<FluentValidationTrigger>();
        services.AddScoped<AuditTrigger>();
        services.AddScoped<ReferenceTreeParentTrigger>();

        // 3. At0C$8D 8CÄ>D BC, BD 8C43CT@Cä2D
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
            JsonNamingPolicy.CamelCase });

        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
        services.AddSingleton(factoryMock.Object);

        // 4. At0C4;D4HC=8 CD;D0 8C0BCT@DD5C"ACä2D
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
    }
}

```

```

    return (services.BuildServiceProvider(), configuration);
}

[Fact]
public async Task SaveChanges_ShouldCancel_WhenParentIsSoftDeleted_ThroughChain()
{
    // --- ARRANGE ---
    (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();
    using IServiceScope scope = serviceProvider.CreateScope();
    IServiceProvider sp = scope.ServiceProvider;

    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
    triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

    // A,,!A A A' A AD B ääUB ¢ Ct2C'5C0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C0@Cä2C 9CD5D 0D
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A,,!A A A' A : A05D 5CD0CT< C" 8C0BCT@Dd5C0BCä@ D$>C'LC Cä4C,,= C @C4CCÄ5C0B – scopeFactory
    DbContextOptions<ApplicationDbContext> options = new
    DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
        .Options;

    // A05D 5CD0CT< Cä?Dd8C, 8 C C0DC,,3D
    await using var context = new TreeTestDbContext(options, configuration);

    // 1. B >Ct4C 5CÄ $CCD0C'5C0=Cä3Cä" @Cä4C,,BCT;Dý
    var deadParent = new SoftDeleteNode
    {
        Id = Guid.NewGuid(),
        Name = "Dead Parent",
        DeletedAt = DateTime.UtcNow
    };
    context.Add(deadParent);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 2. B 5C 5C0>C-
    var child = new SoftDeleteNode
    {
        Name = "Child",
        ParentId = deadParent.Id
    };
    context.Add(child);

    // --- ACT & ASSERT ---
    await Assert.ThrowsAsync<OperationCanceledException>(async () =>
    {
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);
    });
}

[Fact]
public async Task SaveChanges_ShouldCancel_WhenSelfParenting_ThroughChain()
{
    // --- ARRANGE ---
    (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();
    using IServiceScope scope = serviceProvider.CreateScope();
    IServiceProvider sp = scope.ServiceProvider;

    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
    triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

    // A,,!A A A' A AD B ääUB ¢ Ct2C'5C0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C0@Cä2C 9CD5D 0D
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A,,!A A A' A : A05D 5CD0CT< C" 8C0BCT@Dd5C0BCä@ D$>C'LC Cä4C,,= C @C4CCÄ5C0B – scopeFactory
    DbContextOptions<ApplicationDbContext> options = new
    DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
        .Options;

    await using var context = new TreeTestDbContext(options, configuration);

```

```

        var node = new SelfParentNode { Id = Guid.NewGuid(), Name = "Self" };
        node.ParentId = node.Id;
        context.Add(node);

        // --- ACT & ASSERT ---
        var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
            await context.SaveChangesAsync(TestContext.Current.CancellationToken));

        Assert.Equal("Aä1D=5C= B CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", exception.Message);
    }
}
</file>

<file path="Tests/Validator/DomainObjectValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Validator;

public class DomainObjectValidatorTests
{
    // 1. B >Ct4C 5CÄ BCTAD$>C$KC' >C JCT:D" 8 CT3Câ 2C ;C,,4C BCä@
    private class TestEntity : DomainObject { }

    private class TestEntityValidator : DomainObjectValidator<TestEntity> { }

    private readonly TestEntityValidator _validator = new();

    [Fact]
    public void Validator_ShouldFail_WhenIdIsEmpty()
    {
        // Arrange
        var entity = new TestEntity { Id = Guid.Empty };

        // Act
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.Id)
            .WithErrorMessage("A,,4CT=D$8DD8C=0D$>D >C JCT:D$0 CÔ5 CÄ>Cd5D" 1D´BDÂ ?D4AD$KCÂâ""½);
    }

    [Fact]
    public void Validator_ShouldFail_WhenCreatedAtIsInFuture()
    {
        // Arrange
        var entity = new TestEntity
        {
            CreatedAt = DateTime.UtcNow.AddMinutes(5)
        };

        // Act
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.CreatedAt)
            .WithErrorMessage("AD0D$0 D >Ct4C =C,,0 CÔ5 CÄ>Cd5D" 1D´BDÂ 2 C CCDCD"5CÄâ""½);
    }

    [Fact]
    public void Validator_ShouldPass_WhenObjectIsValid()
    {
        // Arrange
        var entity = new TestEntity(); // Id C, 7&V FVD B 7C ?Cä;CÔ0DäBD 0 C" :Cä=D BD CC=BCä@CR 1C 7Cä2D´E C

        // Act
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

        // Assert
        result.ShouldNotHaveAnyValidationErrors();
    }
}
</file>

<file path="Tests/Validator/ReferenceBaseValidatorTests.cs">

```

```

using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;
#
namespace Promatis.Net.ApplicationModel.Tests.Validator;
#
public class ReferenceBaseValidatorTests
{
    // B$D BCä2C 0 D CD"=CäAD$L C, 5E 2C ;C,,4C BCä@
    private class TestReference : ReferenceBase { }
    private class TestReferenceValidator : ReferenceBaseValidator<TestReference> { }

    private readonly TestReferenceValidator _validator = new();

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData("A")] // AÄ5CÖLD,,5 2 D 8CÄ2Cä;Cä2
    public void Name_ShouldHaveErrors_WhenInvalid(string? invalidName)
    {
        var model = new TestReference { Name = invalidName! };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name);
    }

    [Fact]
    public void Name_ShouldHaveError_WhenTooLong()
    {
        var model = new TestReference { Name = new string('A', 251) };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name)
            .WithErrorMessage("Aö@CT2D´HCT=C <C :D 8CÄ0C´LCÖ0Dò 4C´8CÖ0 CÖ0C,,<CT=Cä2C =C,,0 (250 D 8CÄ2Cä;C
    }

    [Theory]
    [InlineData("VALID_CODE_123")]
    [InlineData("REF1")]
    [InlineData("")] // B$5Cö5D L DÖBCä ?D >C"4CTB D4ACö5D,,=Cí
    [InlineData(null)] // A, MD$> D$>Cd5
    public void Code_ShouldBeValid_WhenCorrectFormat(string? code)
    {
        var model = new TestReference { Code = code };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldNotHaveValidationErrorFor(x => x.Code);
    }

    [Theory]
    [InlineData("C>CEö=C ö@D4AD :Cä<")]
    [InlineData("lower_case")]
    [InlineData("CODE-1")] // B$8D 5 CtöCö@CTICT=Cí
    public void Code_ShouldHaveError_WhenInvalidFormat(string invalidCode)
    {
        var model = new TestReference { Code = invalidCode };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Code);
    }

    [Fact]
    public async Task ValidateValue_ShouldReturnErrors_OnlyForSpecificProperty()
    {
        // Arrange
        var model = new TestReference
        {
            Name = "", // AäHC,,1Cö0
            Code = "INVALID-CODE" // AäHC,,1Cö0
        };

        // Act
        // Aö@Cä2CT@Dö5CÄ @C 1CäBD2 2C HCT3Cä 4CT;CT3C BC f Æ-F FUF ÇVR 4C´O Cö>C´O Name
        IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
            nameof(TestReference.Name));
        IEnumerable<string> codeErrors = await _validator.ValidateValue(model,
            nameof(TestReference.Code));
    }
}

```



```

    // Assert
    Assert.Contains(nameErrors, e => e.Contains("A0C,,<CT=Cä2C =C,,5 Cä1D07C BCT;DÄ=Câ""½
    Assert.Contains(codeErrors, e => e.Contains("A0CB <Cä6CTB D >CD5D 6C BDÄ BCä;DÄ:Câ ;C BC,,=C,,FD2""½
}

[Theory]
[InlineData(" Name")]
[InlineData("Name ")]
public void Name_ShouldHaveError_WhenHasLeadingOrTrailingSpaces(string invalidName)
{
    var model = new TestReference { Name = invalidName };
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);

    result.ShouldHaveValidationErrorFor(x => x.Name)
        .WithErrorMessage("A0C,,<CT=Cä2C =C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄAD0 8C´8 Ct0C0C0GC,,2C BDÄAD0 ?D >
}

[Fact]
public async Task ValidateValue_ShouldIgnoreOtherPropertiesErrors()
{
    // Arrange: Cä1C ?Cä;D0 =CT2C ;C,,4C0K0
    var model = new TestReference { Name = "A", Code = "low_case" };

    // Act: C$0C´8CD8D CCT< B$ A´,A0 Name
    IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
nameof(TestReference.Name));

    // Assert
    Assert.Single(nameErrors); // B$>C´LC0> CäHC,,1C00 CD;C,,=D² 8CÄ5C080
    Assert.DoesNotContain(nameErrors, e => e.Contains("A0CB""² 00 D,,8C >C0 :Cä4C 1D´BDÄ =CR 4Cä;Cd=Ci
}
}
</file>

<file path="Tests/Validator/ReferenceTreeBaseValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Validator;

public class ReferenceTreeBaseValidatorTests
{
    // B$5D BCä2C 0 D CD"=CäAD$L CD;D0 ?D >C$5D :C, 0C AD$@C :D$=Cä3Cä :C´0D AC
    private class TestNode : ReferenceTreeBase<TestNode> { }

    // B$5D BCä2D´9 C$0C´8CD0D$>D
    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

    private readonly TestNodeValidator _validator = new();

    [Fact]
    public void Validator_ShouldFail_WhenParentIdIsEqualToId()
    {
        // Arrange
        var nodeId = Guid.NewGuid();
        var node = new TestNode
        {
            Id = nodeId,
            ParentId = nodeId, // AäHC,,1C00: ID D >C$?C 4C 5D" A ParentId
            Name = "Self-referencing node"
        };

        // Act
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.ParentId)
            .WithErrorMessage("Aä1D05C0B C05 CÄ>Cd5D" 1D´BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");
}

[Fact]
public void Validator_ShouldFail_WhenParentIdIsEmptyGuid()
{
    // Arrange

```

```

var node = new TestNode()
{
    ParentId = Guid.Empty, // AäHC,,1Cð0 Cð> C$BCä@Cä<D2 ?D 0C$8C`Cð
    Name = "Empty ParentId"
};

// Act
TestValidationResult<TestNode>? result = _validator.TestValidate(node);

// Assert
result.ShouldHaveValidationErrorFor(x => x.ParentId)
    .WithErrorMessage("B >CD8D$5C`LD :C,,9 C,,4CT=D$8DD8Cð0D$>D =CR <Cä6CTB C KD$L CðCD BD`< GUID");
}

[Fact]
public void Validator_ShouldPass_WhenParentIdIsDifferent()
{
    // Arrange
    var node = new TestNode()
    {
        Id = Guid.NewGuid(),
        ParentId = Guid.NewGuid(),
        Name = "Valid child node"
    };

    // Act
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);

    // Assert
    result.ShouldNotHaveValidationErrorFor(x => x.ParentId);
}

[Fact]
public void Validator_ShouldPass_WhenParentIdIsNull()
{
    // Arrange (Að>D 5CÔL CD5D 5C$0)
    var node = new TestNode()
    {
        Id = Guid.NewGuid(),
        ParentId = null,
        Name = "Root node"
    };

    // Act
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);

    // Assert
    result.ShouldNotHaveValidationErrorFor(x => x.ParentId);
}
}
</file>
</files>

```